

重 庆 邮 电 大 学
CHONGQING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

硕士学位论文

MASTER THESIS



论文题目 基于载荷变异注入的 WAF
漏洞检测方法研究

学科专业 网络空间安全

学 号

作者姓名

指导教师

学 院

学校代码	10617	UDC	xxxxxx
分 类 号	xxxxxx	密级	

学 位 论 文

基于载荷变异注入的 WAF 漏洞检测方法研究

指导教师

申请学位级别	硕士	学科专业	网络空间安全
答辩委员会主席	某某某 教授	论文答辩日期	2021 年 5 月 20 日
学位授予单位和日期	重庆邮电大学		2021 年 6 月

Research on WAF Vulnerability Detection method Based on Payload Mutation Injection

A Master Thesis Submitted to
Chongqing University of Posts and Telecommunications

Discipline	Cyberspace Security
Student ID	
Author	
Supervisor	
School	

重庆邮电大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文中不包含其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在论文中以明确方式标明。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期： 年 月 日

重庆邮电大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☐ 公开论文

☐ 涉密论文，保密____年，过保密期后适用本授权书。

（请在以上方框内选择打“√”）

作者签名：

导师签名：

日期： 年 月 日

摘 要

Web 应用长期处于高强度对抗环境，SQL 注入、跨站脚本与命令执行等输入驱动型攻击在现实系统中仍然频繁出现。Web 应用防火墙（Web Application Firewall, WAF）作为常见的边界防护组件，通常依赖规则库与语法特征实现检测拦截；在云化与托管化趋势下，规则持续迭代且行为具有动态性，使黑盒评估只能获得拦截/放行与有限响应特征，传统随机变异或固定调度的测试方法往往收敛慢、成本高，难以稳定定位可放行且语义有效的绕过样本与规则缺口。

针对 Web 应用高强度对抗环境下 WAF 规则持续迭代与行为动态性带来的评估难题，基于随机变异或固定调度的黑盒测试方法存在收敛慢、成本高、难以稳定定位有效绕过样本与规则缺口的问题。本文以 WAF 的黑盒评估为研究对象，旨在在不依赖内部规则与源代码的条件下，提出一种基于载荷变异与启发式搜索的 WAF 漏洞自动化检测方法，构建可复现、可解释的检测链路，以在有限查询预算内稳定发现绕过路径并刻画潜在的规则缺口。

首先，基于上下文无关文法构建统一攻击语法规则库，并利用抽象语法树实现语义保留的节点级变异，构建结构合法且等价多样的候选载荷空间；其次，引入最小区分探测集与多信号响应指纹快速探测 WAF 行为建立动态规则映射，采用 Beta-Bernoulli 模型在线更新以获得稳定的拦截概率估计，并结合硬/软剪枝策略大幅压缩搜索空间；最后，将变异路径与注入上下文选择建模为序列决策问题，提出融合先验风险与经验回报的蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）调度策略，并设计漂移触发重探测与子树衰减机制，以自适应云 WAF 策略的动态变化。

在多类云 WAF 与开源 WAF 上针对 SQLi、XSS、CMDi 三类任务的评估结果表明，相比无先验树搜索、多种基线调度策略及同类工程工具，本文方法在有效绕过成功率、查询成本与收敛速度上均表现更优，且在规则漂移场景下能够灵敏捕捉防御策略变动，及时修正失效的拦截概率映射，有效消除非平稳环境导致的性能衰退，恢复对高价值绕过样本的稳定输出能力，证明了先验建模与搜索调度闭环的有效性及工程实践可行性。综上，本文为 WAF 防护能力评估与规则缺口挖掘提供了一条结构化、可扩展且可复现的黑盒检测路径，可为 WAF 规则调优与持续回归验证提供有力的技术支撑。

关键词：Web 应用防火墙，漏洞检测，对抗性载荷，蒙特卡洛树搜索

ABSTRACT

Web applications are constantly exposed to a highly adversarial environment, and input-driven attacks such as SQL injection, cross-site scripting, and command execution remain prevalent in real-world systems. Web Application Firewalls (WAFs), as common perimeter defense components, typically rely on rule databases and syntactic signatures to detect and block threats; Amid trends toward cloudification and managed services, rules undergo continuous iteration and exhibit dynamic behavior, limiting black-box evaluations to capturing only interception/allowance outcomes and limited response characteristics. Traditional testing methods relying on random variations or fixed schedules often suffer from slow convergence and high costs, making it difficult to reliably identify valid and semantically correct samples and rule gaps.

In the highly adversarial environment of web applications, where WAF rules undergo continuous iteration and behavior is dynamic, traditional black-box testing methods based on random mutation or fixed scheduling suffer from slow convergence, high costs, and difficulty in reliably identifying valid bypass samples and rule gaps. This thesis focuses on the black-box evaluation of WAFs, aiming to construct a reproducible and interpretable detection chain without relying on internal rules or source code, in order to reliably discover bypass paths and characterize potential rule gaps within a limited query budget.

To achieve these objectives, this thesis proposes an automated WAF vulnerability detection method based on payload mutation and heuristic search. First, a unified attack grammar rule library is constructed using context-free grammars, and node-level mutations that preserve semantic meaning are implemented via abstract syntax trees to build a candidate payload space that is structurally valid and semantically diverse; Second, we introduce the minimum partition detection set and multi-signal response fingerprints to rapidly detect WAF behavior and establish dynamic rule mappings. We employ the Beta-Bernoulli model for online updates to obtain stable estimates of interception probabilities, and combine hard/soft pruning strategies to significantly reduce the search space; Finally, we model the selection of mutation paths and injection contexts as a sequential decision-making problem. We propose a Monte Carlo Tree Search (MCTS) scheduling strategy that integrates prior risk and empirical returns, and design drift-triggered re-detection and subtree decay mechanisms to adapt to the dynamic changes in cloud WAF policies.

Evaluation results across multiple cloud WAFs and open-source WAFs for three attack types—SQLi, XSS, and CMDi—demonstrate that, compared to tree search without prior knowledge, various baseline scheduling strategies, and similar engineering tools, the method proposed in this thesis performs better in terms of effective bypass success rate, query cost, and convergence speed. Furthermore, in rule drift scenarios, it can more rapidly identify rule drift, reconstruct the security detection system, and restore its effectiveness, thereby demonstrating the validity of the closed-loop approach combining prior modeling and search scheduling, as well as its feasibility in engineering practice. In summary, this thesis provides a structured, scalable, and reproducible black-box testing approach for evaluating WAF protection capabilities and identifying rule gaps, offering robust technical support for WAF rule optimization and continuous regression testing.

Keywords: Web Application Firewall, Vulnerability Detection, Adversarial Payloads, Monte Carlo Tree Search

目 录

摘 要	I
ABSTRACT	II
图目录	VII
表目录	VIII
第 1 章 绪论	1
1.1 研究背景及意义	1
1.1.1 研究背景	1
1.1.2 研究意义	2
1.2 国内外研究现状	3
1.2.1 WAF 安全分析技术	3
1.2.2 自动化漏洞检测与模糊测试技术	4
1.2.3 对抗性攻击载荷生成技术	5
1.2.4 现有研究存在的问题分析	6
1.3 论文研究的主要内容	6
1.4 论文组织结构	7
第 2 章 相关理论基础	9
2.1 引言	9
2.2 Web 应用层攻击与防御	9
2.2.1 SQL 注入攻击	9
2.2.2 XSS 跨站脚本攻击	9
2.2.3 命令注入攻击	10
2.3 形式化语言与自动机理论	10
2.3.1 Chomsky 文法体系	10
2.3.2 上下文无关文法与巴科斯范式	11
2.4 抽象语法树与程序分析	12
2.5 蒙特卡洛树搜索算法基础	13
2.6 本章小结	14
第 3 章 基于语义保留的对抗性载荷空间构建策略	15
3.1 引言	15
3.2 统一攻击语法规则库设计	15
3.2.1 基于 BNF 的多攻击类型语法定义	15

3.2.2	SQL 注入文法规则设计	16
3.2.3	XSS 攻击文法规则设计	17
3.2.4	命令注入攻击文法规则设计	18
3.2.5	文法规则的可扩展性与演进机制	18
3.3	对抗性载荷解析与抽象语法树构建	19
3.3.1	词法分析与 AST 构建	20
3.4	语义保留的节点级变异操作	20
3.4.1	等价替换变异	21
3.4.2	多层次编码变异	22
3.4.3	注释与空白符干扰插入	24
3.5	基于规则探测的载荷空间剪枝优化	26
3.5.1	WAF 防护规则探测机制	26
3.5.2	规则映射表的构建	28
3.5.3	高风险节点与路径的剪枝策略	29
3.6	本章小结	33
第 4 章	基于蒙特卡洛树搜索的载荷空间优化搜索策略	34
4.1	引言	34
4.2	基于 MCTS 的载荷搜索空间优化算法设计	35
4.2.1	搜索树节点状态设计与属性定义	36
4.2.2	选择策略与 UCT 评估机制	37
4.2.3	扩展、模拟与回溯机制	40
4.3	动态自适应策略更新机制	43
4.3.1	实时反馈监控与成功载荷解析	43
4.3.2	节点风险等级动态评估	44
4.3.3	面向 WAF 行为变化的动态适应机制	45
4.4	本章小结	48
第 5 章	实验评估与结果分析	49
5.1	实验环境与数据集	49
5.2	评估指标与对比方法	50
5.2.1	评估指标	50
5.2.2	对比方法	50
5.3	统一规则库有效性验证	51
5.4	语义保留变异有效性验证	53

5.5 WAF 绕过测试	55
5.5.1 搜索/调度策略方法对比测试	55
5.5.2 同类研究方法与工程工具对比测试	56
5.5.3 典型绕过样本与规则缺口分析	57
5.6 调度策略性能对比分析	59
5.6.1 收敛速度对比	59
5.6.2 查询成本与早期有效性	60
5.6.3 概念漂移场景下的恢复能力	61
5.6.4 模块整体性能统计	62
5.6.5 参数敏感性分析	62
5.7 本章小结	64
第 6 章 工作总结和展望	65
6.1 工作总结	65
6.2 展望	65
参考文献	67
作者简介	73
1. 攻读学位期间的研究成果	73
1.1 申请（授权）专利	73
1.2 参与的科研项目及获奖	73
致 谢	74

图目录

图 1-1 总体技术路线图 Fig.1-1 Overall technology roadmap	8
图 3-1 Payload 语法解析树 Fig.3-1 Payload syntax parsing tree	20
图 3-2 基于规则探测的载荷空间剪枝优化方法 Fig.3-2 Rule-Based payload space pruning optimization method	33
图 4-1 基于 MCTS 的在线自适应载荷搜索流程 Fig.4-1 MCTS-Based online adaptive load search process	35
图 5-1 探测集规模 $ D $ 与映射质量 (F1) 关系 Fig.5-1 Relationship between detection set size $ D $ and mapping quality (F1)	53
图 5-2 不同方法在固定预算下的收敛曲线 ($B = 2000$) Fig.5-2 Convergence curves of different methods under a fixed budget ($B=2000$)	60
图 5-3 首次有效成功查询成本分布 Fig.5-3 First successful query of cost distribution	61
图 5-4 概念漂移场景下的恢复能力对比 Fig.5-4 Comparison of recovery capability in con- cept drift scenarios	61
图 5-5 超参数 (c, β) 对 SR 的敏感性分析 Fig.5-5 Sensitivity Analysis of SR to Hyperpa- rameters (c, β)	63
图 5-6 超参数 (c, β) 对 QC_{first} 的敏感性分析 Fig.5-6 Sensitivity Analysis of Hyperparam- eters (c, β) on QC_{first}	63
图 5-7 超参数 (c, β) 对 T_{rec} 的敏感性分析 Fig.5-7 Sensitivity Analysis of Hyperparameters (c, β) on T_{rec}	64

表目录

表 3-1 顶层规则 Table3-1 Top-Level Rules	16
表 3-2 逻辑注入与布尔盲注文法 Table3-2 Logic injection and boolean blind injection syntax	16
表 3-3 联合查询文法 Table3-3 Joint query grammar	17
表 3-4 字符串连接文法实例 Table3-4 String concatenation syntax examples	17
表 3-5 盲注延时函数文法实例 Table3-5 Blind delay function syntax example	17
表 3-6 XSS 攻击文法 Table3-6 XSS attack syntax	18
表 3-7 命令注入攻击文法 Table3-7 Command injection attack syntax	18
表 3-8 ssti 攻击文法 Table3-8 SSTI attack syntax	19
表 3-9 顶层规则更新 Table3-9 Top-level rule update	19
表 3-10 应用层、协议层编码矩阵	22
表 3-11 Log4j2 远程代码执行漏洞文法 Table3-11 Log4j2 remote code execution vulnerability syntax	25
表 5-1 统一规则库有效性 Table5-1 Validity of the unified rule repository	52
表 5-2 规则探测/映射/剪枝链路消融对比 ($B = 2000$) Table5-2 rule detection/mapping/pruning vs. link ablation comparison ($B=2000$)	53
表 5-3 语义保留变异有效性 Table5-3 Semantic preservation variation validity	54
表 5-4 语义保留变异算子消融对比 ($B = 2000$) Table5-4 Semantic preservation variation operator ablation comparison ($B=2000$)	54
表 5-5 单轮绕过成功率 (%) Table5-5 Success rate per round of bypassing (%)	56
表 5-6 不同 WAF 与攻击类型下的有效绕过成功率 (SR) Table5-6 Effective bypass (SR) across different WAFs and attack types	57
表 5-7 不同方法总体性能统计 Table5-7 Overall performance statistics for different methods	62
表 5-8 UCT' 超参数最优组合 Table5-8 UCT' Optimal Hyperparameter Combination	62

第 1 章 绪论

1.1 研究背景及意义

1.1.1 研究背景

Web 应用承载了政务、金融、教育与电商等关键业务的交互入口，其攻击面往往由复杂输入、多方组件与快速迭代共同形成。近年来，云原生架构与 API 化将大量业务逻辑暴露在 HTTP/HTTPS 之上，使攻击者能够绕过传统边界，直接应用层构造高维度输入以触发解析、拼接与执行链路中的脆弱点。OWASP Top 10 (2025) 显示^[1]，注入类缺陷，如 SQL 注入、命令注入以及跨站脚本，鉴权与访问控制相关问题仍是 Web 领域最重要的风险来源，并在解释中强调漏洞往往表现为跨组件/跨边界的复合路径。这一结构性变化说明 Web 风险已不再是单点缺陷主导，而是输入、身份认证、配置安全与供应链攻击相互叠加的结果^[1]。在 API 侧，OWASP API Security Top 10 (2025)^[2]进一步指出对象级授权、鉴权绕过、资源消耗与安全配置缺陷等问题在真实系统中具有高发生率，且经常与传统注入/脚本类漏洞形成联动，例如，先通过权限缺口扩大攻击面，再通过注入/反序列化触发后端执行。

从攻击输入的形态看，Web 请求并非单一字符串，而是由 URL 路径、查询参数、表单体、JSON/XML、Cookie、多部件上传、甚至头部字段共同组成的多通道载荷。在真实流量中，同一语义可以存在多种等价表达，字符编码、多层转义、空白符与注释插入、大小写/同义替换、结构重排等都能改变表面模式而保持后端解释一致。这类语义保持的变形使基于规则匹配与特征工程的防护更易出现边界缺口^[3]，也让攻击更接近高维输入空间的搜索问题^[4]。同时，现代 Web 框架链路中经常存在多重解析与归一化^[5]，边缘代理、WAF 引擎、应用框架、对象关系映射/模板引擎、数据库驱动各自持有解析器与解码规则。解析差异一旦被利用，攻击者可构造前置组件认为无害、后端组件解释为恶意的输入，形成绕过与语义穿透。

面向云环境的 WAF 还具有明显的工程特征，其策略往往以托管规则/行为检测/挑战与限速组合交付，规则组会随时间滚动升级，黑盒评估的同输入同结果假设容易被打破。研究者在 WAFManis^[6]的系统研究中指出，WAF 与后端 HTTP 解析器之间的协议级差异会引入可迁移的绕过面，并通过自动化方法在多种主流 WAF 与 Web 框架上发现大量协议层规避案例。WAFFLED^[7]进一步围绕 content-type 与请求分段等细节，利用解析分歧在多种云 WAF 上确认了大规模绕过样本，并给出了可落地的缓解思路，如 HTTP 规范化代理。这些工作共同表明在云化、组件化

与多解析器并存的现实条件下，WAF 评估不再是测几条 payload 看拦不拦的简单问题，而需要能够在弱反馈条件下稳定定位规则边界与解析敏感分支的系统方法。因此，如何在不依赖 WAF 内部规则与源代码的条件下，构造具有语义一致性且可系统搜索的对抗性载荷，并以有限预算稳定定位 WAF 防护缺陷与策略缺口，成为当前 Web 应用安全测试与防护评估中的关键问题。

另外，WAF 的拦截链路本身也可能暴露可观测的侧信道^[8,9]。许多规则集依赖复杂正则表达式、多阶段解码与归一化^[10]，某些输入会触发显著的匹配开销上升并体现在响应延迟或连接行为上。对于黑盒测试而言，这类延迟信号并不用于拒绝服务攻击本身，而更像是规则路径复杂度的外显指标。当同类 token—上下文组合对应的处理耗时呈现统计显著抬升时，往往意味着该分支在规则匹配或解码链上更敏感、更值得投入后续变异预算。以开源生态为例，ModSecurity^[11] 及其兼容实现常配合核心规则集 (OWASP Core Rule Set, CRS)^[12] 运行，CRS 采用多阶段处理与异常评分机制，规则命中会累加不同严重度的异常分数，当总分超过阈值时触发拦截或挑战，从而在覆盖面与误报之间取得折中。CRS 还提供偏执等级 (Paranoia Level, PL) 分级，允许在更严格的检测与更高的误报风险之间调节，这一机制在 CRS 官方文档与云厂商托管规则说明中均有明确描述，同时，MK Anuvarshini 等人^[13] 也对 CRS 进行了评估与增强的实证研究，分析了 OWASP CRS 在不同配置下对性能指标的影响。

在云托管场景下，Managed Rules 的持续更新使防护能力随时间演进，但也引入评估视角的漂移问题^[14]，同一业务在不同时间窗口的拦截行为可能变化，黑盒评估结果存在失效风险。托管规则与挑战/限速策略还可能引入非规则因素的干扰，例如拦截页重定向、连接复位、验证码挑战等会让单纯依赖 HTTP 状态码的判定产生误差。

综上所述表明，攻击者可利用不同组件对同一请求的解析分歧构造可通过 WAF 但能在后端触发攻击语义的输入。也有工作从底层协议栈与实现细节出发系统分析 WAF 的可绕过面，如 WAFManis^[6] 与 WAFFLED^[7] 分别从处理链路与解析差异视角给出系统性证据。除检测精度问题外，WAF 在性能层面同样暴露出可观测的侧信道，部分规则依赖复杂正则表达式或多轮解码与归一化，可能在特定输入下触发显著的计算开销放大。对于黑盒测试而言，这类延迟信号既可能反映规则路径的复杂度，也可能提示值得深入探索的敏感解析分支。

1.1.2 研究意义

面向 WAF 的黑盒评估长期存在两类硬约束。一类来自输入空间，攻击载荷空间巨大且高度结构化，随机变异很容易产生无效样本，导致预算消耗快、收敛慢；

另一类来自反馈空间，多数情况下只能观察到放行/拦截与有限的响应侧信道，云 WAF 的限速、挑战与规则更新又会引入噪声与漂移，使得风险估计不稳定。现有实践里，很多评估依赖经验技巧与少量载荷，难以对规则边界与绕过机理给出可复现、可解释的证据链。

因此，开展面向 WAF 的结构化黑盒评估研究具有重要的理论与工程价值。在方法层面，针对弱反馈与大规模输入空间并存的特点，将攻击语义空间进行结构化建模，并结合统计决策机制实现有效搜索，为输入驱动型安全测试提供一种从经验驱动向数据驱动与模型驱动转变的技术路径，有助于提升评估过程的稳定性与可重复性。在工程实践中，通过在有限请求预算内提高有效样本比例，可降低测试成本并提高绕过路径发现效率，从而支持对不同类型 WAF 的一致性评估。在应用层面，基于测试过程中获得的绕过样本及其响应特征，可以为规则配置优化、误报与漏报分析提供依据，进而为防护策略的持续调整提供参考。

1.2 国内外研究现状

1.2.1 WAF 安全分析技术

针对 WAF 的安全分析研究可以概括为三条主线，分别是规则与引擎机制理解、绕过机理挖掘、黑盒评估与测试体系。规则与引擎机制理解侧重于从规则集、引擎实现与配置参数出发解释拦截原因与误报来源。以 OWASP Core Rule Set^[12] 为代表的公开规则集给出了异常评分、执行阶段与日志字段的规范化描述，使研究者能够在可控环境中复现命中路径、分析误报根因并讨论阈值配置对拦截效果的影响。云厂商在托管 CRS 或自研规则时也会暴露类似概念，例如 PL 分级与误报权衡，从工程侧印证了规则强度与可用性之间的系统性联系。这一类研究^[17] 为如何定义更稳健的黑盒判定信号提供了基础，当 WAF 的决策不是单条规则拦截而是评分与策略组合时，测试方法更需要面向行为模式而非单一状态码。面向云场景，托管 WAF 往往以托管规则组、行为检测与挑战/限速的组合提供能力。厂商文档通常明确规则会定期升级并提供变更记录，这意味着同一载荷在不同时间窗口的行为可能不同。从评估角度看，这类时间相关变化可被视为概念漂移，需要通过重探测或权重衰减等机制保持模型有效。

绕过机理挖掘近年出现了更强的系统性。一类工作从协议与实现层面对 WAF 的通用弱点进行归纳，刻画请求规范化、解码与过滤链路中可能产生的差异。另一类工作聚焦解析分歧，利用边缘层与后端对同一输入的不同解释制造语义穿透，从而在放行条件下触发后端执行。WAFManis^[6] 从协议层规避出发，强调请求解析差异会让 WAF 与后端对同一请求产生不一致理解，并提出自动化发现这类漏洞的方

法,在多种主流 WAF 与 Web 框架上验证了其普遍性,WAFFLED^[7]则进一步将解析分歧扩展到 JSON、multipart、XML 等常见 content-type,报告了跨多家云 WAF 的大量绕过样本,并提出 HTTP 规范化代理用于缓解。这类研究的共同点在于绕过不再依赖少量的技巧 payload,而是揭示了输入空间中可系统挖掘的结构性缺口。

除传统规则型 WAF 外,学习型^[18]或混合型^[19]WAF 也逐渐出现。相关研究尝试引入对抗学习增强规则引擎对新型变形的泛化能力,表明了检测器与攻击者之间会形成动态对抗。虽然学习型 WAF 能在某些场景提升泛化,但其对抗鲁棒性仍受到可保持语义的输入变形挑战。这一趋势强调了评估方法的两个需求,一是输入生成要具备语义保持与结构化可控,二是反馈建模要能应对策略更新与噪声干扰。因此,有必要通过使用语义保持变异、稳健概率估计与动态搜索的组合实现对复杂 WAF 系统的自动化安全评估与防御边界探测。

在黑盒评估方面,现有研究普遍面临弱反馈与不可复现的问题。一方面,黑盒测试通常只能依赖有限的外部观测信号,如用状态码、响应长度或延迟等来推断 WAF 的决策结果,这种低信息密度反馈使得评估过程对噪声高度敏感,进而影响测试结论的稳定性与一致性。另一方面,在实际部署环境中,云 WAF 往往引入多种动态防护机制,如挑战页、重定向、速率限制及连接级阻断等,这些机制并不直接对应规则命中,而是作为流量控制或抗自动化手段存在,使得拦截行为的语义更加复杂。具体而言,挑战机制与速率限制本质上属于抗自动化与资源保护策略,其触发条件通常依赖于请求频率、行为模式或风险评分,而非单一载荷特征。同时此类机制既可能被自动化技术绕过,也可能对正常请求产生干扰,从而进一步模糊攻击与正常行为之间的判定边界。在此背景下,若仅依赖 403/200 等单一信号进行判定,容易将限速或挑战误判为规则命中,或将偶发放行误判为安全缺口,进而降低评估结果的可信度与可复现性。

1.2.2 自动化漏洞检测与模糊测试技术

自动化漏洞检测与模糊测试(fuzzing)^[20]通过持续生成输入并观测异常行为来发现缺陷,是现代漏洞挖掘的重要方法论。近年来,模糊测试技术路线在系统性总结与工程化方面发展迅速,Manès 等^[21]在 fuzzing 综述中对覆盖引导、语法引导、符号执行与混合策略进行了系统归纳,并指出输入结构约束与反馈信号设计是决定有效性的关键因素。对服务端 Web 应用而言,输入空间由多参数、多编码与状态相关交互共同构成,反馈多为远程弱信号,使得传统灰盒 fuzzing 中常用的覆盖率导向难以直接适用。Dharmaadi 等^[22]综述系统梳理了服务器端 Web fuzzing 框架的关键技术路线,包括参数推断、请求序列建模、状态恢复与轻量化反馈利用,并指出状态空间爆炸与有效变异器缺失仍是主要瓶颈。

为了提高有效输入比例，语法/文法驱动 fuzzing 将输入约束在合法语法集合中，通过语法导向的变异与组合覆盖更丰富的结构路径。在 WAF 场景中，输入不仅要语法可解析，还要能在后端触发攻击语义，因此载荷空间更需要显式语义约束与上下文约束。随着大语言模型 (Large Language Model, LLM) 能力普及，LLM 辅助 fuzzing^[23] 进一步降低了手工语法与变异器的构建成本。Fuzz4All^[24] 表明 LLM 可以在多种语言场景下生成与变异输入并提升覆盖率。不过，在 WAF 绕过与漏洞触发的组合目标下，单纯的生成可解析输入仍不足够，输入需要同时满足后端语义触发与边界拦截规避，这要求对语义保持、编码链与上下文约束进行显式建模。

在反馈驱动的调度策略方面，遗传算法、强化学习与树搜索等方法被用于解决候选输入数目巨大时的预算分配问题。以 Alphuzz^[25] 为例，其在种子与变异树上引入蒙特卡洛搜索思想，以更系统的方式平衡探索与利用并加速收敛。

现实部署环境中，一旦目标前置 WAF，扫描器与 fuzzing 请求很可能在边界被拦截，导致测试信号无法触达后端因此，有必要通过将 fuzzing 的输入生成能力与对抗性变形结合，通过探测与剪枝先获得可解释的先验，再以结构化搜索完成绕过与语义触发的联合优化。

1.2.3 对抗性攻击载荷生成技术

对抗性攻击载荷生成的核心思想是在是在保持输入语义或目标功能完全不变的前提下，通过对输入样本施加受约束的微小变形，诱导底层检测器产生错误的分类判断^[26]。这一思想与 WAF 的绕过机制具有天然的对应关系。由于传统 WAF 及部分学习型检测引擎高度依赖固化的正则表达式、特征签名或有限的上下文模式，攻击者往往能够利用前后端组件之间的解析阻抗失配，通过多重编码链嵌套、内联注释插入、等价函数替换以及语法重排等手段，对恶意载荷的表面形态进行复杂的混淆。这种变形使得载荷在经过 WAF 的检测引擎时呈现为安全或未知的模式，但在到达最终的后端数据库或浏览器解释器时，依然能够被正确还原并执行原有逻辑，从而在不改变后端解释结果的情况下成功规避拦截。

近期研究^[28] 尝试引入强化学习或生成对抗网络来自动化探索 WAF 的检测盲区，或利用对抗学习提升规则引擎对新型变异的泛化能力。然而，现有的诸多自动化生成框架大多将 Web 载荷视为扁平的字符串序列，主要依赖字符级或词级别的随机扰动，如随机插入、拼接或字符翻转。这种非结构化的变异虽然增加了样本的表面多样性，但在实际对抗中极易破坏 SQL 语句或跨站脚本严格的语法框架，导致生成的载荷不仅产生大量的无效测试查询，如产生 500 内部错误等，也大大降低了搜索算法的收敛效率。

与图像或文本对抗样本不同，Web 载荷的语义保持必须同时满足后端解析、执

行链路与上下文约束，因此变形过程需要上下文感知能力。将载荷抽象为由语法规则生成的树结构，并在树上施加语义保持的节点级变异，有助于将对抗生成从经验技巧提升为可分析、可复现的系统方法。

1.2.4 现有研究存在的问题分析

现有研究虽然在 WAF 绕过机理、Web fuzzing 与对抗样本生成方面取得进展，但针对黑盒 WAF 漏洞检测的完整链路仍存在若干空缺。主要集中在语义建模、弱反馈建模与搜索收敛三方面。在语义建模方面，许多绕过方法强调技巧性变形或特定解析差异，缺乏统一的语义建模与可扩展的多攻击类型载荷空间，多以字符级、协议层的经验式混淆为主，依赖人工总结的绕过模式，难以在 SQL 注入、XSS 与命令执行等多类型攻击间共享结构化方法。在弱反馈建模方面，黑盒反馈常被简化为单次硬判定，未充分考虑真实云环境下的动态响应特性，忽略云 WAF 限速、挑战页与偶发误判带来的噪声，导致风险估计不稳定，剪枝与调度难以长期有效。在搜索收敛方面，搜索策略多停留在随机、贪心或遗传等启发式层面，对高维、结构化的变异空间适配性不足，难以在指数级结构空间内兼顾探索与利用，尤其在规则更新导致行为漂移时，搜索容易陷入局部最优。

1.3 论文研究的主要内容

本文围绕基于载荷变异注入的 WAF 漏洞检测方法展开，目标是在黑盒条件下用可复现、可解释的方式发现 WAF 的规则缺口与绕过路径，并尽可能减少试探式枚举带来的预算浪费。整体技术路线以构建结构化载荷空间、反馈先验建模与统计决策搜索为主干，并对其进行可复现实验评估，形成从输入生成到证据输出的闭环。为突出题目中的载荷变异注入，本文将生成流程显式拆分为变异与注入两个环节。第三章聚焦于变异，统一语法规则库与节点级语义保持变异，并通过探测、映射与剪枝得到可用先验。第四章聚焦于注入的执行与调度，在给定预算下决定注入方式，并在反馈回流中更新搜索树统计量，从而实现对有效样本的快速收敛与对云 WAF 漂移的在线适应。最终以变异、注入、反馈与更新的方式稳定逼近有效样本。

第三章围绕 SQL 注入、XSS 与命令注入等典型 Web 攻击场景，构建了统一的多攻击类型上下文无关文法规则库，将载荷解析为抽象语法树 (Abstract Syntax Tree, AST)。这一转变使载荷变异从字符级扰动提升为语义保持的节点级操作，如等价替换、多层编码等，既能维持攻击语义，又能在离散结构空间中系统探索规则边界，并显式记录变异路径以保证可解释性与可复现性。在载荷空间规模随文法展开呈指数增长的现实条件下，进一步引入先验获取与空间压缩的前置环节，避

免将预算消耗在明显无效的区域。在黑盒条件下，设计最小区分探测集 D ，利用多信号响应指纹 y 稳健刻画 WAF 的真实处置行为，降低限速与偶发异常的干扰。随后，引入 Beta-Bernoulli^[27] 在线更新机制，将拦截概率作为可学习量进行持续修正，从而构建动态映射表 M 。基于 M 提供的拦截倾向风险估计，在 AST 生成阶段分别执行硬剪枝以源头压缩空间，以及执行软剪枝保留高风险区域的探索延后性。最终，软剪枝保留的先验权重能够无缝对接后续的搜索策略，将结构空间的探索转化为带有可用偏置的系统化进程。

第四章围绕如何在巨大候选空间内合理分配有限查询预算建立优化搜索策略。本文引入蒙特卡洛树搜索（MCTS）对载荷空间进行在线规划，将产生式选择、节点变异、编码策略与上下文拼接等决策序列组织为搜索树，利用上置信界的探索与利用机制在全局范围内逐步逼近有效路径。映射表 M 与剪枝得到的权重被显式融入选择策略，使搜索从一开始就具备对高潜力结构的偏好，同时又保留必要探索以发现规则边界的薄弱点。考虑到云 WAF 的规则可能随时间漂移，进一步引入漂移检测与子树衰减机制，当行为分布发生显著变化时，通过对子树统计量施加衰减、触发局部重探测与更新先验，使搜索能够在不完全重启的情况下恢复有效性，保持评估结论在时间维度上的可用性与可复现性。

第五章围绕可复现评估构建实验体系，验证上述方法链路的有效性与必要性。实验部分搭建可控的测试环境与对比基线，围绕统一规则库覆盖能力、语义保持变异的有效性、WAF 绕过成功率与查询成本、收敛速度与漂移恢复时间等指标开展系统评估，并通过消融实验分别去除探测集 D 、贝叶斯在线更新、硬/软剪枝、漂移自适应等关键模块，量化各模块对整体性能的贡献。该评估设计说明了的本方法能否在有限预算内更快产生可放行且语义触发的有效样本，以及当 WAF 行为随时间变化时，本方法能否保持稳定收敛并给出可解释的绕过证据链。整体技术路线图如图1-1所示。

1.4 论文组织结构

全文共六章。第1章介绍研究背景、意义与国内外研究现状，明确本文的研究问题与技术路线。第2章给出 Web 攻击与防御基础、形式化语言与抽象语法树相关理论，并引入蒙特卡洛树搜索的基本概念。第3章提出基于语义保留的对抗性载荷空间构建策略，并给出规则探测与剪枝优化方法。第4章设计基于 MCTS 的载荷空间优化搜索策略与动态自适应机制。第5章给出实验评估与结果分析。第6章总结全文工作并展望后续研究方向。

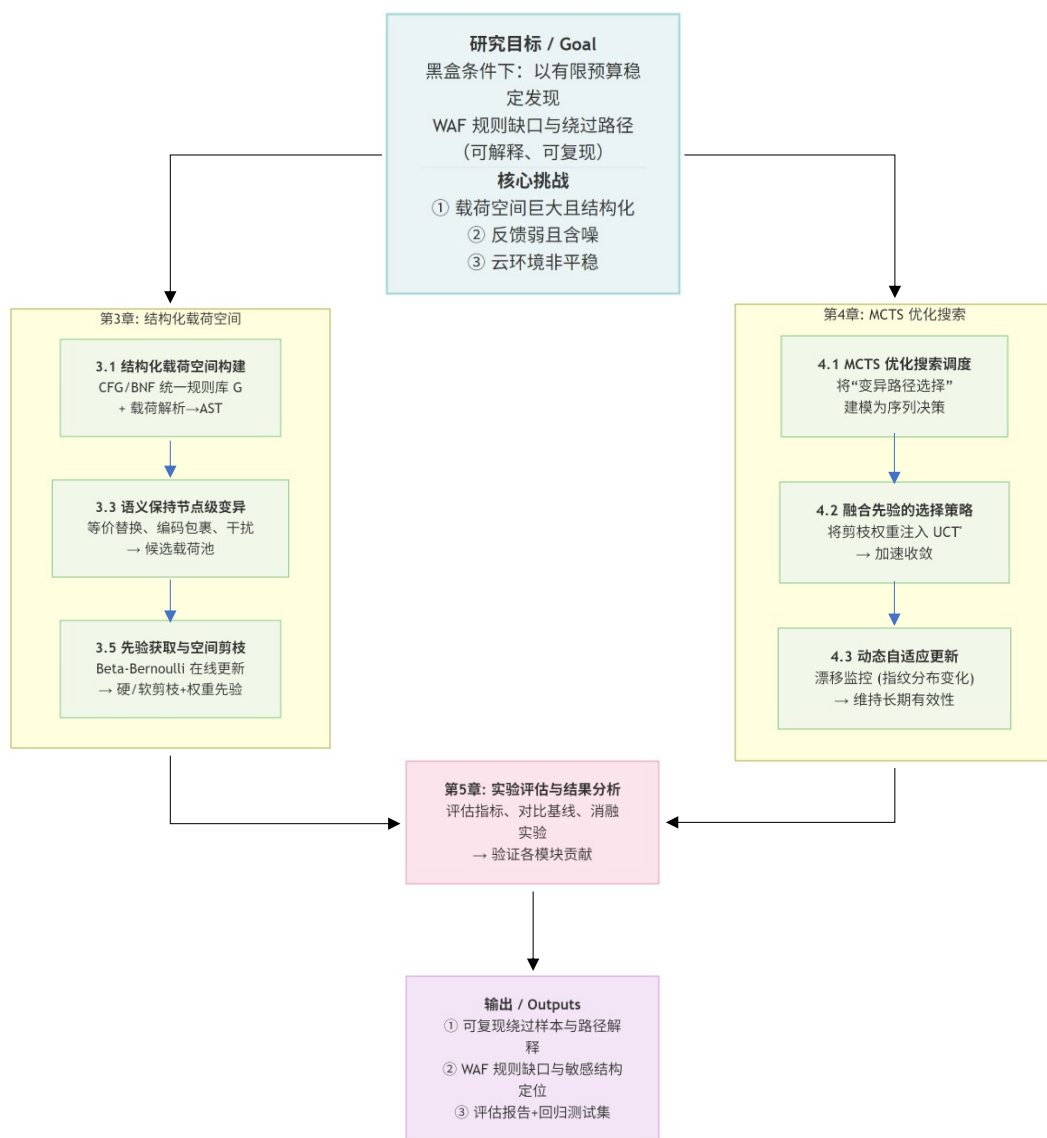


图 1-1 总体技术路线图
Fig.1-1 Overall technology roadmap

第 2 章 相关理论基础

2.1 引言

本章从 Web 攻击与防御机理、CFG/BNF、AST 结构化变换, 以及 MCTS 统计决策框架四个层面, 围绕攻击语义建模、语法解析与搜索优化等关键技术展开, 为后续方法论给出必要支撑。

2.2 Web 应用层攻击与防御

Web 攻击与防御的关键难点在于输入并不是单一字符串, 而是由 URL/Query、表单体、JSON/XML、Cookie、Header、multipart 分段以及多层编码共同构成。同一段字面输入在不同解析器、不同解码顺序、不同上下文下可能对应完全不同的语义^[28]。近年来关于 WAF 绕过的系统性研究显示^[29,30], 协议与解析差异本身就能形成可迁移的绕过面, 边界设备与后端框架对请求结构、content-type、参数组织方式的解释不一致, 会导致 WAF 看到的安全请求在后端变成可触发语义的攻击请求。这类现象在针对 WAF 的自动化发现研究^[31,32]中被大量证实。

2.2.1 SQL 注入攻击

SQL 注入 (SQL Injection, SQLi)^[33] 攻击者将恶意 SQL 片段注入到应用构造的查询语句中, 借由后端数据库的解析与执行实现越权读取、数据篡改或系统命令执行等目的工程上常见的 SQLi 变体包括依赖错误信息的回显注入、通过布尔条件构造的盲注、以及以延迟函数为代表的盲注^[34]。这些形态并不要求载荷在字面层面像攻击, 只要能够在数据库解析层形成等价语义即可^[35]。

在防御层面, 参数化查询与预编译语句被视为最可靠的工程手段^[36], 输入校验与最小权限配置可进一步降低攻击面。OWASP 的 SQL 注入防护建议强调, 避免拼接 SQL、使用参数化接口、对动态查询片段采用严格白名单约束是核心原则^[37]。

2.2.2 XSS 跨站脚本攻击

跨站脚本 (Cross-Site Scripting, XSS)^[38] 利用应用在输出位置未对不可信输入进行恰当编码或过滤的缺陷, 将脚本代码注入到页面中并在受害者浏览器侧执行。XSS 既可以发生在 HTML 文本上下文, 也可能出现在属性值、URL、CSS、JavaScript 字符串或 DOM API 的不同上下文中。不同上下文对字符转义的要求差异显著, 因此 XSS 的语义保持变形往往体现为标签/事件处理器组合、编码链与分隔符逃逸的多样化^[40]。

在防御实践中^[39]，输出编码与上下文敏感的转义规则是首要手段，配合 CSP、HttpOnly/SameSite Cookie 等机制可降低攻击成功概率。OWASP 的 XSS 防护建议强调必须区分输出上下文并采取对应编码策略^[41]。WAF 对 XSS 的检测通常依赖标签、事件、协议与脚本片段的特征规则，但对多层编码、大小写与分隔符扰动、以及浏览器容错解析带来的等价表达仍难以完全覆盖^[42]。

2.2.3 命令注入攻击

命令注入 (Command Injection, CMDi)^[43] 通常发生在应用将外部输入拼接进系统命令并通过 shell 执行的场景。攻击者通过插入命令分隔符、管道符、重定向符或子命令替换等方式改变原始命令的执行结构，进而执行任意系统指令。

在防御实践中，避免直接调用 shell、使用语言内置 API、对参数采用严格白名单与安全转义是主流建议。OWASP 的权威建议明确指出，最可靠的策略是避免直接调用 OS 命令，优先使用语言内置库或安全 API。若必须调用，应将参数限制在白名单集合并采用安全转义。WAF 对命令执行类载荷的识别往往聚焦分隔符、敏感命令关键字与危险函数模式，但当攻击语义通过编码与结构重排隐藏在看似无害的参数中时，仍存在被放行的可能^[44]。

2.3 形式化语言与自动机理论

2.3.1 Chomsky 文法体系

形式化语言用于刻画哪些字符串是合法输入，其核心价值在于把输入空间从不可控的字符组合，提升为可推导、可约束、可分析的结构集合。Chomsky 文法体系^[45]按生成能力将文法划分为 0-3 型，0 型为短语结构文法、1 型为上下文有关文法、2 型为上下文无关文法 (Context-Free Grammar, CFG)、3 型为正则文法。上下文无关文法^[46]能够刻画大量具有递归结构的语言。它可以表达递归结构、嵌套结构与可选分支，覆盖了大量协议与编程语言片段的核心语法特征。将攻击载荷建模为形式语言并不等价于只生成合法输入，其真正价值在于把载荷的结构约束显式化，使后续的变异、剪枝与搜索都能围绕结构语义展开，而不是在字符串空间盲目试错。对安全测试而言，越强的表达能力往往意味着更高的构造与解析成本；越弱则难以覆盖真实输入中的嵌套结构、组合分支与递归模式。本文选择 CFG 的原因是它既能表达攻击载荷中常见的递归结构，如 SQL 表达式嵌套、XSS 标签与属性组合、命令片段的连接与重定向，又能用成熟的解析算法在工程上稳定落地。与纯字符串变异相比，这种结构化描述的优势有合法性可控、变异位置可定位、语义保持更容易被定义为树上的等价重写。但是，Web 载荷并不等同于一门完整编

程语言，它处在 HTTP 参数、服务端解析、应用拼接与后端解释器的参数传递链路上，语义由多方组件共同决定。也正因如此，安全测试中常见的有效载荷往往具备可解析但不一定规范与语义依赖上下文的特征。使用 CFG 建模的意义并不是追求形式化完备，而是把最关键的结构骨架如 token 组合、嵌套层次与上下文分支等固定下来，让后续搜索把预算集中在更可能改变 WAF 判定边界，但不破坏后端解释的变形上。这一思路与近年面向安全的语法/文法驱动 fuzzing 研究方向一致，通过结构约束提升深层路径覆盖与有效样本比例。以 Gramatron^[48] 为例，其讨论了 CFG 在安全测试中的典型角色，并给出将文法转换为自动机以改善采样偏置与提升变异效率的系统方法。

2.3.2 上下文无关文法与巴科斯范式

上下文无关文法 (CFG) 是形式语言理论中的核心概念，广泛用于精确描述程序设计语言及各类结构化数据的语法规则。在数学形式上，CFG 通常被定义为一个四元组 $G = (V, \Sigma, R, S)$ ^[49]。其中， V 代表非终结符的有限集合，表示可进一步推导与展开的语法变量； Σ 代表终结符的有限集合，表示语言中不可再分的最小基础符号，如具体的字符或关键字，且满足 $V \cap \Sigma = \emptyset$ ； R 表示产生式规则的有限集合，其形式通常为 $A \rightarrow \alpha$ ，其中 $A \in V$ 且 $\alpha \in (V \cup \Sigma)^*$ ，即规则左侧必须是单一的非终结符，右侧是由终结符和非终结符组成的混合序列； S 为起始符号，满足 $S \in V$ 。所谓上下文无关，是指产生式左侧的非终结符 A 的替换规则完全不受其所处上下文环境的限制。这一独立推导特性，使得 CFG 极为适合用于构建具有层次化和嵌套特征的抽象语法树 (AST)。

巴科斯范式 (Backus-Naur Form, BNF)^[50] 是对 CFG 产生式的一种工程化记法，常用于编程语言与协议的语法描述。在 BNF 表达下，非终结符表示可展开的结构单元，终结符对应最终输出的具体 token。例如可以将 SQLi、XSS 与命令执行载荷分别抽象为若干非终结符，如 `<payload>`、`<expr>`、`<tag>`、`<op>` 等，并通过产生式组合形成统一规则库。这本质上是在建立一套可枚举、可解析、可回放的载荷生成协议，任一 payload 都应当能映射到一棵推导树，从而使变异从字符级扰动变成结构级操作。当 payload 被解析为树结构后，变异可以被定位到某个节点、某条路径或某个上下文分支。比如 SQLi 中 UNION SELECT 与 OR 1=1 属于不同的语义骨架分支，XSS 中 tag/attribute/event/url-scheme 属于不同上下文敏感组合，命令注入中 separator/pipe/substitution/redirection 属于不同执行结构。把这些差异显式写进产生式，等价于把攻击经验固化为结构化空间，使搜索不再在无意义字符串海里碰运气，而是在语义可控的分支树里做预算分配^[51]。

WAF 绕过里最常见的技巧之一是编码链，如 URL/HTML/JS/Unicode/双重编码

等。在 CFG 中可把编码器建模为可组合的产生式，如 $\langle \text{enc} \rangle ::= \text{url} \mid \text{html} \mid \text{js} \mid \text{unicode} \mid \dots$ ，并允许嵌套，这样多层级编码变异就成为树上的结构替换或子树包裹，而不是对字符串做不可追踪的替换。WAFFLED^[7] 在研究 WAF 绕过时，也强调了对复杂 content-type 与解析差异的系统化生成与变异，其中就包含以预定义文法生成结构化 HTTP 请求，再在非攻击片段处做可控变异的思路^[52]。进一步从实现层面来看，BNF 便于将语法规则自动转换为解析器或生成器，给定输入字符串，可通过解析获得推导树。给定推导树，可按终结符序列反向生成字符串。双向映射是本文语义保留变异注入方法的关键前置，载荷先被解析为树，再在树上施加变异，最后再线性化为可发送的 HTTP 参数字符串。通过将变异约束在合法推导路径上，能够显著降低无效输入比例。

2.4 抽象语法树与程序分析

抽象语法树 (AST) 是对语法结构的抽象表示，通常省略与语义无关的冗余符号，仅保留构造与层级关系。在编译原理与程序分析中，AST 是语义分析、优化与代码生成的基础数据结构^[53]。与完整的解析树相比，AST 更适合进行结构化变换与等价重写，因此也成为本文载荷变异的核心表示。把这一思想迁移到 WAF 绕过的载荷生成上，AST 的价值主要体现在两点，把语义保持具体化为树上的等价变换和让变异注入具备可定位性与可解释性。

以 SQLi 为例，WAF 常见规则关注关键字与运算符序列，但后端数据库关注的是解析后的查询结构。若把 payload 仅当作字符串，很多看似微小的字符变动其实会破坏语法，使得输入根本无法触达后端解释器。相反，某些表面差异巨大的变形却可能解析为同一棵或等价的语法树。AST 的引入可以把保持攻击语义界定为变异前后 AST 在关键语义节点，例如谓词逻辑关系、函数/运算符作用域、XSS 的可执行片段与上下文逃逸结构、CMDi 的命令连接与执行边界上保持一致，或可证明等价。而把编码、注释、空白与同义 token 替换视为语义不变的表层重写。在满足语法合法与语义保持约束的前提下，对 AST 的局部子树做替换、包裹或重排。安全测试里之所以强调受约束，是因为不加约束的树编辑会迅速引入大量不可解析样本，造成预算浪费。Gramatron 等人^[48] 讨论过传统 grammar fuzzing 的局限，指出仅做局部、小幅度的树变异往往难以快速合成复杂触发串，因此需要更结构化、可控但幅度更大的变异操作，并通过自动机构造缓解采样偏置。AST 层面的等价替换与编码包裹就是把这种更大尺度但仍合法的变异落实到攻击载荷域的方式。

2.5 蒙特卡洛树搜索算法基础

蒙特卡洛树搜索 (MCTS) ^[15] 是一类在大规模决策空间中进行启发式规划的通用方法, 通过选择、扩展、模拟与回溯的循环逐步逼近最优策略。其核心思想是通过重复的随机采样构建局部搜索树, 并利用统计学方法在不确定性环境下动态逼近最优策略。在形式化执行过程中, MCTS 的单次迭代通过四个紧密衔接的步骤驱动搜索树的生长 ^[16]。首先, 算法从根节点出发, 依据特定的树策略向下递归遍历, 直至找到某个尚未被完全展开的叶子节点, 以此完成节点状态的选择。随后, 若该节点未达到最终的终止状态, 算法将根据环境规则生成相应的合法动作节点, 从而实现树结构的扩展。紧接着, 算法从新扩展的节点切入, 利用诸如随机游走等默认策略进行快速的模拟推演, 直至到达最终状态并计算出该路径的评估回报。最后, 这一回报值会沿着先前的搜索路径进行反向回溯, 借此更新沿途所有节点的经验访问次数与期望价值估计, 从而为下一轮的决策提供更精确的统计指导。在搜索空间巨大并且计算能力有限的情况下, 这种启发式搜索算法可以充分权衡探索和利用, 以优化搜索过程。

在选择阶段, MCTS 在处理强化学习中的探索与利用困境问题时, 通常被建模为多臂赌博机 (Multi-Armed Bandit, MAB) 问题 ^[56], 其核心是上限置信区间 (Upper Confidence Bounds, UCB) 公式, 即以均值的置信上限为代表它的预估值。具体来说, 它会计算出每一个节点的预估值的均值, 并以一个置信上限作为这个均值的参考。Auer 等 ^[57] 基于霍夫丁不等式提出了 UCB1 算法, UCB1 算法赋予每个动作一个乐观估计, 同时 UCB1 给出了有限时间遗憾界分析。Kocsis 与 Szepesvári ^[55] 将 UCB 引入树结构决策, 提出 UCT (Upper Confidence bounds applied to Trees), 把在某一状态下选哪个动作的问题转化为在树节点上用 bandit 规则选分支, 将多臂赌博机的上置信界思想引入树搜索, 使得节点选择可以用一个统一的评分函数实现。由于 WAF 黑盒检测的预算分配本质上就是一个决策问题, 每次请求成本近似固定, 但候选输入空间呈指数增长, 盲目均匀探索会导致每条路径都试一点, 但没有一条试够深, 最终收敛慢且容易错过有效绕过边界。UCT 把预算集中到更可能产出高回报的分支, 同时又不会把尚未充分探索的分支完全丢掉, 正好吻合要解决的结构化空间中的收敛效率目标。

在工程理解上, 可以将 UCT 视为平均回报和不确定性补偿 ^[58] 两方面。平均回报对应利用项, 反映该动作的经验效果。不确定性补偿通常与访问次数的对数与反比平方根相关, 使得访问较少的动作获得更高补偿, 从而被优先探索。随着访问次数增长, 不确定性项自然衰减, 搜索会逐步集中到更有希望的分支。这种可解释的探索—利用平衡机制, 使 MCTS 在候选空间极大且难以建立解析模型的场景

中具备优势。

2.6 本章小结

本章从攻击语义、防御机理与算法理论三个层面给出了本文方法的必要前置。SQLi、XSS 与命令执行的本质在于输入在不同上下文中被解释为结构而非数据，这使得语义保持的载荷变形成为 WAF 绕过与漏洞触发的共同基础。形式化语言与 BNF 提供了载荷空间的可描述与可枚举机制，AST 为语义保持变异与可解释路径追踪提供结构化载体，MCTS 则为在指数级搜索空间中进行预算分配与全局收敛提供通用框架。

第3章 基于语义保留的对抗性载荷空间构建策略

3.1 引言

随着 Web 架构的演进，WAF 的防御机制正从静态特征匹配向语义分析与机器学习转型，这对自动化评估手段提出了更高要求。然而，现有的黑盒测试工具在生成对抗性载荷时普遍面临有效性与对抗性权衡的困境，一方面，传统的非结构化随机变异极易破坏载荷的严格语法结构，导致后端无法解析执行而产生伪绕过现象。另一方面，依赖人工维护的静态规则库存在严重滞后性，难以覆盖生僻语法、特定数据库方言及多层组合编码，无法有效应对复杂的逻辑嵌套与未知威胁。针对上述问题，本章提出一种基于上下文无关文法的对抗性载荷构建策略。不同于传统的字符级变异，本策略引入了编译原理中的语法分析技术，将攻击载荷建模为抽象语法树（AST）。其主要包含统一语法规则库构建、语义保留的变异算子设计、动态剪枝优化三部分。

3.2 统一攻击语法规则库设计

为了实现对多种 Web 攻击类型的统一描述与生成，本节构建了一个可扩展的统一攻击语法规则库。该规则库屏蔽了底层攻击类型的差异，将 SQL 注入、XSS 及命令执行等攻击抽象为统一的语法生成问题。

3.2.1 基于 BNF 的多攻击类型语法定义

首先定义上下文无关文法 $G = (V, \Sigma, R, S)$ 。其中 S 是文法的起始符号，可与多模态攻击载荷层次化解析树中的节点及子树相映射。 V 是有限非终结符集合，代表多模态攻击场景下的各类语法结构，如 $\langle \text{Condition} \rangle$ ，用于扩展攻击载荷的生成空间，描述文法迭代推导的中间状态，即潜在的对抗性多模态攻击载荷生成目标。 Σ 是有限终结符集合，是无法通过语法规则进一步推导的基础符号，对应多模态攻击中的实际攻击字符，如 $'$, SELECT，也就是生成的攻击载荷实际内容。为了灵活适配多模态攻击的多样化需求，可通过代码动态生成部分终结符的值，以此提升攻击载荷的多变性。 R 是预先定义的文法规则集合，也称为产生式，形式为 $A \rightarrow \alpha$ ，本文采用扩展巴科斯范式（EBNF）来描述 R ，用于将原始多模态攻击载荷迭代转换为语义等价的对抗性多模态攻击载荷，同时保留其攻击语义的正确性与恶性。为了支持多模态攻击，定义顶层规则如表3-1所示。

这种模块化设计使得系统具有极强的扩展性。例如，当需要支持 LDAP 注入

表 3-1 顶层规则
Table3-1 Top-Level Rules

顶层规则
<code><Attack_Vector> ::= <SQL_Injection> <XSS_Payload> <OS_Command_Injection></code>

时，只需在顶层规则中增加 `<LDAP_Injection>` 并定义相应的子文法即可。

3.2.2 SQL 注入文法规则设计

SQL 注入 (SQLi) 利用的是数据库解释器与 WAF 解析器之间的语义差异 (Semantic Gap)。基于 ISO/IEC 9075 SQL 标准及主流数据库特性，本文构建了深度覆盖的 SQL 文法。

1. 逻辑注入与布尔盲注文法 逻辑注入与布尔盲注的核心在于构造语义等价的恒真或恒假表达式，从而干预或接管后端的查询逻辑。该文法不仅覆盖了标准的逻辑连接词，还引入了位运算符，如 `&`, `|`，与比较算符，如 `REGEXP`, `RLIKE`, `IS` 作为等价替换分支。同时，结合十六进制或嵌套表达式等复杂操作数，使得生成的逻辑判断片段能够在保持恒真/恒假语义的前提下，呈现出高度碎片化且极具迷惑性的表层语法结构。定义文法规则表如表3-2所示。

表 3-2 逻辑注入与布尔盲注文法
Table3-2 Logic injection and boolean blind injection syntax

逻辑注入与布尔盲注文法
<code><Logic_Injection> ::= <Prefix> <Logic_Operator> <Expression> <Suffix></code>
<code><Logic_Operator> ::= "AND" "&&" "OR" " " "XOR" "&" " "</code>
<code><Expression> ::= <Comparison> <Function_Call> "(" <Expression> ")"</code>
<code><Comparison> ::= <Operand> <Comp_Op> <Operand></code>
<code><Comp_Op> ::= "=" "<" "LIKE" "REGEXP" "RLIKE" "IS"</code>
<code><Operand> ::= <Number> <String> <Hex_String> <Bit_Operation></code>

2. 联合查询文法 (Union-based) 针对 UNION SELECT 这一高频拦截特征，文法还需包含混淆变体。为了突破此类连续字符串特征的封堵，本文针对该结构的文法定义引入了细粒度的混淆变异机制。定义文法规则表如表3-3所示。

统一攻击语法规则库的设计难点在于应对不同数据库管理系统 (DBMS) 之间的语法差异 (Dialect Discrepancies)。现有的单一模板工具往往因忽视这种差异而导致攻击载荷在特定环境下失效。本节以字符串连接操作和盲注延时为例，分析

表 3-3 联合查询文法
Table3-3 Joint query grammar

联合查询文法
<Union_Select> ::= <Union_Key> <All_Opt> <Select_Key> <Column_List> <Union_Key> ::= "UNION" "/*!UNION*/" "/*!50000UNION*/" "UnIoN" <Select_Key> ::= "SELECT" "/*!50000SELECT*/" "SELECT DISTINCT"

文法实例化的差异。文法实例化差异中字符串连接文法实例如表3-4所示。

表 3-4 字符串连接文法实例
Table3-4 String concatenation syntax examples

字符串连接文法实例
MySQL 环境：依赖 CONCAT() 函数。 文法生成实例：UNION SELECT CONCAT(user, 0x3a, password) FROM users
Oracle 环境：使用双竖线 操作符。 文法生成实例：UNION SELECT user ':' password FROM users
SQL Server 环境：使用加号 + 操作符。 文法生成实例：UNION SELECT user + ':' + password FROM users

文法实例化差异中盲注延时函数文法实例如表3-5所示。

表 3-5 盲注延时函数文法实例
Table3-5 Blind delay function syntax example

盲注延时函数文法实例
MySQL：SLEEP(5) 或 BENCHMARK(1000000, MD5(1))。
PostgreSQL：PG_SLEEP(5)。
SQL Server：WAITFOR DELAY '0:0:5'。

本节通过定义抽象的 <String_Concat> 和 <Time_Delay> 非终结符，先将字符串拼接、延时执行这类核心攻击语义与底层实现分离，再利用提前采集的目标环境指纹，精准匹配对应数据库、服务器支持的具体语法与函数。同时在生成阶段根据目标指纹选择具体的终结符实现，从而自动适配上述差异，确保生成的测试载荷的语义有效性。

3.2.3 XSS 攻击文法规则设计

跨站脚本攻击（XSS）的变异空间主要集中在 HTML 标签的属性及 JavaScript 伪协议上。针对现代浏览器和 WAF 对 XSS 的过滤特性，定义文法规则表如表3-6所示。

表 3-6 XSS 攻击文法
Table3-6 XSS attack syntax

XSS 攻击文法
<code><XSS_Payload> ::= <Script_Tag> <Event_Vector> <Pseudo_Protocol> <SVG_Vector></code> <code><Script_Tag> ::= "<script" <Whitespace> ">" <JS_Code> "</script>"</code> <code><Event_Vector> ::= "<" <Tag_Name> <Whitespace> <Event> "=" <Quote> <JS_Code> <Quote> ">"</code> <code><Tag_Name> ::= "img" "body" "iframe" "details" "audio"</code> <code><Event> ::= "onload" "onerror" "onmouseover" "ontoggle"</code>

文法中加入了 `<SVG_Vector>` 和 MathML 标签，这些非标准的 HTML 标签常被 WAF 忽略，但在现代浏览器中仍能执行脚本。

3.2.4 命令注入攻击文法规则设计

命令注入依赖于 Shell 解释器的管道符与连接符。本研究结合 Linux 与 Windows 特性，定义文法规则表如表3-7所示。

表 3-7 命令注入攻击文法
Table3-7 Command injection attack syntax

命令注入攻击文法
<code><OS_Cmd> ::= <Command_1> <Separator> <Command_2></code> <code><Separator> ::= ";", " ", " ", "&&" "%0a" "\$()" "`</code> <code><Command> ::= <Bin_Path> <Space> <Args></code> <code><Bin_Path> ::= "cat" "/bin/cat" "c"at" "c\at" "c\$@at"</code>

`<Bin_Path>` 规则展示了 Shell 的拼接特性。例如 `c"at` 在 Bash 中等价于 `cat`，但 WAF 的字符串匹配规则通常只会查找连续的 `cat`。一些 WAF 依赖精准字符串匹配，无法识别这类拆分后的变形指令，难以触发对应的拦截规则，进而实现有效绕过。同时该规则兼顾 Linux 与 Windows 双系统 Shell 的解析差异，具备良好的跨环境适用性，能够有效提升命令注入载荷的绕过成功率与场景覆盖度。

3.2.5 文法规则的可扩展性与演进机制

尽管前文定义的文法规则覆盖了主流的 SQL 注入、XSS 及命令执行攻击，但网络攻防具有高度的动态性。新的中间件特性，如 Fastjson 漏洞、异构数据库方言，如 ClickHouse、Redis 以及新型攻击范式，如 GraphQL 注入层出不穷。若系统的规则库是静态封闭的，将很快面临失效的风险。

为了解决规则完备性的问题，本文区别于硬编码的设计模式，采用了核心抽象

与插件化扩展的文法架构。该架构允许在不修改解析引擎核心代码的前提下，通过配置文件或外部接口来动态适配新的威胁场景。

本文中阐述的统一攻击语法规则库遵循正交解耦原则。顶层文法解析器被设计为一个通用的工具语句生成虚拟机，而具体的攻击规则被封装为独立的描述文件（如 .bnf 或 .json 格式）。需要对新的攻击类型进行拓展时，例如需要支持服务端模板注入（Server-Side Template Injection, SSTI）攻击检测时，无需重写 AST 构建算法，只需在规则库目录中新增 ssti_rules.bnf 文件，ssti 攻击文法示例如表3-8所示。系统启动时会自动扫描并加载该子文法树。

表 3-8 ssti 攻击文法
Table3-8 SSTI attack syntax

ssti 攻击文法
<pre>/* ssti_rules.bnf */ <SSTI_Injection> ::= <Jinja2_Vector> <FreeMarker_Vector> <Velocity_Vector> <Jinja2_Vector> ::= "{" <Python_Gadget> "}" "%<Control_Flow>%" <Python_Gadget> ::= "config.__class__" "lipsum.globals" "request['application']"</pre>

随后，在顶层配置文件中注册该模块，更新顶层规则如表3-9所示。

表 3-9 顶层规则更新
Table3-9 Top-level rule update

顶层规则更新
<pre>/* root.bnf (修改前) */ <Attack_Vector> ::= <SQL_Injection> <XSS_Payload> <OS_Command_Injection> /* root.bnf (修改后 - 新增 SSTI 分支) */ <Attack_Vector> ::= <SQL_Injection> <XSS_Payload> <OS_Command_Injection> <SSTI_Injection></pre>

解析引擎会自动将 <SSTI_Injection> 挂载到根节点 <Attack_Vector> 的选择分支中，实现功能的扩展。

3.3 对抗性载荷解析与抽象语法树构建

为了对攻击载荷进行精细化的变异操作，需要将原始字符串解析为抽象语法树（AST）。这一过程是将非结构化数据结构化的关键步骤。

3.3.1 词法分析与 AST 构建

词法分析器是 AST 构建的第一步。系统采用有限自动机扫描原始攻击载荷，将其分割为最小的语义单元 (Token)。语法分析器接收 Token 流，依据 3.2 节定义的 BNF 规则，采用递归下降分析法构建 AST。AST 的每个节点代表一个语法单元，节点属性包含类型 (Type)、值 (Value) 及可变异标识。接下来将以 SQL 注入攻击为例，来阐述抽象语法树的构建过程。

首先主要的主要 Token 类型定义有 T_KEYWORD: SQL 关键字 (SELECT, UNION)、T_OPERATOR: 运算符 (=, OR, --)、T_LITERAL: 字面量 ('admin', 105)、T_COMMENT: 注释符 (#, --, /*...*/)。对于载荷 ' OR 1=1 #，生成的 AST 结构与具体 Payload 解析为 AST 的示意图如图3-1所示。

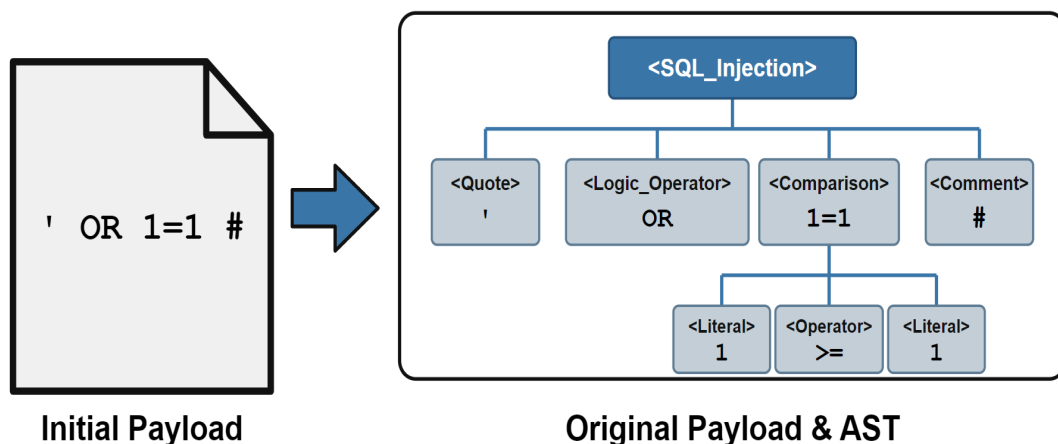


图 3-1 Payload 语法解析树
Fig.3-1 Payload syntax parsing tree

该攻击语句对应的抽象语法树以 SQL_Injection 为根节点，其下包含四个子节点。第一个子节点为 Quote，取值为单引号 '；第二个子节点为 Logic_Operator，取值为逻辑运算符 OR；第三个子节点为 Comparison，取值为 1=1，并可进一步递归解析为由操作数 1、等号 = 与操作数 1 构成的子树；第四个子节点为 Comment，取值为注释符号 #。整棵抽象语法树完整解析了 SQL 注入攻击语句的语法结构与关键语义特征。

3.4 语义保留的节点级变异操作

基于 3.3 所构建的 AST，本节将详细阐述如何通过 AST 变换来生成对抗样本。与传统的 Fuzzing 不同，本策略的核心约束是语义保留，即确保变异后的 Payload

在后端执行效果不变。

3.4.1 等价替换变异

等价替换是利用解释器的多态性，将 AST 中的终结符替换为语义相同的其他符号。传统的变异方法主要利用逻辑与运算的多态性与函数同义词替换的方式进行。在 MySQL 等数据库中，逻辑运算符常存在多种语法表达形式，例如 AND 可完全等价替换为 &&，OR 可替换为 ||。同时，攻击者也会运用数学恒等式与逻辑变换，将常规的表达式转换为语义相同但形式不同的结构，例如利用 $\text{NOT}(A < B)$ 来替代直接的等值判断 $A=B$ ，或使用数学上的对称性，如 $\text{NOT}(A)=\text{NOT}(B)$ 来表示原始的等值关系，从而扰乱基于关键字的检测规则。并且数据库的内置函数往往存在功能重叠的不同函数，这为变体生成提供了空间。例如，进行字符串截取操作时， $\text{SUBSTR}(s,1,1)$ 、 $\text{MID}(s,1,1)$ 与 $\text{LEFT}(s,1)$ 可交替使用，实现相同效果。此外，空白符变异也是一种基础而有效的干扰手段，普通的空格可以被替换为 URL 编码形式的 $\text{TAB}(\%09)$ 、换行符 $(\%0a)$ 、回车符 $(\%0d)$ 或垂直制表符 $(\%0b)$ 等，这些空白符在 SQL 解析时通常被忽略，却能有效绕过检测引擎预期的字符串模式。

然而，传统变异方式依赖于人工预定义的规则库，其变体集合有限。而根据 ICSE 2024 上 Fuzz4All^[24] 等研究揭示的新范式，通过将微调后的大语言模型嵌入代码的抽象语法树处理流程，可在特定节点上自动生成海量、语法合法且语义保持一致的代码变体。这种方式有效克服了人工经验在面对组合爆炸时的局限性，从而生成传统静态规则库或人工经验难以覆盖的复杂等价形式，例如，将简单的逻辑真值 $1=1$ 智能地替换为如 $\text{strcmp}(1,1)=0$ 这类函数调用形式。这意味着代码对抗技术正从基于规则的变异迈向基于深层语义理解的生成，极大地扩展了攻击面的广度与隐蔽性，对现有防御体系构成了系统性挑战。

为了验证 AST 节点级等价替换的有效性，本文选取了业界通用的开源 WAF 规则集——ModSecurity Core Rule Set (CRS) v3.3^[12] 作为对抗目标。CRS 中包含一条著名的规则 942100，专门用于检测 $1=1$ 等常见逻辑测试模式。

原始攻击载荷为 'OR 1=1'。由于规则 942100 利用正则匹配形如 $x=x$ 的结构，该载荷会直接接触发拦截。根据本章提到的变异方式，AST 变异与绕过过程将首先将载荷中的 $1=1$ 解析为 <Comparison> 子树，随后执行两类语义等价变异。一是通过函数变换，引入 MySQL 的 STRCMP() 函数，利用 $\text{!STRCMP}(1,1)$ 与 $1=1$ 语义等价特性，生成变异载荷 'OR !STRCMP(1,1) --'；二是基于集合特性，使用 IN 操作符将结构替换为 $1 \text{ IN } (1)$ ，生成载荷 'OR 1 IN (1) --'。两类变异均保留攻击语义，同时规避正则规则检测。

通过 AST 层面的语义保留变异，成功将一个被高分拦截的特征载荷转化为了

多个语义一致但在形态上完全不同的隐蔽载荷，且严格遵循了 SQL 语法规范。

3.4.2 多层级编码变异

编码变异策略的核心在于利用 WAF 的解码引擎与后端应用服务器，如中间件、数据库、解释器等之间存在的解析逻辑不一致性，即安全领域常述的语义鸿沟。为了系统化地评估和构建对抗样本，本研究基于 HTTP 协议栈的层级结构，本文构建了一个覆盖应用层、协议层的二维编码矩阵（见表3-10）。该矩阵形式化地定义了 AST 变异引擎在不同层级上可调用的变异算子。

表 3-10 应用层、协议层编码矩阵

Table 3-10 Application layer and protocol layer encoding matrix

协议层级	变异类别	编码/混淆技术	典型 Payload 示例	对抗原理
L7 应用层	基础字符编码	标准 URL 编码	%27	利用基础解码绕过简单的字符串匹配。
	多重编码	多重 URL 编码	%27 -> %2527	递归解码差异： WAF 解一次， 后端解两次。
	异构字符集	IIS Unicode/ EBCDIC	%u0027	字符集映射差异： 利用特定中间件的私有解码器。
	UTF 变异	Overlong UTF-8	%C0%A7	利用后端宽容的 UTF-8 解析器还原非法编码。
L7 协议层	HTTP 传输编码	Chunked Encoding	Transfer- Encoding: chunked	流重组差异：利用分块机制打断正则匹配的连续性。
	HTTP 走私	CL-TE / TE-CL	Content-Length / Transfer- Encoding	边界识别差异： 前后端对请求结束位置判定不一致。
	协议头变异	Header Fuzzing	H ost: target (插入空白)	元数据解析容错： 利用 Web Server 对 RFC 的违规容忍。

基于上述矩阵，本研究将变异策略具体展开为两个维度，这两个维度分别为字符级编码和协议层混淆。字符级编码主要通过对 Payload 内容的字符集表示进行混淆，借助不同解码行为的差异绕过正则匹配。该方式包含多种实现形式，其一遵循 RFC 3986 标准的标准 URL 编码，将特殊字符转换为 %HH 格式。其二是多重编码，利用 WAF 仅单次解码，而后端完整链路会多次解码的差异，使 Payload 在检测阶段呈正常百分号字符串，在服务端还原为恶意内容。其三是非标准异构编码，利用

Microsoft IIS 对%uXXXX 格式、IBM EBCDIC 编码等底层字符集解析差异实现混淆。协议层混淆不直接修改 Payload 内容，而是对 HTTP 请求结构与元数据实施变异。总体分为两类方法，一类方法是基于 AST 根节点的协议头变异，结合 CCS 2021 (T-Reqs)^[59] 的研究思路，在 Host 等头部字段插入非法空白符，使 WAF 与后端服务器对请求边界解析不一致。另一类为分块传输编码，通过设置 Transfer-Encoding: chunked 将 Payload 拆分到多个数据块，并可加入分块扩展参数，干扰 WAF 的流重组过程，从而降低检测准确率。

为了验证基于抽象语法树的多层级编码变异方法在实际攻防对抗中的有效性，本文选取了一个受主流云 WAF 保护的反射型跨站脚本（XSS）漏洞场景进行深入剖析。在该场景中，目标系统采用典型的现代 Web 架构，前端部署了具备实时检测与拦截能力的云 WAF，后端由 Nginx 作为反向代理，并承载基于 Spring Boot 开发的应用程序。初始的攻击载荷为经典的 `<script>alert(1)</script>`。测试表明，该 WAF 策略能够对 GET 请求参数进行规范化解码，并有效拦截包含 `<script>` 等敏感 HTML 标签的明文或简单编码载荷。

变异引擎通过一个系统化的、多层次正交叠加的变异策略，成功构造出可绕过该防御体系的对抗样本。其核心过程始于应用层语义保持变异，在载荷的抽象语法树节点层级，引擎将关键字符 `<` 替换为其等价的十六进制 HTML 实体形式 `<`，从而生成中间态载荷 `<script>alert(1)</script>`。此举旨在保留载荷的最终执行语义，同时改变其表层语法形态。随后，引擎实施协议层编码混淆变异，对上述的中间态载荷执行双重 URL 编码。经过双重 URL 编码后的载荷形态呈现为 `%253cscript%253ealert(1)%253c%2fscript%253e`。此步骤的绕过机理在于，云 WAF 在检测时通常会执行单层 URL 解码，将 `%253c` 还原为 `%3c`。然而，多数规则引擎的检测逻辑止步于此，仅匹配形如 `%3cscript` 的单层编码模式或明文 `<script>`，而未能深入识别双重编码解码过程中产生的中间形态 `%3c`，从而导致安全策略误判并放行该请求。攻击载荷的最终还原发生在目标系统内部，当请求抵达后端 Spring Boot 应用时，Spring MVC 框架在参数绑定过程中会自动执行完整的 URL 解码，将 `%3c` 还原为字符 `<`。随后，该载荷被嵌入 HTTP 响应并返回至客户端浏览器。现代浏览器在解析 HTML 文档时，会将实体 `<` 正确解释为字符 `<`，从而完整复原原始的恶意脚本并予以执行。

为进一步提升攻击的隐蔽性与成功率，变异引擎还引入了传输层传输机制变异，将请求方法从 GET 切换为 POST，并启用 HTTP 块传输编码机制，将关键词如“script”分割为“sc”、“ri”、“pt”等独立数据块进行发送。该技术利用了部分 WAF 设备对 POST 请求体进行流式重组时的深度限制或性能优化策略，使得关键特征在检测时被进一步模糊化，显著降低了载荷被静态或浅层动态分析识别的概率。

通过抽象语法树驱动的、将应用层内容多态性、协议层编码混淆性与传输层传输机制多态性进行系统化正交组合的方法，能够构造出极具穿透力的复合式攻击向量。这种立体化的变异策略，有效突破了依赖于单一维度检测，如关键词匹配、固定层数解码的传统纵深防御体系。

3.4.3 注释与空白符干扰插入

在 WAF 的检测流程中，词法分析是正则匹配的前置步骤^[60]，其核心任务是将连续的输入数据流解析为具有明确类型的离散令牌，从而为后续的语法与语义分析奠定基础。针对这一防御机理，本节所提出的变异策略聚焦于词法分析层，其核心思想是，通过在对载荷的抽象语法树关键节点之间，精心插入不影响后端解释器最终语义的无关数据，主动构造一种结构化的干扰，破坏 WAF 对恶意特征的连续性识别，使其无法提取出完整的关键令牌，或诱导 WAF 解析器与后端数据库解析器产生令牌化歧义，诱使二者对同一段数据流做出不同的结构化解释，从而令 WAF 的检测模型失效。

具体而言，该策略主要通过以下两类技术路径实现，分别为内联注释注入和垃圾数据填充与缓冲区压力干扰。内联注释注入利用 SQL、JavaScript 等语言中定义的注释语法，将单一的敏感关键词或操作符在物理层面进行分割。例如，将攻击载荷中的 UNION SELECT 变异为 UN/**/ION SE/**/LECT。其绕过机制在于解析层面的不对等性，WAF 的正则检测引擎若未集成完整的语法预处理模块以剥离这些注释，便无法在词法流中匹配到连续的 UNION 和 SELECT 令牌。然而，后端数据库服务器在解释执行 SQL 语句前，其词法分析器会严格遵循语言规范，将注释内容视为无效字符予以忽略，从而能够无缝地重组并正确执行被分割的恶意指令。这实质上是利用了防御方与执行方在语法纯净度处理上的差异。垃圾数据填充与缓冲区压力干扰通过在 Payload 内部、参数之间或请求首尾部，插入大量无意义的空白符或冗余的随机参数名-值对，对 WAF 的检测引擎施加性能与资源压力。一方面，极度复杂的模式与超长字符串可能引发正则表达式引擎的灾难性回溯，导致计算资源在模式匹配过程中被耗尽，从而触发超时或绕过。另一方面，若输入数据的总体长度超过了 WAF 检测模块预分配的内存缓冲区或处理深度上限，部分 WAF 产品出于保证业务连续性的考虑，会进入故障开放状态，即自动截断超长部分或跳过对后续数据流的深度检测，直接将其放行。通过在词法层系统性地植入语义无关的干扰元素，攻击者能够有效扰乱 WAF 赖以工作的令牌化基础，或利用其性能边界实现强制绕过。

为了验证本研究提出的 AST 变异方法在处理复杂、递归式攻击载荷时的有效性，本文以典型的 Log4j2 远程代码执行漏洞（CVE-2021-44228）为实验对象。该

漏洞底层利用了 JNDI 注入的 Lookup 机制其核心在于 Log4j2 框架对 $\${}$ 占位符支持深度的插值替换与动态求值。这种机制使得恶意 Payload 的结构天然具备上下文无关文法的深层递归嵌套特性，是检验本文结构化嵌套变异算法规避连续特征检测能力的理想评估对象。在实际对抗中，WAF 通常会针对 jndi、ldap 或 rmi 等连续特征协议字符串实施严格的正则匹配与拦截。为突破此类静态签名封堵，高级绕过载荷往往需要利用 Log4j2 内置的默认值分隔符（如:-）或格式化修饰符（如 lower:、upper:）对敏感关键字进行多层级的字符割裂与语义等价混淆。为实现这一对抗过程的自动化与可控性，本研究深入剖析了 Log4j2 解析引擎的底层求值逻辑，将上述碎片化绕过技巧严格抽象为树节点上的推导分支，并扩展定义了如下的巴科斯范式以约束其生成空间，如表3-11所示。

表 3-11 Log4j2 远程代码执行漏洞文法
Table3-11 Log4j2 remote code execution vulnerability syntax

Log4j2 远程代码执行漏洞文法

```

<Log4j_Payload> ::= "${" <Lookup_Content> "}"
<Lookup_Content> ::= <JNDI_Structure> | <Obfuscation_Wrapper>
/* 核心 JNDI 结构定义 */
<JNDI_Structure> ::= <JNDI_Key> ":" <Protocol> "://" <Host> "/" <Path>
<JNDI_Key> ::= "jndi" | <Case_Obfuscation>
/* 协议层混淆：利用 Lookup 功能的 lower/upper 指令进行关键词拆解 */
<Case_Obfuscation> ::= "${lower:j}ndi" | "${upper:j}ndi" | "${lower:j}${upper:n}di"
<Obfuscation_Wrapper> ::= <JNDI_Key> ":" <Log4j_Payload> ":-" <Random_String> "}" | "::-"
<Log4j_Payload>

```

在针对 Log4j 等复杂漏洞的利用过程中，所采用的变异策略核心在于充分利用目标系统解析逻辑与防御设备静态检测之间的根本性差异。具体而言，攻击载荷通过抽象语法树（AST）生成如 $\${\${::-j}ndi:rmi://...}$ 的嵌套结构，其绕过原理深度依赖于 Log4j2 解析器对 $\${...}$ 占位符所特有的递归求值机制。WAF 依赖的正则检测引擎通常在处理此类理论上可无限递归的嵌套语法时存在局限，往往只能捕获最外层或最内层的固定文本模式，而无法完整跟踪并理解其动态展开过程，从而导致关键的恶意特征（如 jndi）在词法层面被有效分割与隐藏。与此同时，为进一步规避对 ldap:// 及 rmi:// 等协议指纹的静态规则匹配，变异引擎会应用动态混淆规则，生成如 $\${lower:l}${lower:d}ap://$ 的变体。此类 Payload 在静态流量中仅呈现为无害的字符串与函数指令，然而当其在目标系统的 Log4j2 环境中被执行时，内置的 lower 等函数会在运行时动态求值并完成拼接，最终在内存中还原出完整的恶意协议头。这一静态混淆与动态还原的过程，使得依赖流量镜像与模式匹配的传统防御设备几乎无法在攻击发生前有效识别威胁，从而凸显了基于语义理解与动

态行为分析的下一代防御技术的必要性。

3.5 基于规则探测的载荷空间剪枝优化

由上下文无关文法与多类型变异共同构成的载荷生成空间呈现显著的组合爆炸特征，若在黑盒反馈条件下直接进行无差别搜索，将导致有效样本命中率低、收敛速度慢以及查询预算消耗过快等问题。尤其在云 WAF 场景下，测试者通常只能获得拦截/放行这一弱反馈信号，且存在限速、封禁与响应噪声，使得测试成本进一步增大。与近年来 Web 应用模糊测试研究总结一致，受限于观测与预算约束时，输入空间扩展必须与反馈利用机制结合，才能维持可用的检测效率。

为提升收敛效率，本节在语义保留载荷空间构建的基础上，引入基于主动学习^[61]的规则探测与剪枝机制，通过构建最小区分探测集获得高辨识度的 WAF 行为信息，并以多信号响应指纹增强黑盒观测的稳定性。并在此基础上建立动态规则映射表 M ，以风险—收益联合准则驱动硬剪枝与软剪枝，并在检测过程中引入概念漂移检测以适配云端策略的持续演化，从而形成探测、映射、剪枝与再探测的完整体系架构。

3.5.1 WAF 防护规则探测机制

在黑盒评估场景下，WAF 的内部规则与解析逻辑通常不可见，系统能够获得的反馈信息主要来自 HTTP 层面的拦截结果与有限的响应特征。因此，在进行大规模对抗载荷搜索之前，有必要通过主动探测建立对目标 WAF 防护边界的初步认知，为后续规则映射表构建与空间剪枝提供稳定的先验依据。本节提出一种面向资源约束环境的规则探测机制，其核心思路是在有限查询预算内，通过少量但区分度高的探测请求尽可能揭示 WAF 对不同结构片段与上下文组合的敏感性，从而避免在映射表尚未收敛时进行高成本的盲目扩展。该思路与近年服务器端 Web 模糊测试综述^[62]中强调的成本约束下的反馈利用与输入空间扩展控制方向一致。

为统一后续描述，本文在探测阶段引入如下符号体系： D 表示最小区分探测集，由少量探测载荷构成； y 表示一次探测请求对应的多信号响应指纹； κ 表示由 y 归纳得到的响应模式标签或指纹簇类别； M 表示规则映射表，用于建立载荷结构与上下文的风险关联； P_{block} 表示在当前 WAF 策略下被拦截的概率估计。

探测载荷的构建采用最小区分探测集策略。传统的正交化基准探测载荷能够覆盖较多 token 与上下文组合，但在真实云 WAF 场景中，过多探测会明显增加触发限速、挑战策略以及测量噪声累积的风险，进而降低探测结论的可靠性。为此，本文在正交化思想基础上进一步约束探测规模，从 CFG 的关键非终结符展开集合与典型上下文位置集合中，选取一组规模尽可能小、但对规则边界具有较强区分能

力的探测载荷集合 D 。 D 的目标并非覆盖全部变体，而是以有限请求次数最大化区分不同拦截行为模式，确保在初始阶段即可获得足以驱动剪枝的关键风险信息，从而提高后续搜索阶段的单位预算产出。 D 的构造以第三章建立的上下文无关文法为基础。设统一攻击语法规则库在给定攻击类型与上下文能够生成候选载荷集合 u 。这里 u 由 CFG 的浅层展开得到，即优先选取关键非终结符的最小语义闭合实例，使候选载荷仅包含少量可归因的触发 token，避免深层组合导致的因素混杂。以 SQLi 为例，候选集中的样本通常围绕字符串界定符、逻辑连接词、比较运算符与注释终止符构造。 u 不追求规模，而是保证每条样本都可被解释为某一类结构触发因素的代表。

为了提高黑盒观测的稳健性，本文进一步采用多信号响应指纹刻画探测结果。仅依赖 HTTP 状态码，例如 403/200 的判断拦截逻辑容易受到旁路行为影响，例如 WAF 返回自定义挑战页、发生重定向、连接复位或出现异常超时等，这些现象可能并非直接由规则命中导致。为此，本文将一次探测的响应抽象为特征向量，如公式(3-1)所示。

$$y = \langle code, \Delta t, len, header, redirect, conn \rangle \quad (3-1)$$

式中 $code$ 表示状态码， Δt 表示响应耗时， len 表示响应体长度或包长特征， $header$ 表示关键响应头集合，如 Server、Set-Cookie、Cache 相关字段差异， $redirect$ 表示跳转行为， $conn$ 表示连接状态，如正常返回、连接复位、超时等。在实现层面，可将响应头差异编码为集合距离或哈希特征，并结合 y 对响应行为进行聚类，形成拦截模式簇，从而更稳健地区分规则拦截、限速策略与后端异常等非同质行为。该机制能够显著降低探测阶段的误判概率，使后续映射表构建更贴近真实防护逻辑，而非被运行态噪声牵引。

在多信号响应指纹中，响应耗时 Δt 可作为识别 WAF 内部匹配复杂度的关键侧信道指标。由于 WAF 规则通常依赖复杂的模式匹配，特定输入可能触发正则引擎或解析链路的性能开销显著放大。因此，在探测阶段，若观察到某类 $(Token_i, Context_j)$ 组合对应的 Δt 呈现统计学意义上的显著上升，系统将直接将其标记为高代价规则路径，并在后续搜索中赋予该路径更高的探索优先级。需要强调的是，本文对延迟信号的采集仅作为反映规则解析敏感性的侧面指标，用以引导变异预算的最优分配，而非出于触发拒绝服务的目的。

针对云 WAF 规则持续更新导致先验映射表 M 易随时间失效的问题，本研究引入了概念漂移触发的自适应重探测机制^[64]。该机制通过维护历史观测窗口 W_{old} 与近期观测窗口 W_{new} ，实时监控两者的拦截模式分布差异，如拦截率或指纹簇标签的变化。当分布差异超过预设阈值时，系统将自动触发快速重探测，仅对关键的

token/context 子集进行校准更新^[65]。这种基于数据分布变化的动态监控策略，能够以极低的探测成本维持映射表 M 的时效性，从而避免系统在已失效的规则映射上继续消耗有限的搜索预算。

通过上述规则探测机制，本研究在黑盒条件下获得了对 WAF 防护边界的可操作认知，为下一节规则映射表 $M(Token, Context) \rightarrow P_{block}$ 的构建提供基础输入，并为硬剪枝与软剪枝提供风险先验来源。同时，多信号指纹与漂移触发信号将在 MCTS 搜索中进一步发挥作用，作为搜索节点状态的辅助观测量与自适应更新依据，从而形成探测、映射、剪枝、搜索、反馈与重探测的完整体系。

3.5.2 规则映射表的构建

在完成最小区分探测集 D 的黑盒探测后，获得探测集 D 对应的多信号响应指纹 y 及响应模式标签 κ 后需要将分散的探测反馈转化为可被系统利用的结构化知识，以支撑后续载荷空间的定向压缩与搜索引导。为此，本章构建动态规则映射表 M ，其目的并非复现 WAF 内部的具体规则逻辑，而是在黑盒条件下建立载荷结构特征、上下文位置与拦截风险之间的概率关联，从而为剪枝策略与搜索策略提供可解释的先验依据。

具体而言，令 $Token_i$ 表示载荷抽象语法树 (AST) 中可被 WAF 规则敏感捕捉的关键语义节点或符号片段（例如由 CFG 终结符或节点类型抽取的敏感单元），令 $Context_j$ 表示载荷注入的上下文位置，如 URL 参数、请求体字段、Header、Cookie 等。映射表以二元组 $(Token_i, Context_j)$ 为索引，输出其被拦截概率估计，如公式(3-2)所示。

$$M(Token_i, Context_j) \rightarrow P_{block} \in [0, 1] \quad (3-2)$$

式中 P_{block} 表示在目标 WAF 的当前策略下，该组合触发拦截的风险水平。通过对指纹进行聚类或模式判别，可以得到较稳定的拦截类型标签 κ ，从而区分规则拦截与非规则性干扰，使得后续 P_{block} 的更新更接近真实防护逻辑而非运行态噪声。这一处理不仅提升了映射表的鲁棒性，也为后续搜索过程中的失败原因归因提供依据。

在概率估计方面，映射表不采用单次观测的硬判定，而是通过在线累积更新获得更稳定的风险估计。本文采用的方法是将 P_{block} 视为可学习的概率量，并采用 Beta-Bernoulli 建模，如公式(3-3)所示。

$$P_{block}(i, j) \sim Beta(\alpha_{ij}, \beta_{ij}) \quad (3-3)$$

式中 $Beta$ 分布不是概率本身，而是概率的分布。它把拦截/放行看成硬币正反面，

α_{ij} 表示拦截事件的累计证据强度 (block 的计数), β_{ij} 表示放行事件的累计证据强度 (allow 的计数), 当观测到拦截事件时更新 $\alpha_{ij} \leftarrow \alpha_{ij} + 1$, 放行事件则更新 $\beta_{ij} \leftarrow \beta_{ij} + 1$, 从而得到后验期望 $\hat{P}_{block}(i, j)$, 如公式(3-4)所示。

$$\hat{P}_{block}(i, j) = \mathbb{E}[P_{block}(i, j)] = \frac{\alpha_{ij}}{\alpha_{ij} + \beta_{ij}} \quad (3-4)$$

式中如果 α 很大、 β 很小, 则期望接近 1, 则极可能拦截; 如果 α 很小、 β 很大, 则期望接近 0, 则极可能放行。该形式能够随观测次数增长自然提高估计置信度, 并降低随机噪声对映射结果的影响, 适合云 WAF 的弱反馈环境。

由于云 WAF 的规则库与检测策略具有持续演化特征, 映射表在时间维度上可能出现退化。为保证映射知识的时效性, 本文引入了概念漂移触发的局部校准机制, 系统维护近期窗口与历史窗口的拦截分布或指纹簇分布, 一旦观测到显著偏移, 便触发快速重探测, 仅对关键 $(Token_i, Context_j)$ 子区域重新采样更新, 从而避免全量重建带来的额外成本。概念漂移的监控与无监督环境下的更新机制尤其适用于无显式标签、仅有行为反馈的在线系统场景。

3.5.3 高风险节点与路径的剪枝策略

在获得映射表 $M(Token, Context) \rightarrow P_{block}$ 后, 下一步需要将风险信息投影到载荷空间的结构化表示上, 使其能够直接作用于 AST 生成与变异过程, 从而减少无效扩展并提升黑盒测试收敛速度。剪枝策略的目标并不等价于最大化放行率, 因为过度追求放行可能导致搜索偏离具有漏洞发现价值的结构区域, 更合理的目标是在有限查询预算内提高有效样本密度, 并保持对潜在缺口路径的覆盖能力。因此, 本节采用硬剪枝与软剪枝相结合的方式, 并引入风险与收益联合评分, 使剪枝既具备压缩空间的强约束能力, 也具备与后续优化搜索相兼容的导向能力。

剪枝首先需要对路径风险进行聚合估计。对于 AST 上的一条变异路径 $path$, 其整体拦截风险可以由路径包含的 token-上下文组合通过映射表 M 聚合得到, 如公式(3-5)所示。

$$P_{block}(path) = 1 - \prod_{(i,j) \in path} (1 - \hat{P}_{block}(i, j)) \quad (3-5)$$

聚合的方式可选择最大风险、加权和或乘积等形式, 用以描述路径整体拦截倾向, 分别对应保守风险估计、累计风险估计或联合风险估计。在本文的研究中, 期望的聚合既不是只看最大, 也不是纯线性累加, 而是至少命中一个就危险。因此 P_{block} 的计算选择乘积/联合风险的计算方式, 多个中等风险叠加时会明显增大, 更符合联合触发的直觉。

在此基础上, 为避免高风险路径被一刀切删除而丢失潜在缺口, 本文进一步

引入 Risk-Reward 联合评分, 对可探索性与结构收益进行统一度量, 如公式(3-6)所示。

$$Score(path) = \lambda \cdot (1 - P_{block}(path)) + (1 - \lambda) \cdot Gain(path) \quad (3-6)$$

式中 $(1 - P_{block}(path))$ 表示路径的放行潜力, $Gain(path)$ 表示结构收益, 可由引入新语法节点、覆盖新上下文组合或与已探索样本差异度等构成, 用于衡量路径在 token-context 组合层面的新增覆盖价值。前文中已经构建得到映射表 $M(Token_i, Context_j)$, 设 $Seen$ 表示历史已覆盖的 $(Token_i, Context_j)$ 配对集合, 则 $Gain(path)$ 可由公式(3-7)表示为:

$$Gain(path) = \frac{|\{(Token_i, Context_j) \in path \wedge (Token_i, Context_j) \notin Seen\}|}{|\{(Token_i, Context_j) \in path\}|} \quad (3-7)$$

式中 $Seen$ 由探测阶段样本以及在线搜索过程中已生成载荷累积得到并动态更新。参数 $\lambda \in [0, 1]$, 用于调节探索与利用的平衡。该评分的核心意义为剪枝不再仅以风险高低为唯一依据, 而是以风险与收益的权衡为准则, 从而保持搜索的全局性与效率。

硬剪枝适用于高置信必拦的结构区域, 即当某些节点或组合在多次观测中稳定触发拦截 (例如 $\mathbb{E}[P_{block}] \rightarrow 1$, 且响应指纹标签 κ 一致) 时, 系统在 AST 生成阶段直接禁用相关产生式展开或变异算子分支, 将搜索空间从源头进行压缩。硬剪枝的作用是显著减少无效查询, 并将预算集中到更可能产生有效对抗样本的替代结构上, 例如等价编码、结构重排或语义保持的替换路径。由于该策略发生在生成阶段, 其效果通常比搜索阶段的事后过滤更直接、更节省预算。

软剪枝则面向那些风险较高但仍可能存在规则覆盖缺口的路径区域。这类路径不宜完全删除, 而应降低其被选中概率并延后探索, 使其保留为低优先级候选, 从而兼顾效率与覆盖。软剪枝的实现方式可以将 $Score(path)$ 转换为路径权重并注入到后续搜索策略中, 可定义 $w(path)$, 如公式(3-8)所示。

$$w(path) = \exp(\tau \cdot Score(path)) \quad (3-8)$$

式中 τ 控制权重分布的陡峭程度, 较大的 τ 将放大不同路径评分之间的差异, 使搜索更倾向于优先探索高评分路径; 较小的 τ 则保留更多均匀探索空间, 避免过早收敛于局部结构。这样一来, 剪枝并非简单地删分支, 而是在保留分支的同时对预算分配施加导向, 避免局部最优导致的搜索塌缩。更重要的是, 软剪枝得到的权重 $w(path)$ 可以作为第四章 MCTS 选择策略的先验偏置, 从而在 UCB/UCT 框架下实现更合理的探索—利用平衡。近年来在 fuzzing 调度研究中引入 MCTS/Monte Carlo Planning 的工作^[66]表明, 通过上置信界机制进行预算分配能够显著提升在

大规模候选空间中的收敛效率，这为软剪枝与 MCTS 的结合提供了方法论依据。

剪枝策略并非一次性完成，而是随搜索过程持续修正。系统在每轮变异与测试后将新反馈回流至映射表 M ，动态更新 $(Token_i, Context_j)$ 的风险估计；当出现与映射表显著不一致的观测（如理论低风险路径反复失败、高风险路径偶发放行）时，将其记录为不一致事件并累计到漂移监控模块，一旦满足触发条件则执行局部重探测并同步修正剪枝权重，从而使剪枝在云 WAF 规则更新下保持稳定有效。该闭环机制使形成了从探测先验到空间压缩的完整链路，同时该机制使剪枝策略同时具备压缩空间的静态能力与随策略变化自我修正的在线能力，为第四章的动态自适应搜索提供稳定的信息底座。

为了在有限的探测预算下高效收缩庞大的载荷生成空间，并为后续的树搜索策略提供量化的先验指导，本节提出了一种基于规则探测的载荷空间剪枝优化方法。该算法首先对上下文无关文法进行浅层展开，并利用最小不确定性策略构造具备高信息增益的初始探测集。在与目标 WAF 交互后，系统根据提取的响应指纹在线更新各 Token-Context 对的 Beta 后验概率，从而构建出动态拦截映射表。根据该先验映射表，系统对生成空间实施双重维度优化，一方面通过硬剪枝直接禁用高置信度的必拦分支，以收缩文法约束。另一方面则综合路径的拦截概率与新结构覆盖收益，量化计算出软剪枝得分并转化为搜索权重。本节所提出的基于规则探测的载荷空间剪枝优化算法运行过程如算法 3-1 所示。

算法 3-1 基于规则探测的载荷空间剪枝优化方法

Data: CFG 规则库 G , 目标上下文集合 $\{Context_j\}$, 探测预算 B_{probe} ,

候选探测池规模 L , Beta 先验参数 α_0, β_0 ;

Result: 映射表 $M(Token_i, Context_j) \rightarrow \hat{P}_{block}(i, j)$, 剪枝后的生成约束 G' ,

软剪枝路径权重 $w(path)$;

- 1: **Initialize** $\alpha_{ij} \leftarrow \alpha_0, \beta_{ij} \leftarrow \beta_0$ for all $(Token_i, Context_j)$;
 - 2: **Initialize** $M(Token_i, Context_j) \leftarrow 0.5$;
 - 3: // ——— Step-1: 构造探测集 D ———
 - 4: $U \leftarrow GenerateProbePool(G, \{Context_j\}, L)$; // CFG 浅层展开候选探测池
 - 5: $D \leftarrow \emptyset$;
 - 6: **while** $|D| < B_{probe}$ **do**
 - 7: $d^* \leftarrow SelectProbeByUncertainty(U \setminus D, M)$; // 最小区分/最大信息增益
 - 8: $D \leftarrow D \cup \{d^*\}$;
 - 9: **end while**
 - 10: // ——— Step-2: 探测执行与映射表在线更新 ———
 - 11: **for** d **in** D **do**
 - 12: $y \leftarrow Fingerprint(SendHTTP(d))$; // 响应指纹 y
-

算法 3-1 基于规则探测的载荷空间剪枝优化方法（续）

```

13:   $\kappa \leftarrow ClusterMode(y)$ ; // 模式簇  $\kappa \in \{allow, block, \dots\}$ 
14:   $pairs \leftarrow ExtractTokenContextPairs(d)$ ; // 解析出  $(Token_i, Context_j)$ 
15:  for each  $(Token_i, Context_j)$  in  $pairs$  do
16:    if  $IsBlocked(\kappa)$  then
17:       $\alpha_{ij} \leftarrow \alpha_{ij} + 1$ ; // 拦截证据累积
18:    else
19:       $\beta_{ij} \leftarrow \beta_{ij} + 1$ ; // 放行证据累积
20:    end if
21:     $\hat{P}_{block}(i, j) \leftarrow \alpha_{ij} / (\alpha_{ij} + \beta_{ij})$ ; // Beta 后验期望
22:     $M(Token_i, Context_j) \leftarrow \hat{P}_{block}(i, j)$ ;
23:  end for
24: end for
25: // —— Step-3: 硬剪枝生成约束（得到  $G'$ ） ——
26:  $G' \leftarrow Copy(G)$ ;
27: for each production/mutation rule  $r$  in  $G'$  do
28:    $(Token_i, Context_j) \leftarrow RuleSignature(r)$ ;
29:   if  $M(Token_i, Context_j) \geq \tau_h$  then
30:      $DisablePlainBranch(r)$ ; // 禁用高置信必拦明文分支
31:   end if
32: end for
33: // —— Step-4: 软剪枝权重  $w(path)$  ——
34: for each candidate path in  $PathCandidates(G')$  do
35:    $P_{block}(path) \leftarrow Agg(\{M(Token_i, Context_j) \mid$ 
      $(Token_i, Context_j) \in path\})$ ;
36:    $Gain(path) \leftarrow NewPairRatio(path, M)$ ; // 轻量收益：新增组合比例
37:    $Score(path) \leftarrow \lambda \cdot (1 - P_{block}(path)) + (1 - \lambda) \cdot Gain(path)$ ;
38:    $w(path) \leftarrow \exp(\tau \cdot Score(path))$ ; // 转为可用于调度的权重
39: end for

```

通过最小区分度策略高效提取 WAF 响应指纹以及依托 Beta 后验校准机制，将动态拦截概率量化为稳定的先验知识。一方面利用硬剪枝机制无情剔除高置信度的无效明文分支以收缩搜索空间，另一方面依据得分为后续的 MCTS 算法注入了变异探索偏好。基于规则探测的载荷空间剪枝优化方法流程图如图3-2所示。

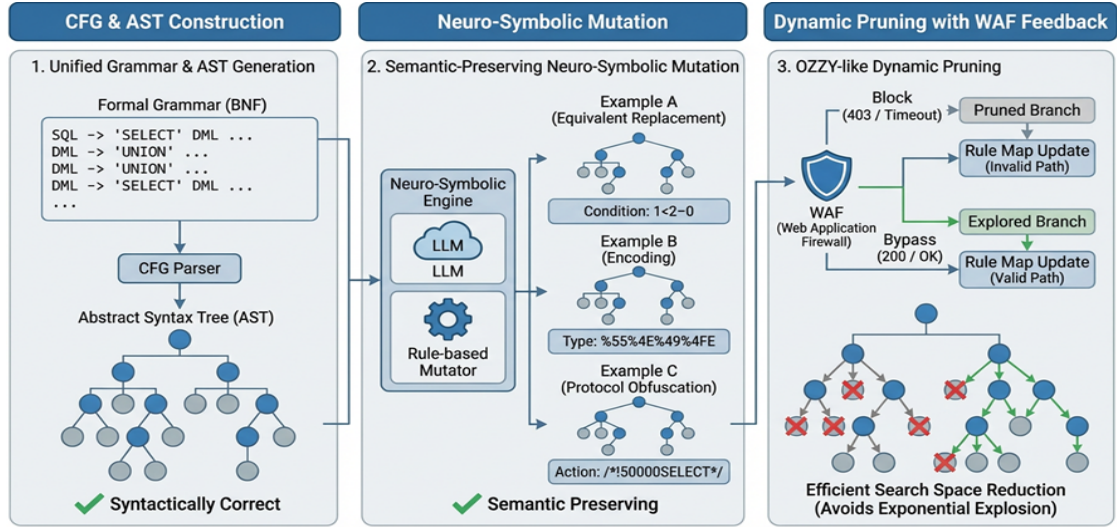


图 3-2 基于规则探测的载荷空间剪枝优化方法

Fig.3-2 Rule-Based payload space pruning optimization method

3.6 本章小结

本章围绕语义保留对抗载荷空间构建展开，建立了统一 CFG 规则库与 AST 表示，并在节点级变异的基础上引入规则探测与剪枝机制。通过最小区分探测集 D 及多信号响应指纹 y 获取稳定的黑盒观测，再借助规则映射表 M 对 $(Token_i, Context_j)$ 的拦截风险 P_{block} 进行在线估计，并结合硬剪枝与软剪枝压缩搜索空间，本章为后续载荷搜索提供了可解释的风险先验与路径权重。尽管剪枝显著降低了冗余扩展，剩余空间仍具有有效样本稀疏与反馈噪声存在等特征，需要进一步在有限预算下实现更优的探索分配与收敛控制。基于此，下一章将以剪枝后的空间为对象，引入蒙特卡洛树搜索对载荷变异路径进行优化调度，并将 M 与路径权重作为选择策略的先验输入，以实现更稳定的黑盒检测效率提升。

第4章 基于蒙特卡洛树搜索的载荷空间优化搜索策略

4.1 引言

第三章在上下文无关文法 (CFG) 约束下构建了语义保留的对抗载荷空间, 并通过抽象语法树 (AST) 节点级变异保证载荷结构正确性。进一步地, 3.5 节引入规则探测与剪枝机制, 得到动态规则映射表 $M(Token, Context) \rightarrow P_{block}$, 并据此完成硬剪枝与软剪枝, 将大规模冗余分支提前过滤, 显著降低了搜索空间规模。

然而, 即便经过剪枝, 剩余空间仍存在两个关键问题: 有效样本稀疏以及反馈信号弱且噪声大。由于抽象语法树 (AST) 的节点变异组合呈指数级增长, 且生成的载荷必须严格遵循底层解析引擎的语法规则, 在黑盒反馈条件下, 可放行且保持语义的样本密度依旧较低, 简单随机或固定优先策略难以保证收敛效率。并且在云 WAF 场景下往往只能观测拦截/放行及有限响应特征, 容易受到限速、挑战页面、网络抖动等因素影响, 这些不可控的外生变量极易引发测试结果的状态污染与奖励误判, 导致变异算法难以持续提炼出对特定绕过手法的稳定评价指标, 使搜索引导更困难。相关 fuzzing 研究^[67]指出, 在高维输入空间与弱反馈条件下, 测试效率高度依赖搜索策略的资源分配机制。

为解决上述问题, 本章提出一种基于蒙特卡洛树搜索 (MCTS) 的载荷空间优化搜索策略, 在变异路径选择上引入探索与利用平衡机制, 将搜索过程建模为树结构决策问题。具体而言, 搜索树的节点表征载荷的中间变异状态, 边则代表具体的等价替换或多层编码算子, 借此系统能够完整记录并追溯每一条变异轨迹的收益状态。同时, 将第三章得到的规则映射表 M 、Risk-Reward 评分与漂移触发信号作为先验信息与动态调节依据, 在 UCB1/UCT 选择策略中体现剪枝结果的导向作用, 这一机制从根本上克服了传统 MCTS 在面对庞大状态空间时不可避免的冷启动盲目探索困境, 强力引导测试资源规避高风险区域。通过将全局统计先验与局部树节点的实时反馈相深度融合, 系统能够动态剥离无效的语法变异分支, 精准聚焦于规避性与语义有效性并存的脆弱子空间, 从而在有限查询预算下实现更快的有效载荷收敛与更稳定的检测表现。该思路与近年在 fuzzing 种子调度中引入 MCTS/Monte Carlo Planning 的工作一致^[66], 即通过统计回报与上置信界控制在大规模候选空间中的资源分配。最终在高度不确定的黑盒对抗环境中构建出稳健的自动化评估方法。整体的流程规划如图4-1所示。



图 4-1 基于 MCTS 的在线自适应载荷搜索流程

Fig.4-1 MCTS-Based online adaptive load search process

4.2 基于 MCTS 的载荷搜索空间优化算法设计

在完成载荷语法空间构建与剪枝之后, 剩余空间不再是无约束的随机字符串组合, 而是一个由 CFG 产生式与 AST 节点变异共同决定的结构化空间。该空间仍然具有明显的组合爆炸特征, 同一条攻击语义可能对应大量等价表达, 且不同上下文位置会放大这种差异。若继续采用随机或固定优先级策略分配查询预算, 效率瓶颈仍会很突出。本章将载荷变异路径选择视作一个序列决策问题, 通过蒙特卡洛树搜索 (MCTS) 在有限预算下实现更稳定的资源分配, 使搜索更快集中到更可能放行且同时仍具备语义有效性的结构区域。

本文将注入视为一个可控的执行模块, 而非简单的网络请求发送。注入执行模块以搜索树动作序列 (如产生式展开、节点变异、编码链选择) 作为输入, 将中间状态的 AST 线性化为 payload, 并根据所选 $Context_j$ 完成字段嵌入与请求构造。当 $Context_j$ 为 URL/Query 时, 将 payload 置于参数值并按编码链 $Enc(\cdot)$ 处理; 当 $Context_j$ 为表单或 JSON/XML 请求体时, 将 payload 注入目标字段并保持格式可解析; 当 $Context_j$ 为 Header/Cookie/multipart 等时, 则按相应语法规则插入并处理分隔符与边界。模块输出包括 allow/block 判定、多信号响应指纹 y (包括状态码、时延、长度、头部差异、重定向、连接状态等), 以及语义触发判定结果, 用于回报计算与策略更新。该模块使第三章的结构化变异能够以上下文一致的方式注入到真实 HTTP 事务中, 从而将黑盒评估过程落实为可复现的执行闭环。

MCTS 之所以适合这一任务, 是因为它不依赖目标函数的显式梯度, 也不需要白盒可见的规则细节, 仅利用黑盒反馈即可逐步形成哪些变异路径更值得继续投

入预算的统计偏好。语义保持载荷搜索的总算法骨架如算法 4-1 所示。

算法 4-1 基于 MCTS 的载荷空间优化搜索

Data: Initial State s_0 , Prior $\Phi(s, a)$, Action Set $A(s)$, Budget B , Top-k k ,
 Reward Function $r(\cdot)$, Shaping Reward $\tilde{r}(\cdot)$

Result: Best payload set $\mathcal{P}^*$

```

1: Initialize tree  $T$  with root  $s_0$ ;
2:  $\mathcal{P}^* \leftarrow \emptyset$ ;
3: for  $iter = 1$  to  $B$  do
4:    $s \leftarrow s_0$ ;
5:    $\pi \leftarrow \text{EmptyPath}$ ;
6:   // Selection: 按 UCT' 从根走到可扩展叶子
7:   while  $\text{FullyExpanded}(s)$  and not  $\text{Terminal}(s)$  do
8:      $a \leftarrow \text{SelectByUCTPrime}(s, \Phi)$ ; // 见算法 4-2
9:      $\pi \leftarrow \pi \oplus a$ ;
10:     $s \leftarrow \text{NextState}(s, a)$ ;
11:  end while
12:  // Expansion: 扩展 top-k 候选动作
13:   $\text{ExpandTopK}(s, k, \Phi)$ ; // 见算法 4-2
14:  // Simulation: 从 AST 状态实例化 payload 并执行请求
15:   $\text{payload} \leftarrow \text{InstantiatePayload}(s, \pi)$ ; // 见算法 4-3
16:   $y \leftarrow f(\text{SendHTTP}(\text{payload}))$ ;
17:   $\text{reward} \leftarrow r(y, \text{payload}) + \tilde{r}(y, \text{payload})$ ; // 严格成功 + 轻量 shaping
18:  // Backpropagation: 回传更新统计量
19:   $\text{Backpropagate}(T, \pi, \text{reward})$ ; // 更新  $N, W, \bar{Q}$ 
20:  if  $\text{reward}$  is SUCCESS then
21:     $\mathcal{P}^* \leftarrow \mathcal{P}^* \cup \{\text{payload}\}$ ;
22:  end if
23:  // Online Adaptation: 更新映射表  $M$ 、风险、漂移检测与子树衰减
24:   $\text{AdaptiveUpdate}(T, y, \text{payload})$ ; // 见算法 4-4
25: end for
26: return  $\mathcal{P}^*$ ;

```

4.2.1 搜索树节点状态设计与属性定义

本章将载荷变异过程抽象为一棵搜索树 T 。树中每个节点对应一个当前载荷状态，其本质为在 CFG 约束下可解析的 AST 表示。为了使搜索过程能够显式利用第三章的先验信息，节点状态不仅包含载荷结构，还包含风险与收益估计。MCTS

的树并非对 URL 或请求序列的枚举，而是对载荷变异决策序列的枚举。一个节点状态 s 需要能够唯一刻画当前处于何种载荷结构位置与变异上下文，同时又不至于维度爆炸。实践中更适合使用结构化摘要的方式定义状态，定义状态 s 由 AST 的局部结构片段与上下文位置共同组成，如公式(4-1)所示。

$$s = \langle AST, Context, \pi_r, \pi_g, \kappa \rangle \quad (4-1)$$

式中, AST 表示当前载荷的抽象语法树; $Context$ 表示载荷注入的上下文位置 (URL 参数、请求体字段、Header、Cookie 等); π_r 表示风险先验, 由规则映射表 M 聚合得到; π_g 表示收益先验, 由 Risk-Reward 评分或结构增益估计得到; κ 表示响应指纹簇标签或其统计分布, 用于区分不同拦截模式。其中风险先验可通过 π_r 得到, 如公式(4-2)所示。

$$\pi_r = \text{Agg}(M(\text{Token}_i, \text{Context})) \quad (4-2)$$

式中将当前 AST 所包含的关键 token 节点集合映射为拦截概率估计, 如取最大风险、加权平均或乘积等。收益先验可由第三章的 $\text{Score}(\text{path})$ 得到, 如公式(4-3)所示。

$$\text{Score}(\text{path}) = \lambda \cdot (1 - P_{\text{block}}) + (1 - \lambda) \cdot \text{Gain}(\text{path}) \quad (4-3)$$

将状态 s 可执行的动作记为 a , 动作 a 对应一类语义保留变异算子 (节点级替换、编码层级变化、空白/注释扰动等), 并且受到第三章硬剪枝规则约束, 对 P_{block} 近似为 1 的路径不再展开相应动作分支, 从而保证动作集合规模可控。为了便于计数与回溯, 动作通常以算子类型、作用节点、参数的形式编码, 使得不同动作实例可以被唯一识别并累计访问次数与回报。

从第三章的结果中可以得到两个量, 其一是规则映射表 M , 表示某类 token 在某类上下文下被拦截的概率估计。映射表的输出记为 $P_{\text{block}} \in [0, 1]$, 数值越大表示越可能被 WAF 拦截。其二是软剪枝得到的路径权重 $w(\pi)$, 它反映某条变异路径在当前阶段的探索价值。 $w(\pi)$ 并不等价于低风险路径优先, 它更强调值得探索的结构方向。一些路径可能风险较高但更有机会触发解析边界或规则缺口, 此时软剪枝往往不会直接删除它, 而是通过权重控制其被选择的频率。

4.2.2 选择策略与 UCT 评估机制

MCTS 的选择阶段用于在状态 s 的可选动作集合中决定下一步优先尝试的动作。经典做法是 UCT, 其核心思想是把多臂赌博机中的上置信界 (UCB) 思想用于树搜索, 使搜索能够在利用已知高回报动作与探索未知动作之间保持平衡。UCT

的典型形式如公式(4-4)所示。

$$UCT(s, a) = \bar{Q}(s, a) + c \cdot \sqrt{\frac{\ln(N(s) + 1)}{N(s, a) + \varepsilon}} \quad (4-4)$$

式中, $N(s, a)$ 表示动作 a 在状态 s 下被选择的次数, $N(s)$ 表示状态 s 被访问的总次数, ε 是防止分母为零的平滑常数, 由于 MCTS 在扩展新动作时会出现 $N(s, a) = 0$ 的情况, 如果没有 ε , 探索项会发散为无穷, 造成数值不稳定甚至使选择过程不可控。引入 ε 后, 新动作仍然会因为访问次数小而被优先探索, 但不会在数值上产生极端放大。其中, ε 的取值不需要刻意调参, 其作用接近数值保护项。通常取一个极小值即可, 例如 $\varepsilon = 10^{-6}$ 或 $\varepsilon = 10^{-8}$ 。在实践实现中, 只要保证 ε 相对 $N(s, a)$ 的典型量级足够小, 它不会对选择排序产生实质影响, 而仅仅起到稳定计算的作用。参数 $c > 0$, 其作用是调节探索项相对于平均回报项 $\bar{Q}(s, a)$ 的权重。直观而言, c 越大, 算法越倾向于选择那些访问次数 $N(s, a)$ 较小的动作; c 越小, 选择更依赖历史回报 $\bar{Q}(s, a)$, 搜索会更快集中在当前看起来有效的少数动作上。平均回报 $\bar{Q}(s, a)$ 表示动作 a 的经验有效性, 本文将其实定义为累计回报与访问次数之比, 如公式(4-5)所示。

$$\bar{Q}(s, a) = \frac{W(s, a)}{N(s, a)} \quad (4-5)$$

式中, $W(s, a)$ 为累计回报总和。这样定义的优点是更直观, 如果回报是 0/1 二值, $\bar{Q}$ 可以直接理解为成功比例的经验估计。在 WAF 黑盒绕过测试中, c 的设置与搜索空间规模和奖励稀疏程度关系密切。若载荷空间较大、规则映射尚不稳定、或绕过样本极其稀少, 过早降低探索会导致搜索陷入局部路径并产生偏置, 此时更适合使用较大的 c 。相反, 当映射表已经较稳定, 且已出现一定数量的成功样本, 继续保持过强探索会浪费预算并降低收敛速度, 此时可适当减小 c , 让搜索更偏向利用。在 UCT 的原始工作里, 选择策略源自 UCB1 的上置信界推导, 要求奖励是有界的。在这种设置下, 探索项的常见理论建议是 $c = \sqrt{2}$, 很多实验中会把它当默认起点。这一点在 UCT 经典论文以及 UCT 的通行解释中都有明确说明^[55]。但大量 MCTS 改进与综述均指出^[68], 探索常数在不同任务、不同分支规模与不同奖励稀疏度条件下需要经验调整, 并且不存在统一的全局最优值。在本文的情景中, 若以二值回报 (成功 = 1, 失败 = 0) 为主, 且每轮预算在千级请求量, $c \approx 1.0 \sim 1.4$ 往往能取得较好的平衡; 当预算明显更小, 例如仅几百次请求时, 可以适当降低 c , 避免大量预算消耗在短期无效探索上。

但是将传统的 UCT 公式直接用于 WAF 绕过会遇到一个问题, 有效绕过样本往往极少, 很多动作在相当长一段时间内都得不到正反馈, 导致 $\bar{Q}$ 接近 0, 选择主要被探索项驱动, 行为近似随机。然而第三章提供的规则映射与剪枝权重可以缓

解这一现象，因此本文将第三章先验显式注入选择阶段，构造带先验偏置的 UCT 公式如公式(4-6)所示。

$$UCT'(s, a) = \bar{Q}(s, a) + c \cdot \sqrt{\frac{\ln(N(s) + 1)}{N(s, a) + \varepsilon}} + \beta \cdot \Phi(s, a) \quad (4-6)$$

UCT' 在 UCT 的基础上增加了先验项 $\Phi(s, a)$ ，参数 $\beta \geq 0$ 位于 $UCT'(s, a)$ 的先验项之前，其作用是控制第三章先验信息（即规则映射表与软剪枝权重）对选择策略的影响程度。由于黑盒 WAF 场景中早期回报往往非常稀疏，如果完全依赖 $\bar{Q}(s, a)$ ，树策略会长期处于近似随机的探索状态；先验项的意义在于用结构风险与可探索性为搜索提供方向感，但它也可能带来先验偏置风险，即映射表尚不稳定时过度依赖先验会错过真实缺口路径。因此， β 的选取原则可以概括为：早期偏小、后期可增大。当搜索刚开始、映射表 M 仍在快速变化时， β 不宜过大；当系统已经通过多轮反馈形成稳定的 κ 模式与拦截概率估计时， β 可以逐步提高，使预算更快聚焦到高潜力结构区域。

先验项的构造需要与第三章的语义空间与剪枝机制一致，因此本节将其定义为路径权重与风险惩罚的组合，如公式(4-7)所示。

$$\Phi(s, a) = \log(w(\pi \oplus a) + \delta) \cdot (1 - \hat{P}_{block}(s \oplus a)) \quad (4-7)$$

式中， $\pi \oplus a$ 表示在当前变异路径 π 上追加动作 a 得到的新路径， δ 为对数平滑常数，用于避免 w 过小时的数值问题。因为 $w(\pi)$ 的来源是软剪枝权重，它在某些路径上可能非常小，此时若直接取 $\log(w)$ ，会对这些路径施加极强惩罚，使其几乎不可能再被选择，与软剪枝保留低概率探索的设计目标相矛盾。引入 δ 后，权重极小的路径仍会被降低优先级，但不会被数值意义上的无限惩罚彻底压死。与 ε 类似， δ 的主要作用是数值稳定与策略连续性，而不是依赖调参产生策略差异。一般可取 $\delta = 10^{-3}$ 或 $\delta = 10^{-4}$ ，使其相比典型的 $w(\pi)$ 量级足够小，但又能在 $w \rightarrow 0$ 时提供合理下界。 $\hat{P}_{block}(s \oplus a)$ 表示执行动作 a 后到达的新状态的风险估计。这样构造的直觉是明确的，如果某个动作扩展后仍然处在高拦截风险区域， $(1 - \hat{P}_{block})$ 会压低先验得分；如果软剪枝认为该路径值得探索，先验会鼓励该动作更早获得预算，从而加速搜索靠近有效区域。本节的 UCT 选择策略方法如算法 4-2 所示。

算法 4-2 UCT' 选择与 Top-k 扩展 (Selection & Expansion)

Data: State s , Candidate Actions $A(s)$, Statistics $N(s), N(s, a), \bar{Q}(s, a)$,

Prior $\Phi(s, a)$, Constants c, β, ε , Top-k k

Result: Selected action a^* , Expanded children of s

1: **function** SelectByUCTPrime(s, Φ):

算法 4-2 UCT' 选择与 Top-k 扩展 (续)

```

2:   $best\_a \leftarrow \text{None}; best\_score \leftarrow -\infty;$ 
3:  for  $a$  in  $A(s)$  do
4:    if  $N(s, a) = 0$  then
5:       $score \leftarrow +\infty;$  // 未探索动作优先被采样
6:    else
7:       $exploit \leftarrow \bar{Q}(s, a);$ 
8:       $explore \leftarrow c \cdot \sqrt{\ln(N(s) + 1) / (N(s, a) + \epsilon)};$ 
9:       $prior \leftarrow \beta \cdot \Phi(s, a);$ 
10:      $score \leftarrow exploit + explore + prior;$ 
11:    end if
12:    if  $score > best\_score$  then
13:       $best\_score \leftarrow score;$ 
14:       $best\_a \leftarrow a;$ 
15:    end if
16:  end for
17:  return  $best\_a;$ 
18: end function
19: function ExpandTopK( $s, k, \Phi$ ):
20:  // 以 UCT' 的当前评分为依据, 从  $A(s)$  选择 top-k 动作加入树
21:   $scores \leftarrow \text{EmptyList};$ 
22:  for  $a$  in  $A(s)$  do
23:     $score\_a \leftarrow \text{UCTPrimeScore}(s, a, \Phi);$ 
24:     $scores.append((a, score\_a));$ 
25:  end for
26:   $topk\_actions \leftarrow \text{TopK}(scores, k);$ 
27:   $\text{AddChildrenToTree}(s, topk\_actions);$ 
28: end function

```

先验项与探索项在公式中发挥着不同的作用, 探索项强调没试过的动作应该试一试, 先验项强调有理由相信的动作可以先多试几次。两者叠加后, 系统既不会因为先验存在而完全忽略新分支, 也不会因为奖励稀疏而长期随机游走。

4.2.3 扩展、模拟与回溯机制

选择策略确定了下一步优先尝试的动作后, MCTS 需要将该动作对应的新节点纳入搜索树, 这一过程称为扩展。由于第三章的语义保留变异算子组合空间很大, 若扩展阶段一次性加入所有候选动作会导致分支爆炸。为控制树规模, 本文采

用受控扩展的原则，在状态 s 的动作集合中只扩展一部分候选动作，候选集合可根据 $UCT'(s, a)$ 或先验评分 $\Phi(s, a)$ 排序得到的 $\text{top-}k$ 动作构成。该策略的目的不是牺牲搜索充分性，而是在预算受限条件下避免把计算资源消耗在明显无效的枝叶上。

扩展后的新节点通常需要通过模拟（rollout）产生一次完成的测试请求并获得反馈。rollout 是指从新节点出发，在不进一步展开整棵树的情况下，用一个较轻量的策略执行若干步变异直到达到终止条件，最终生成 payload 并发送请求。终止条件通常由最大变异深度、最大编码层级或当轮预算限制决定。rollout 策略若完全随机，会导致大量无意义尝试；若过度贪心，则会把探索空间压缩得过窄。本文将采用弱先验引导的 rollout，当风险估计较高时倾向于选择可降低风险的编码与等价替换动作，当风险适中时允许插入适量扰动以保持多样性。这样设计能够在不显著增加计算成本的前提下，提高模拟阶段得到信息量反馈的概率。

模拟结束后进入回溯阶段。回溯的任务是把一次模拟的结果写回到经过的节点统计量中，使后续选择更具依据。本文将单次请求的基础回报定义为 r ，如公式(4-8)所示。

$$r = \mathbb{I}[\text{allow}] \cdot \mathbb{I}[\text{semantic hit}] \quad (4-8)$$

式中 $\mathbb{I}[\text{allow}]$ 表示 WAF 是否放行， $\mathbb{I}[\text{semantic hit}]$ 表示后端语义是否触发。在黑盒测试中，它的取值来自响应行为判定，当请求未被规则拦截、未触发挑战/限速且能够获得有效业务响应时， $\mathbb{I}[\text{allow}] = 1$ ；当出现明显拦截，例如 403/406、连接复位、持续超时或被识别为挑战簇 κ 时， $\mathbb{I}[\text{allow}] = 0$ 。这种定义避免了把挑战页返回 200 误判为放行，从而保证绕过判定的严谨性。 $\mathbb{I}[\text{semantic hit}]$ 则表示 payload 是否在后端触发了目标攻击语义。它的取值必须按攻击类型分别定义，否则会出现 WAF 放行但攻击无效的假成功。对于 SQL 注入，本文采用布尔差分或响应差分作为语义成立判据，构造语义相反的测试对 True/False，若响应内容或结构差异达到阈值则判定语义触发；对于 XSS，可采用反射确认 payload 在响应 HTML 中可见作为最低判据，并在可控靶场下结合执行回调增强可信度；对于命令注入，则可使用回显匹配或后端延迟证据（如 sleep 语义）进行判定。其他类型攻击方法也可以构造相应的触发条件作为相应的判断依据。

式(4-8)中采用乘积形式并不是形式上的选择，而是为了将成功定义为逻辑与（AND）。如果只用 $\mathbb{I}[\text{allow}]$ 作为成功，就会把大量放行但无效的请求计入成功。如果只用 $\mathbb{I}[\text{semantic hit}]$ ，则会把被拦截的尝试计作成功，破坏绕过任务的真实性。乘积形式保证了成功样本必须同时满足两项条件。该定义强调绕过不等于 HTTP 200，避免将后端无效请求误计为成功。同时回溯更新的方式采用标准累计形式，如公

式(4-9)所示。

$$N(s, a) \leftarrow N(s, a) + 1, \quad W(s, a) \leftarrow W(s, a) + r, \quad \bar{Q}(s, a) = \frac{W(s, a)}{N(s, a)} \quad (4-9)$$

此式表示每一次尝试都会增加该动作的访问次数。若成功，则累计回报增加，平均回报随之更新为成功比例的经验估计。

在奖励稀疏的黑盒环境中，完全的二值奖励可能导致学习信号过弱。因此本文引入轻量奖励塑形，用作靠近成功区域的弱引导。塑形信号来自第三章的多信号响应指纹 y 及其归纳模式 κ 。当请求未触发语义成功，但其响应模式与历史成功样本所在簇相近，或出现明显的延迟代价异常，系统可以给予一个幅度较小的附加奖励 $\tilde{r}$ ，以鼓励树对该方向保持一定探索。由于在真实环境中，可能会出现成功很少的现象，如果所有失败都给 0，树在很长时间里得不到区分信息，收敛速度会很慢。于是使用附加奖励 $\tilde{r}$ ，允许对接近成功或信息量很大的反馈给一点很小的正奖励，用来引导搜索方向。因此本文回报可以写成更完整的形式，如公式(4-10)所示。

$$r_{total} = r + \tilde{r} \quad (4-10)$$

式中 $r \in \{0, 1\}$ 是基础回报，表示真实成功 $\tilde{r} \in [0, \rho]$ 是附加奖励， ρ 是一个很小的正值。这种基于奖励塑形的机制有效缓解了零一离散回报带来的搜索停滞与盲目试错问题，为算法提供了连续且密集的优化梯度。综上所述，基于 MCTS 的载荷搜索空间优化算法可将路径 π 变成最终 payload，AST 到 Payload 的实例化算法流程如算法 4-3 所示。

算法 4-3 AST 到 Payload 的实例化

Data: Initial AST T_0 , Mutation Path $\pi = \langle a_1, a_2, \dots, a_m \rangle$,

Node-level Mutators Ω , Encoder $Enc(\cdot)$

Result: Concrete payload string *payload*

```

1:  $T \leftarrow \text{Copy}(T_0)$ ;
2: for each action  $a_i$  in  $\pi$  do
3:    $(node\_v, operator\_op) \leftarrow \text{DecodeAction}(a_i)$ ;
4:    $T \leftarrow \text{ApplyMutation}(T, node\_v, operator\_op)$ ; // 节点级语义保持变异
5: end for
6:  $payload\_raw \leftarrow \text{Linearize}(T)$ ; // AST 线性化为字符串
7:  $payload \leftarrow \text{Enc}(payload\_raw)$ ; // 可选：多层编码/上下文编码
8: return  $payload$ ;
```

4.3 动态自适应策略更新机制

在真实部署环境中，WAF 的拦截逻辑并不是静态不变的。云 WAF 往往以规则订阅、热点策略下发、阈值动态调节等形式持续演进，同一类 payload 在不同时段可能出现截然不同的行为反馈。若将第三章探测阶段得到的规则映射表 M 视为一次性结论，并在整个测试过程中保持不变，搜索过程会逐渐积累偏差。一部分已经被更新规则封锁的路径仍被反复尝试，导致查询成本上升。另一部分新出现的薄弱路径则难以及时获得预算。为保证搜索策略在规则漂移与行为噪声并存的条件下仍能稳定收敛，本节引入动态自适应更新机制，使映射表 M 、路径权重 $w(\pi)$ 与搜索树统计量能够根据实时反馈持续校准。

这种思路在概念漂移检测研究中具有明确的工程动机^[69]，当系统只能观测到输入和输出行为而缺少显式标签时，必须通过分布变化与行为统计异常来触发更新，而不是等待长期累积误差后再整体重启。无监督漂移检测的研究工作^[70]对这一问题给出了较系统的分类与实践建议。同时，从信息熵与不确定性的角度刻画漂移的研究也表明^[71]，仅使用单一信号难以可靠识别漂移，需要结合多维观测特征构建稳定的变化判据^[72]。

4.3.1 实时反馈监控与成功载荷解析

动态更新的基础是记录足够多的信息，但不过度依赖单一信号。本研究在每次请求后保留放行/拦截结果，并记录第三章定义的多信号响应指纹 y ，再将 y 归类为响应模式 κ 。采用 κ 的原因在于，实际环境中常见的挑战页、限速、连接复位等行为可能造成状态码与响应体表现不稳定，如果仅依据 200/403 更新映射表， P_{block} 容易被噪声样本污染。将响应归入模式簇后，可以对明显属于策略性流控的 κ 降权处理，提高映射表更新的稳定性。

当出现绕过成功样本时，系统会回溯其变异路径 π 并重新解析为 AST，提取导致成功的关键结构片段与动作组合。该信息被用于两项更新，一是将相关 token-context 组合在 M 中的风险估计向更容易放行方向校准，二是提升相似状态下相似动作的先验评分，使树搜索能够更快复现成功结构，而不是仅记录一条孤立字符串。例如，对于同一类上下文 $Context_j$ ，若多次成功样本都依赖关键字等价替换、多层编码与注释截断这一结构组合，则该组合对应的状态与动作对会在树统计量中获得更高的平均回报 $\bar{Q}(s, a)$ ，并在后续选择中更频繁地被采样。类似思想在 MCTS 中体现为成功路径的结构关联可被抽象为树上的优势分支，从而提升预算分配效率。这种结构化归因的更新方式比记住成功 payload 文本更具可迁移性，也更符合本研究的语义空间建模初衷。

4.3.2 节点风险等级动态评估

规则映射表 M 提供了某 token 在某上下文下被拦截的概率估计，记为 $P_{block} \in [0, 1]$ 。为了让映射表能参与树搜索决策，需要将其聚合为状态层面的风险估计 $\hat{P}_{block}(s)$ 。这里的 s 是搜索树节点状态，它由当前上下文与 AST 结构摘要共同决定，因此 $\hat{P}_{block}(s)$ 可以理解为该结构在该上下文下被拦截的倾向。 $\hat{P}_{block}(s)$ 的值越大，表示继续沿该结构方向扩展越可能触发 WAF 拦截。

然而， $\hat{P}_{block}(s)$ 属于先验估计，它的可靠性取决于探测阶段的样本覆盖与当前规则是否发生漂移。与之互补的是树搜索的经验回报 $\bar{Q}(s, a)$ ，它由真实请求反馈累计得到，反映某动作在当前环境下的实际有效性。为了在动态环境中兼顾先验与经验，本研究引入融合风险 $R(s, a)$ ，用于对状态—动作对给出更稳健的风险等级，如公式(4-11)所示。

$$R(s, a) = \lambda \cdot \hat{P}_{block}(s \oplus a) + (1 - \lambda) \cdot (1 - \bar{Q}(s, a)) \quad (4-11)$$

式中， $s \oplus a$ 表示在状态 s 上执行动作 a 后到达的新状态， $\hat{P}_{block}(s \oplus a)$ 来自第三章规则映射表 M 的先验拦截倾向，数值越大，表示更可能被 WAF 拦截。 $\bar{Q}(s, a)$ 是动作 a 的平均回报，来自树搜索统计量的经验成功水平（也可以理解为这个动作在当前环境中有多可信）。 $\bar{Q}$ 越大表示越有效，因此 $1 - \bar{Q}(s, a)$ 表示经验失败倾向，回报越差，该项越大。 $\lambda \in [0, 1]$ 是这两类证据的融合系数，用于控制先验与经验的权重比例。当 $\lambda \rightarrow 1$ 时，系统几乎完全依赖映射表先验，风险判断主要由 P_{block} 决定；当 $\lambda \rightarrow 0$ 时，系统主要依赖经验统计，风险判断接近于失败越多越不选。 λ 会根据情况进行动态调整，如果 λ 固定不变，系统很容易落入两个不理想状态；如果 λ 长期偏大，系统会持续避开它认为高风险的结构，结果是漂移后依然沿旧路径走，修正速度很慢；如果 λ 长期偏小，搜索早期 $\bar{Q}(s, a)$ 几乎没有统计意义，因为 $N(s, a)$ 很小，成功回报稀疏且噪声较大，风险主要由 $1 - \bar{Q}(s, a)$ 决定，系统会在大量动作间无差别摇摆，收敛速度下降。

经验项 $\bar{Q}(s, a)$ 是否可信，核心取决于访问次数 $N(s, a)$ 。如果只试了 1-2 次， $\bar{Q}$ 波动非常大；如果试了几十次， $\bar{Q}$ 的方差会显著收敛。因此，本文提出基于观测置信度的自适应融合系数 $\lambda(s, a)$ ，把 λ 设计为关于 $N(s, a)$ 的递减函数，如公式(4-12)所示。

$$\lambda(s, a) = \frac{n_0}{n_0 + N(s, a)} \quad (4-12)$$

式中 $n_0 > 0$ 是一个先验强度常数， n_0 来自于第三章映射表先验 P_{block} ，表示先验映射表对单个 (s, a) 的等效样本量，它决定了经验统计需要积累到什么程度，系统才会从先验主导转向经验主导。如果真实环境中的 WAF 环境噪声更大，如挑战或限

速多，可以取更大的 n_0 ，让先验保持影响更久；如果探测阶段质量高且漂移频繁，可以取小一些，让经验更快接管。

因此当 $N(s, a)$ 较小时，经验回报 $\bar{Q}(s, a)$ 的统计不稳定，系统通过较大的 λ 更多依赖 P_{block} 。随着 $N(s, a)$ 增大，经验估计逐步可靠， λ 单调下降，风险评估自然转向数据驱动。特别地，当 $N(s, a) = n_0$ 时有 $\lambda = 0.5$ ，此时先验与经验贡献相当，因此 n_0 也可视为先验主导向经验主导的切换尺度。

在云 WAF 漂移情形下，完全依赖先验会放大误差，完全依赖经验则会在早期反馈稀疏时退化为随机游走。融合风险提供了一种折中机制，先验用于提升早期方向感，经验用于在规则变化后快速纠偏。这种处理保证了高风险路径不会被过早放弃，从而为潜在规则缺口保留探索空间，这一点与软剪枝不一刀切的原则完全一致。

4.3.3 面向 WAF 行为变化的动态适应机制

由于在线更新只能解决缓慢变化的偏差累积问题，但在云 WAF 的突发更新情形下，仅依靠渐进更新可能会出现短期完全失效的现象，原本成功率较高的结构方向突然全线失败，树仍在旧分布附近消耗预算。为了在这种情况下以较低成本恢复有效搜索，本节引入概念漂移触发的局部重探测机制。

漂移检测的作用是一个开关，当系统判断当前 WAF 行为已经发生结构性变化时，就要触发局部重探测与映射表校准，把搜索从错误方向拉回可收敛状态。无监督漂移检测在工业监控场景里非常常见，Hinder 等^[73,74] 对这类设定做过系统性梳理，强调了仅靠行为反馈、无标签条件下，需要比较时间窗口的分布差异来触发漂移响应。漂移检测的输入不依赖标签，而来自响应模式簇 κ 的统计分布。系统需要维护两个时间窗口，历史窗口 W_{old} 与近期窗口 W_{new} 。窗口中记录的是响应模式标签序列或其频率分布。窗口长度取决于希望检测的漂移尺度，依照实际 WAF 情况选取一个固定值固定窗口长度，建议选取范围在 50-200 之间，若窗口过小，则会导致噪声过大，挑战/偶然超时会导致误报；若窗口过大，则漂移发生后反应慢，恢复时间变长。同时令 $p_{old}(\kappa)$ 表示在 W_{old} 内各响应模式的概率分布， $p_{new}(\kappa)$ 表示在 W_{new} 内的分布。漂移检测的目标是判断这两个分布是否发生了显著变化。为了实现这一点，可以使用对称且稳定的分布距离作为漂移度量，本文中选取 Jensen-Shannon 距离 D_{JS} 衡量近期窗口与历史窗口的响应模式分布差异。该距离具有对称、有界与数值稳定等特性，适用于对离散模式标签 κ 的分布变化进行无监督监控。并且这种基于 JS 的滑动窗口漂移检测在数据流漂移研究中很常见，已经有工作直接使用 JS divergence 作为窗口分布差异度量来触发漂移。并且本文中需要检测的漂移本质是 WAF 行为模式的变化，例如，直接拦截簇比例上升、挑战/限速

簇突然增多、放行簇快速下降、后端错误簇出现新的占比等，这些变化正好对应分布差异度量擅长捕捉的对象。Sun 等^[75]从不确定性/熵角度讨论漂移检测，也强调了在无监督场景下用信息论指标刻画变化具有可操作性。因此，当满足如公式(4-13)所示条件时，触发校准过程。

$$D_{JS}(p_{new}(\kappa), p_{old}(\kappa)) > \tau_d \quad (4-13)$$

式中， τ_d 是漂移阈值，用于控制触发灵敏度。阈值 τ_d 的设定可以通过无漂移基线统计得到，在规则相对稳定的一段区间内，计算多组窗口的 D_{JS} 分布，取其高分位数作为阈值。无监督漂移检测研究指出，在缺少标签的监控场景中，分布距离与不确定性度量是有效且通用的漂移触发方式。漂移触发后，系统并不重启整个搜索树，而是执行局部重探测。这里的关键是控制代价，如果漂移后重新探测全空间，成本会接近重新训练，失去在线适应意义。因此本文在第三章最小区分探测集 D 的基础上，优先选择那些在近期失败率显著上升、或在映射表 M 中不确定性较高的 token-context 子区域作为校准对象。校准过程的输出是对映射表局部项 $M(Token_i, Context_j) \rightarrow P_{block}$ 的修正，从而更新 $\hat{P}_{block}$ 的聚合结果。由于映射表改变会影响先验项与风险融合，树的选择策略会随之自动调整，快速从旧失效路径转移到更可能有效的新区域。

在执行局部重探测机制时，系统需要区分真实的规则集更新与由于挑战或限速机制引发的假漂移现象。当测试请求的频率或特征触发了云 WAF 的流控阈值时，目标系统通常会主动进入挑战防御模式。在此期间，响应特征与拦截指标 κ 的分布会发生显著的统计偏移。然而，这种偏移主要反映的是 WAF 流量清洗策略的介入，而非底层正则表达式或语义检测引擎的实质性变更。若评估系统将此种瞬态流控误判为概念漂移并触发重探测，不仅会消耗有限的查询预算，还可能导致先验映射表引入大量噪声。为规避此类误触发，本文将漂移检测逻辑与网络及应用层的连接状态特征进行耦合。具体而言，当监控到近期滑动窗口内出现的连接超时、TCP 复位以及特定 HTTP 响应，如 JavaScript 质询、验证码重定向的比例异常上升时，标志着目标防护机制正试图校验客户端的真实浏览器指纹或 IP 信誉度。此时若继续高频探测，极易触发更严厉的长期封禁。因此，系统赋予流控假设更高的置信度，从而挂起规则更新判定。在此状态下，系统将自适应地切换至防御性节流策略，通过降低并发线程数并施加指数退避算法来延长请求间隔。这种主动降级的节流策略不仅能有效稀释攻击请求的时间密度，促使目标系统的信誉惩罚周期自然冷却，同时也能为后续探测恢复合法的会话状态。待目标 WAF 的响应特征恢复至常规平稳状态后，系统再恢复漂移度量的统计工作，以此有效规避在非正常环

境下的无效校准开销。

另一方面，当多维特征确证拦截分布的变化确由 WAF 后端防御规则的实质性更新引起时，如何处置历史搜索数据成为维持算法收敛效率的关键。在此场景下，已构建的蒙特卡洛搜索树结构依然具备较高的复用价值，因其节点间的拓扑关系记录了大量经过解析验证的结构化探索轨迹与语法等价推导过程。底层数据库或模板引擎的语法合法性规则通常是固定的，不会因 WAF 上层拦截规则的变更而失效。若采用全局重置策略，即清空并重建搜索树，将不可避免地导致系统面临冷启动问题，丢失前期消耗大量探测成本积累的载荷结构演化信息。因此，本文针对受影响的树节点引入了统计量软重置机制。对被判定受规则更新影响的子树节点，系统不对其进行物理裁剪，而是对其累积经验回报 $w(s,a)$ 与历史访问次数 $N(s,a)$ 施加特定的时间衰减系数。通过该乘性衰减操作，基于旧规则体系获取的经验权重在 UCT 决策公式中的比重被强制降低，使得失效的经验逐渐淡出评估过程，而非发生突变式的瞬间清零。该软重置策略既能防止失效的先验评估对当前动作选择产生长期误导，又能有效保留该子树在未发生规则漂移区域，如通用语法合法性维持的启发式价值，保障了算法在动态环境中的连续学习能力。从系统宏观运行逻辑来看，当真实漂移触发时，局部重探测模块首先以低成本快速刷新 Token-Context 映射表。随后，更新后的映射表反向修正树节点的先验风险期望与选择策略。最终，校准后的选择策略进一步引导底层变异算子执行后续的路径扩展与有效样本积累，从而在黑盒对抗环境中形成了一个具备高度鲁棒性的自适应检测闭环。综上所述，动态自适应策略更新机制的方法流程如算法 4-4 所示。

算法 4-4 在线自适应更新

Data: Search Tree T , New Observation y , Mode Cluster κ ,

Mapping Table M , Window W_{old}, W_{new} , Parameters $n_0, \tau_d, \gamma, \lambda_{min}$

Result: Updated tree statistics and mapping priors

```

1:  $\kappa \leftarrow g(y)$ ;
2: Push( $W_{new}, \kappa$ ); // 维护近期窗口
3: // Step-1: 更新映射表  $M$  (局部、轻量)
4: UpdateMByFeedback( $M, y$ ); // 更新  $P_{block}$  的局部估计
5: // Step-2: 更新融合系数  $\lambda$  (依赖访问次数)
6: for each visited edge  $(s, a)$  in current path do
7:    $\lambda(s, a) \leftarrow n_0 / (n_0 + N(s, a))$ ; // 经验越多，越偏向经验项
8:   UpdateRiskR( $s, a, M, \bar{Q}, \lambda$ ); //  $R(s, a) = \lambda \cdot \hat{P}_{block} + (1 - \lambda) \cdot (1 - \bar{Q})$ 
9: end for
10: // Step-3: 漂移检测 (基于  $\kappa$  分布)
11: if WindowFull( $W_{new}$ ) and WindowFull( $W_{old}$ ) then
12:    $p_{new} \leftarrow \text{Histogram}(W_{new})$ ;  $p_{old} \leftarrow \text{Histogram}(W_{old})$ ;

```

算法 4-4 在线自适应更新（续）

```

13:   $d \leftarrow \text{JensenShannon}(p_{new}, p_{old});$ 
14:  if  $d > \tau_d$  then
15:      // 漂移触发：降低先验权重，局部重探测校准
16:       $\text{SetGlobalLambdaLowerBound}(\lambda_{min});$ 
17:       $\mathcal{D}' \leftarrow \text{SelectiveReprobeSet}(M);$  // 从关键 token/context 选子集
18:       $\text{RunProbingAndUpdateM}(\mathcal{D}', M);$ 
19:      // 对受影响子树执行软重置（衰减统计量）
20:      for each affected edge  $(s, a)$  in  $T$  do
21:           $N(s, a) \leftarrow \gamma \cdot N(s, a);$ 
22:           $W(s, a) \leftarrow \gamma \cdot W(s, a);$ 
23:      end for
24:  end if
25: end if
26:  $\text{SlideWindow}(W_{old}, W_{new});$  // 滑动窗口
27: return;

```

4.4 本章小结

本章在第三章探测、映射与剪枝的成果之上，提出了面向语义保留载荷空间的 MCTS 优化搜索策略。节点状态以 AST 结构摘要与注入上下文为核心，动作集合与第三章变异算子保持一致，通过累计回报 W 、访问次数 N 与平均回报 $\bar{Q}$ 形成可解释的经验统计。选择阶段采用 UCT 作为基本框架，并将第三章的路径权重与风险估计以先验形式注入，得到带先验偏置的 $UCT'(s, a)$ ，缓解了黑盒场景中奖励稀疏导致的随机游走问题。扩展阶段采用受控扩展抑制分支爆炸，模拟阶段用弱先验 rollout 提升反馈信息密度，回溯阶段以放行且语义触发为统一成功口径更新统计量，确保树策略与第五章评估指标保持一致。

针对云 WAF 的动态性，本章进一步给出了在线更新与漂移适应机制，通过多信号指纹与响应模式 κ 提升反馈判定稳定性，通过融合风险 $R(s, a)$ 平衡先验估计与经验回报，通过漂移触发的局部重探测在较低成本下校准映射表与路径权重。至此，第三章与第四章在机制层面形成闭环，即先验信息引导搜索，搜索反馈修正先验，使得系统在有限预算与动态目标条件下仍具备稳定收敛能力。

第 5 章 实验评估与结果分析

5.1 实验环境与数据集

本章围绕基于载荷变异注入的 WAF 漏洞检测方法开展实验评估，目标在于验证统一语法规则库是否能够覆盖典型 Web 攻击语义、语义保留变异是否能够生成可执行的对抗性载荷，以及基于规则映射剪枝与 MCTS 搜索是否能够在给定预算内获得更高的有效绕过与更快的收敛速度。实验设计同时关注云 WAF 的动态性特征，在规则变化场景下评估策略的稳定性与恢复能力。

实验平台采用 Ubuntu 22.04 LTS，攻击生成与调度模块以 Python 实现，WAF 目标与被测 Web 应用以容器化方式部署。全程采用 Docker 容器隔离部署，杜绝宿主机环境干扰，保证实验环境的一致性与复现性，同时搭建独立日志采集模块，实时记录载荷发送、WAF 响应与应用执行状态。为了覆盖不同部署形态与规则风格，本研究选取四类代表性防护目标：开源规则引擎以 ModSecurity + CRS 作为典型代表、轻量级自定义 WAF 以 Nginx + Lua 规则为代表，以及两类具有动态策略更新特征的云 WAF（以 Cloud WAF-A 匿名表示）。这一设置的目的在于保证实验结论不依赖某一特定厂商实现，同时能够体现云环境中概念漂移对检测流程的影响。同时能规避单一防护场景带来的结论偏差，强化实验结果的普适性与说服力。近期对云 WAF 的绕过研究也指出，不同解析路径与请求语义分歧会显著影响实际拦截效果，因此实验在 WAF 侧与应用侧均保持真实请求解析链路。

被测应用侧选用包含可控漏洞点的开源靶场系统，以便在放行之外进一步判定是否真正触发攻击语义。实验以 DVWA 与 OWASP Juice Shop 作为主要靶场来源，这些系统覆盖 SQL 注入、XSS、命令执行等典型攻击类别，并具备明确的可复现漏洞入口。靶场均设置为高风险等级，还原真实业务场景下的漏洞暴露状态，排除低风险配置对实验结果的弱化影响。在此基础上，为避免结果仅对单一应用结构有效，本研究额外引入若干自构造 API 端点，用于控制输入位置（Query、Header、JSON、Form-data 等）与上下文组合，使第三章语法规则库中的 $Context_j$ 能够被充分验证。

载荷数据集由两部分构成：其一为基于 CFG 生成的规范攻击语句骨架，确保攻击语义完整且可解释；其二为在语义保持约束下进行节点级变异得到的对抗载荷集合，用于验证第三章提出的等价替换、编码变异、空白/注释干扰等算子对绕过效果的贡献。为了保证评估可复现，本章统一采用固定的查询预算 $B = 2000$ ，每一组“攻击类型 $\times$ WAF 配置”重复次数 $R = 30$ ，并使用相同的随机种子控制实验波动，使不同方法间比较具有可比性。

5.2 评估指标与对比方法

5.2.1 评估指标

本研究的评估指标围绕有效绕过、查询效率、收敛特性、语义有效性、漂移恢复能力构建，避免仅以 HTTP 结果判断绕过成功而造成的偏差。

有效绕过成功率：在给定预算 B 内，能够在 WAF 层获得放行且在应用侧判定为语义触发的样本比例 **Success Rate**，记为 SR 。这里将放行与语义触发联合定义，是为了过滤仅因 WAF 误判或后端错误导致的假成功。这一设置与近期针对 WAF 绕过研究中强调的解析差异与语义一致性的评估思路一致。

首次有效成功查询成本：从开始搜索到首次出现有效绕过样本所需的查询次数，用于刻画方法在有限预算下的启动效率与早期探索能力，记作 QC_{first} 。

收敛步数：在搜索过程中达到稳定有效成功率阈值，例如 SR 达到最终水平的某一比例所需的查询步数，用于刻画策略的收敛速度，记作 T_{rec} 。该指标能够体现剪枝先验 +MCTS 对搜索空间收敛的贡献。

语义命中率：在所有通过 WAF 的样本中，满足语义触发判定条件的比例，记作 **SemanticHit**。该指标反映载荷的可执行性，可用于衡量语义保留变异是否真正保持攻击意图，而非仅制造字符串层面的扰动。

漂移恢复时间：在模拟规则更新/行为漂移后，方法恢复到稳定有效成功率水平所需的额外查询量，记作 **Drift Recovery Time**。本指标用于评估第四章自适应机制与第三章映射表在线更新在云 WAF 场景下的实际价值。类似的动态策略变化现象在云 WAF 与真实防护中普遍存在，近期研究也强调了绕过策略在时间维度上的可持续性问题的。

5.2.2 对比方法

为了证明所提方法不是特定启发式的偶然优势，本研究设置了多类对比基线，主要分为三大类：搜索/调度策略基线、同类研究方法基线、工程工具基线。

第一类搜索/调度策略基线覆盖随机、启发式、进化式与学习式策略，并与无先验的 MCTS 进行对照。相关基线方法如下：

Random^[7]：在 CFG 生成空间内随机采样与变异，不引入映射表与搜索调度。

Greedy^[76]：优先选择历史上最常放行的结构分支，强调局部利用。

GA^[77]：采用遗传算法进行载荷变异与选择，适用于大规模组合优化场景。

RL(PPO)^[78]：采用强化学习调度载荷生成动作，利用奖励信号学习变异策略。

MCTS(no-prior)：采用标准 MCTS 搜索，但不注入第三章映射表先验。

MCTS+prior：本文提出的完整方案，即探测集 D 、响应指纹 y 、映射表 M 与

风险估计、剪枝/权重注入、MCTS 自适应搜索的完整方法。

MCTS 思路在 fuzzing 调度领域已经被验证具有有效性，例如 Alphuzz 将 seed-mutation tree 的调度建模为 MCTS 问题，证明上置信界机制能够提升探索效率。本研究在此基础上，将 MCTS 用于语法驱动的对抗载荷空间，并引入可学习风险先验，使其能够直接服务于 WAF 黑盒绕过场景。

除上述搜索/调度策略基线外，为增强横向可比性并回应真实攻防环境中的常用手段，本研究进一步引入同类研究方法基线与工程工具基线，均在相同目标集合、相同注入上下文集合 $\{Context_j\}$ 、相同预算 B 与重复次数 R 下运行，并统一采用本文的有效样本判定标准，需同时满足 allow 与 semantic hit，以避免仅以放行造成的假成功。

同类研究方法基线分为两类。其一为解析差异模板生成基线（WAFFLED 类），该基线围绕 content-type 与字段组织方式的解析分歧构造请求结构模板，例如在 JSON 与表单编码之间切换载荷承载方式、利用同名参数聚合差异、以及利用 multipart 边界与字段分段差异等。实验实现中，该基线采用固定模板集合 τ_{PD} 生成请求骨架，再将攻击片段注入预设字段并按统一编码链 $enc(\cdot)$ 处理后发送请求，采集响应指纹并进行语义触发判定。其二为协议层/规范化差异规避基线（WAFManis 类），该基线围绕 HTTP 归一化与解析顺序差异构造请求层面的结构变体，例如路径规范化与编码分隔符处理差异、重复 header 与空白折叠差异、以及参数解析与聚合顺序差异等。实验实现中，该基线采用固定模板集合 τ_{Proto} 生成请求行与头部/参数组织结构，再执行相同的攻击片段注入与编码链封装，从而对比固定结构模板规避与本文结构化变异、风险先验与搜索调度的综合效果差异。

工程工具基线选取真实渗透测试中常用的载荷生成与注入工具，用于刻画静态 payload 集在 WAF 前置环境下的有效样本密度与稳定性边界。SQLi 选用 sqlmap，XSS 选用 XSSStrike，CMDi 选用 commix，同时使用 Burp Suite Intruder 作为通用注入器基线。为避免工具差异来自爬取范围、认证推断或多阶段交互等非本文关注因素，工具仅用于 payload 生成与注入执行；目标路径、参数位置与上下文集合 $\{Context_j\}$ 与本文实验设置保持一致，并在相同预算 B 下统计 SR 、 QC 与收敛指标。

5.3 统一规则库有效性验证

本节验证第三章提出的统一攻击语法规则库在多攻击类型场景下的覆盖能力与可用性，重点关注两个问题。其一，CFG 规则能否稳定表达 SQLi / XSS / CMDi 的语义骨架，避免因语法定义缺失或歧义导致载荷语义失效，保障后续变异操作

的底层语法基础。其二，3.5 节构造的探测集 D 是否能够在较小查询代价下建立可用的规则映射表 M ，剔除云 WAF 响应噪声与偶发误判干扰，为后续剪枝优化与 MCTS 搜索提供高可信度的先验知识支撑，从根源上提升搜索效率与绕过准确性。

实验中，系统从规则库入口符号出发生成攻击语句骨架，再将其精准注入到 Query、Header、JSON、Form-data 等多种常见请求上下文当中，形成规模可控的探测候选池。探测阶段使用最小区分探测集策略，从候选池中选取规模受控的 D ，并基于响应指纹向量 y 聚类形成拦截模式标签 κ ，最终得到映射表 M 的初始估计。该流程基于目标 WAF 的真实行为反馈构建规则边界认知，而非依赖静态规则假设或人工先验，与 WAFFLED 中通过模糊测试发现解析差异实现绕过的核心思路具有高度一致性，同样强调从实际交互数据中挖掘拦截规律，具备更强的黑盒场景适配性。表5-1中给出了语义骨架中规则规模、Token 的规模以及解析率等关键统计结果。

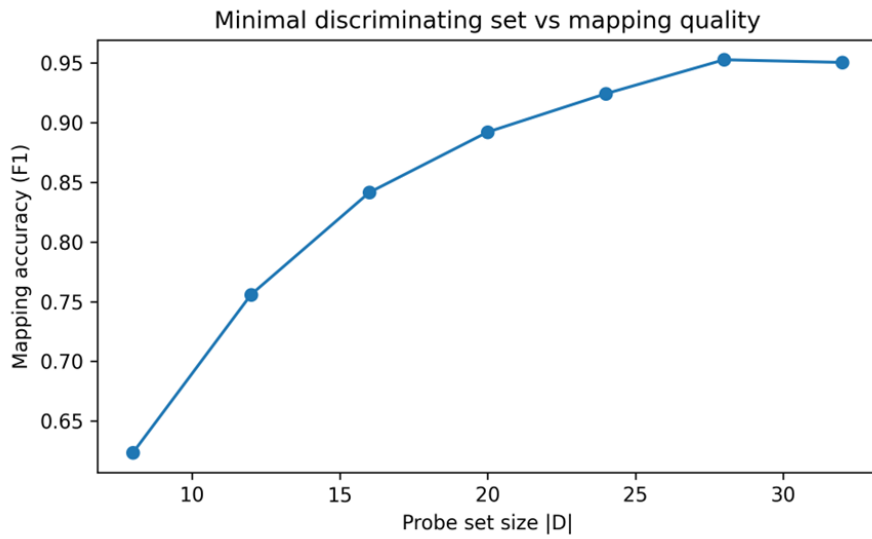
表 5-1 统一规则库有效性
Table5-1 Validity of the unified rule repository

攻击类型	产生式规则数	终结符 Token 数	AST 可解析率	上下文复用率
SQLi	68	41	0.991	0.923
XSS	74	46	0.987	0.908
CMDi	52	35	0.994	0.886

统计结果表明，统一规则库可以在保持高解析成功率的同时，降低多攻击类型规则体系之间的重复定义，使得同一套 AST 操作框架能够复用于多类注入攻击，从而为后续跨攻击类型的统一搜索策略提供基础。统计结果表明，统一规则库针对三类攻击均实现了 98.7% 以上的 AST 可解析率，近乎完全覆盖各类攻击的合法语法变体，同时上下文复用率均保持在 88% 以上，可以在保持高解析成功率的同时，大幅降低多攻击类型规则体系之间的重复定义，有效解决传统分类型规则库冗余度高、复用性差的问题，使得同一套 AST 操作框架能够稳定复用于多类注入攻击。

图5-1给出了探测集规模 $|D|$ 与映射质量（以 F1 衡量）的关系曲线。当 $|D|$ 较小时，映射表对拦截概率的估计受偶然噪声影响较大；随着 $|D|$ 增加，F1 逐渐上升并趋于平稳。该现象说明探测集并非越大越好，规模过大会导致额外查询成本与限速风险积累，而规模过小则无法形成稳定先验。综合考虑成本与收益，本文在后续实验中将 $|D|$ 设置在曲线进入平台段附近，以保证映射信息密度与探测代价之间的平衡。

同时，为了进一步验证探测、映射与剪枝方法的有效性，本节另设置三组消融

图 5-1 探测集规模 $|D|$ 与映射质量 (F1) 关系Fig.5-1 Relationship between detection set size $|D|$ and mapping quality (F1)

实验：移除语义触发判定、移除硬剪枝约束、以及将 Beta-Bernoulli 在线更新替换为单次观测硬判定。表5-2给出了消融对比结果。

表 5-2 规则探测/映射/剪枝链路消融对比 ($B = 2000$)Table5-2 rule detection/mapping/pruning vs. link ablation comparison ($B=2000$)

Variant	Semantic 判定	Hard 剪枝	P_{block} 估计	SR	QC_{first}	T_{rec}	SemanticHit
MCTS+prior	Yes	Yes	Beta-Bernoulli	0.781	310	360	0.860
w/o semantic hit	No	Yes	Beta-Bernoulli	0.842	270	330	0.512
w/o hard pruning	Yes	No	Beta-Bernoulli	0.720	410	520	0.852
w/o Bayesian update	Yes	Yes	Hard decision	0.742	365	460	0.855

5.4 语义保留变异有效性验证

本节验证第三章提出的语义保留节点级变异是否能够在保持攻击意图的同时扩大绕过空间，重点关注可执行性与对抗性之间的协同关系。与传统 fuzzing 中随机扰动扩大覆盖的思路不同，本研究在 CFG 约束下进行 AST 层变异，使生成样本始终可被解析回合法结构，从源头降低无效输入比例，这种结构化生成与搜索调度的思想与约束驱动输入空间探索一致。

实验中，将等价替换、编码变异、注释与空白干扰插入三类算子组合应用于 AST 节点，并在不同 WAF 上执行绕过测试，测试结果如表5-3所示。本节对语义保持能力从两方面验证，其一是语义等价判定通过率，即变异前后语义判定函数一致的比例；其二是变异后可执行率，即载荷在 SUT 上能够触发预期语义效果的比例。语义判定在三类攻击上分别采用不同的可观测性策略，对于 SQLi，以布尔回

显差异与错误回显作为等价依据；对于 XSS，以响应体注入点一致性与页面行为侧证为依据；对于命令执行，以输出回显或延迟侧证作为依据。

表 5-3 语义保留变异有效性
Table5-3 Semantic preservation variation validity

攻击类型	语义等价判定通过率	变异后可执行率	平均变异深度 (节点)	平均 Payload 长度 (字符)
SQLi	0.964	0.972	4.2	138
CMDi	0.948	0.951	4.8	164
XSS	0.956	0.960	3.9	121

结果显示，语义命中率在完整算子配置下保持在较高水平，三类攻击的语义等价通过率与可执行率均超过 94%，且平均变异深度适中、Payload 长度控制在合理区间，既实现有效变形绕过，又未过度冗余导致执行异常。语义命中率在完整算子配置下的高水平表现，说明大多数成功样本不仅通过 WAF，也能在应用侧满足语义触发判定，从而具备真实漏洞检测意义。该结论也与 WAFBOOSTER^[76] 所关注的变异载荷能绕过防护并产生安全影响的问题形成互补，WAFBOOSTER 侧重修补规则缺口，本研究侧重系统化发现可执行绕过样本。

为拆解核心模块的作用效果，排除单一算子主导结果的可能性，保证实验结论严谨可信，并进一步验证语义保持变异机制中各类变异算子对最终载荷质量的独立贡献，本节针对结构化生成模块另设置了三组消融实验：分别移除多层编码变异、除等价替换变异，以及移除空白符与注释干扰变异。表5-4给出了三类变异方式的消融结果。

表 5-4 语义保留变异算子消融对比 ($B = 2000$)
Table5-4 Semantic preservation variation operator ablation comparison (B=2000)

Variant	Operators	SR	SemanticHit
MCTS+prior	Eq+Encode+WS/Comment	0.781	0.860
w/o encoding	Eq+WS/Comment	0.702	0.832
w/o equivalent substitution	Encode+WS/Comment	0.715	0.821
w/o whitespace/comment	Eq+Encode	0.744	0.846

结果显示，移除编码变异后 SR 下降更明显，表明编码分支对绕过字符串匹配类规则具有直接作用；移除等价替换后语义命中率下降更突出，说明结构化替换

更有助于在保持攻击语义的同时拓展可行绕过路径；空白/注释干扰对 *SR* 的影响相对温和，但其低成本扰动仍能提升样本多样性，并在多规则叠加场景下贡献稳定增益。

5.5 WAF 绕过测试

5.5.1 搜索/调度策略方法对比测试

本节评估本文方法在真实 WAF 目标下的绕过发现能力与测试效率。与仅关注最终是否找到绕过样本的评估不同，这里更关注在预算受限的黑盒条件下，不同调度策略在搜索过程中产生高质量候选的能力。为此，实验在相同请求预算 B 下对多种调度策略进行对比，预算配置在每个 WAF $\times$ 攻击类型组合上保持一致；每种方法在同一设置下重复运行 R 次并取均值，以减弱偶然性与噪声干扰。

为了更细粒度地刻画搜索过程中的有效样本密度，本文采用单轮绕过成功率 (Success rate per round) 作为过程指标，在每一轮（或每一批次）变异生成的 payload 中，统计满足有效判定条件的比例。有效判定要求同时满足两项约束，一是请求未被 WAF 拦截 (allow)，二是后端语义触发判定成立 (semantic hit)，以避免仅凭 200/403 等单一信号造成误判。该指标可理解为单次请求命中率或过程成功率，用于衡量每轮输出 payload 的质量与调度策略对有限预算的利用效率。

对比基线覆盖随机、启发式、进化式与学习式策略，并补充无先验树搜索作为对照。Random 在 CFG 生成空间内进行随机采样与变异；Greedy 优先选择历史上放行比例较高的结构分支，强调局部利用；GA 通过选择、交叉与变异推动候选迭代；RL(PPO) 用奖励信号学习动作选择；MCTS(no-prior) 采用标准 MCTS 搜索但不注入第三章映射表先验；MCTS+prior 为本文完整方案，在“探测集 $D \rightarrow$ 响应指纹 $y \rightarrow$ 映射表 M 与风险估计 $\rightarrow$ 剪枝/权重注入 $\rightarrow$ MCTS 自适应搜索”的闭环下进行调度。

表5-5给出了三种 WAF 目标下 SQLi/XSS/CMDi 的单轮绕过成功率对比结果。从总体趋势看，随机策略在有限预算下难以覆盖有效缺口区域；贪心策略虽能更快地趋向低风险路径，但容易陷入局部最优；GA 与 RL(PPO) 在一定程度上提升了有效样本密度，体现出启发式/学习型策略在黑盒测试中的优势。本文方法在引入规则映射表与剪枝先验后进一步提升成功率，而完整方法在多数配置下取得最高绕过发现比例，说明结构化风险先验与树搜索调度的结合能够更有效地在巨大载荷空间中分配有限预算。但是这种 *SR* 往往偏低 (30%-40%)，由于每轮变异生成中大量样本本身就是探索样本，会被拦截，并且在方法中加入了语义保持约束与规则敏感区域，会产生很多必拦路径，因此单轮成功率不会很高。因此，即便方

表 5-5 单轮绕过成功率 (%)
Table5-5 Success rate per round of bypassing (%)

WAF	Attack	Random	Greedy	GA	RL(PPO)	MCTS(no-prior)	MCTS+prior
ModSecurity + CRS	SQLi	18.5	23.4	30.0	34.5	38.3	44.1
	CMDi	13.6	19.3	24.9	28.4	32.0	37.5
	XSSi	15.2	18.3	22.6	25.0	28.0	34.3
Nginx + Lua	SQLi	21.2	24.2	28.9	32.1	36.9	43.0
	CMDi	14.9	19.6	24.1	26.4	29.9	37.0
	XSSi	18.3	21.6	27.9	30.5	33.8	40.5
Cloud WAF-A	SQLi	15.1	18.7	26.1	28.9	33.0	37.6
	CMDi	9.5	14.6	19.4	22.8	26.2	31.9
	XSSi	12.4	19.3	25.2	27.2	32.0	36.8

法有效，过程指标仍可能呈现 30%–40% 的水平，这与黑盒绕过任务的探索属性一致，也为后续引入成本与收敛速度等指标进行综合评价提供了背景。

5.5.2 同类研究方法与工程工具对比测试

在完成搜索/调度策略基线对比后，为增强横向可比性并检验方法在真实攻防语境下的优势，本节进一步引入两类外部基线进行对比测试：同类研究方法基线与工程工具基线。

同类研究方法基线包含两种模板生成器。分别是解析差异模板基线（WAF-FLED 类），围绕 content-type 与字段组织方式构造结构性差异请求骨架；以及协议层/规范化差异规避模板基线（WAFManis 类），围绕 HTTP 归一化与解析顺序构造请求层面的结构变体。工程工具基线选取真实渗透测试中常用的 payload 生成与注入工具，SQLi 使用 sqlmap，XSS 使用 XSSStrike，CMDi 使用 commix，并使用 Burp Suite Intruder 作为通用注入器对照。

表5-6给出了外部基线与本文方法在不同 WAF 与攻击类型下的绕过成功率对比结果。可以观察到，两类同类研究模板基线在部分场景中能够取得中等水平的过程命中率，反映出 parsing discrepancy 与协议/规范化差异在 WAF 规避中的普遍作用；但其性能上限受限于模板集合规模与模板结构的固定性，当面对更大范围的语义等价表达族与多上下文组合时，命中率增长更容易提前饱和。工程工具基线的过程命中率整体偏低且波动更大，尤其在多层编码与上下文差异较强的场景下更易出现放行但语义不成立或语义可成立但被拦截的失配。这一现象说明静态

表 5-6 不同 WAF 与攻击类型下的有效绕过成功率 (SR)
Table5-6 Effective bypass (SR) across different WAFs and attack types

WAF	Attack	Template-PD	Template-Proto	Tools	Burp Intruder	MCTS+prior
ModSecurity + CRS	SQLi	64.0	61.8	sqlmap: 46.7	44.0	77.0
	CMDi	55.4	52.9	commix: 38.6	36.8	65.9
	XSSi	58.5	56.0	XSSStrike: 41.8	38.2	70.5
Nginx + Lua	SQLi	66.2	64.5	sqlmap: 52.3	45.2	80.9
	CMDi	57.0	55.1	commix: 39.1	38.3	69.8
	XSSi	60.1	58.7	XSSStrike: 45.2	40.9	74.2
Cloud WAF-A	SQLi	61.4	59.2	sqlmap: 47.7	41.3	73.9
	CMDi	53.2	50.1	commix: 38.6	31.9	62.2
	XSSi	56.3	54.8	XSSStrike: 42.2	36.1	66.5

payload 集与缺少在线先验更新的注入策略在云 WAF 前置环境下存在明显局限。

相较之下, 本文方法在外部基线对比中仍保持更高的绕过成功率。其原因并不在于依赖单一技巧, 而在于三部分机制形成的闭环: 语法空间中的语义保持变异扩大了可搜索的等价表达族; 映射表 $M(Token_i, Context_j)$ 的在线更新将弱反馈转化为可置信风险先验, 并通过剪枝/权重注入减少无效尝试; MCTS 调度在探索与利用之间动态分配预算, 使搜索能够更快逼近规则边界并保持稳定收敛。因此, 外部基线结果从另一侧验证了本文方法的系统性优势, 而非对特定启发式的偶然收益。

5.5.3 典型绕过样本与规则缺口分析

本节给出企业合作云 WAF 安全测试项目中确认的三条典型绕过实例, 用于展示本文方法在真实生产约束下的可用性与可解释性。三个实例分别覆盖 XSS 绕过与命令注入绕过两类场景, 并满足本文统一的有效样本判定: 请求在 WAF 前置环境下放行 (allow), 且后端语义触发判定成立 (semantic hit), 并在重复次数 R 的复测中保持稳定一致。由于合作项目的合规与保密要求, 本节不披露可直接复现的攻击细节与敏感字段, 仅给出与论文方法相关的结构化证据与机理归因; 完整载荷已在项目交付物中脱敏留存, 用于内部回归测试与修复验证。

XSS 语义保持变形绕过。该实例发生在 $Context_j$ 为脚本相关输出上下文 (如 HTML/JS 上下文边界附近) 的场景中。常规的 XSS 触发片段在目标云 WAF 上能够稳定命中拦截模式簇 κ_{block} 。在本文方法生成的候选中, 某一类载荷在保持脚本执行语义不变的前提下, 通过对执行触发方式与关键 token 组合进行语义保持的结构改写, 使响应指纹 y 从拦截簇转移至放行簇 κ_{allow} 。在放行条件下, 语义触发判

定（基于可控回显/差分响应或安全侧信道）确认浏览器侧可执行语义成立，从而构成有效样本，该行为在 R 次复测中保持稳定。

从 AST 视角，该绕过并非依赖显式危险标签或典型事件属性的直接出现，而是利用了语义保持的表达族，将敏感语义拆分到多个语法单元中，并通过字符串/标识符组合、运行期求值与异常控制流触发等方式完成等价执行。由于 WAF 的规则匹配更依赖静态特征序列（关键字、典型模式），而该类表达在表面 token 分布上更分散，且可能落在规则的边界区域，导致 WAF 的评分/命中不足以触发拦截。映射表 $M(Token_i, Context_j)$ 的在线更新结果显示，与该类分散式触发结构相关的若干 token-context 组合的 $\mathbb{E}[P_{block}]$ 相对较低，软剪枝权重因此未将其完全排除，MCTS 调度在先验引导下对该路径进行了充分探索，从而更快触达缺口。

该类绕过提示规则对非常规触发结构的覆盖不足，建议在不显著提升误报的前提下补充对运行期组合与异常触发类结构的检测，一方面加强对关键语义片段组合关系的识别（而非仅匹配单一危险 token），另一方面对高风险输出点启用更严格的上下文编码策略与 CSP 配置，从根源上降低 XSS 的可执行性。该实例已被最小化并沉淀为回归用例，用于规则升级与策略调整后的对照复测。

命令注入绕过（编码与归一化链路不一致）。该实例对应命令注入场景，注入上下文 $Context_j$ 位于 URL 查询参数或表单字段。常规命令分隔与敏感命令模式在目标云 WAF 上能够稳定触发拦截簇 κ_{block} 。在本文方法生成的候选中，某一类载荷在保持命令拼接/执行语义成立的条件下，通过多层编码与等价变换改变了表面特征，使 WAF 放行并进入 κ_{allow} 。在放行条件下，基于可控副作用的安全验证信号，如无害回显标记或可控延迟特征的语义触发判定确认后端命令语义被执行，从而构成有效样本。多次复测表明该现象稳定，且与限速/挑战等非规则因素无关。

该绕过的关键在于解码/归一化顺序敏感，载荷通过编码链封装使关键结构在 WAF 的归一化视图中未触发相应规则，但在后端或业务组件的解码链路中被恢复为可解释的命令结构。该机理与第三章的多层级编码变异直接对应，在 AST 上可视为编码器节点的插入与组合，属于语义保持的结构包裹。映射表 $M(Token_i, Context_j)$ 的在线更新结果显示，某些与编码器组合相关的 token-context 对在放行样本中频繁出现，其 $\mathbb{E}[P_{block}]$ 下降，提示该区域为规则覆盖薄弱点。在第四章中，这一先验通过权重注入影响 UCT' 的选择倾向，使搜索更少浪费在高置信必拦的明文分隔结构上，而更多探索归一化边界附近的路径。

该类缺口的修复重点是归一化一致性，明确并统一 WAF 与后端对输入的解码层数、解码顺序与规范化策略，避免出现 WAF 只解一层但后端解多层或解码顺序不同的情形。同时对高风险参数启用更严格的归一化与白名单策略，并将该类编码组合纳入规则回归集。项目中已将该样本脱敏最小化，作为回归测试用例固定

在 CI 安全回归流程中。

命令注入绕过（低可见符号组合与参数拆分导致规则失配）。第三个实例同样属于命令注入，但其绕过不依赖单一路径的多层编码，而依赖在载荷结构层面降低规则可见性，攻击语义通过符号级等价变换与参数拆分组合被分散到多个片段中，并通过执行链路在后端重新组合，从而绕过基于固定特征序列的规则匹配。该样本在 WAF 侧呈现放行簇 κ_{allow} ，并在语义触发判定中成立。与实例二相比，该实例在不同 $Context_j$ 下的可迁移性更强，表明其绕过依赖于规则表达能力不足而非单点配置差异。

该绕过体现了组合关系触发的困难，WAF 规则往往对显式分隔符、危险命令关键字与典型拼接模式敏感，但对拆分后再组合的表达族覆盖不足。就本文框架而言，这类样本更依赖第三章的等价替换与结构重排类变异，在 AST 上表现为将关键动作拆分为若干低强度片段，并通过上下文合法的组合方式构成后端可解释语义。映射表 M 以 token-context 为粒度的风险估计在此类场景仍能提供方向性帮助，当单点风险不高但组合后语义成立时，软剪枝不会将其一刀切删除，而是将其作为低优先级候选保留。MCTS 在探索与利用平衡下能够逐步积累对此类组合路径的经验回报，从而提升发现概率。该结果也从侧面说明，将回报定义为 “allow $\times$ semantic hit” 是必要的，仅追求放行会把大量无语义样本计入成功，而语义触发约束能够把预算集中到具有安全意义的缺口上。

针对该类绕过，修复建议侧重于增强规则对跨片段组合关系的表达能力，在不引入显著误报的前提下补充对拆分组合模式的检测，或在关键接口上以业务白名单约束替代通用规则匹配。项目中已将该样本抽象为结构模式回归用例，用于评估规则升级后对组合型绕过的覆盖能力，并作为持续安全运营的回归资产沉淀。

三个实例覆盖了解析/归一化链路差异、编码链顺序敏感以及组合关系导致的规则失配等典型缺口类型，能够从工程侧支撑本文提出的变异、注入、反馈与更新的闭环。与仅输出绕过成功率的统计结论不同，本文方法能够提供与实例对应的结构证据链：变异路径 π 、注入上下文 $Context_j$ 、响应指纹 y 与模式簇 κ 、以及映射表 $M(Token_i, Context_j)$ 的风险变化，从而使绕过结果可解释、可复现，并可转化为规则修复建议与回归测试集，符合企业合作项目发现、修复及回归的落地需求。

5.6 调度策略性能对比分析

5.6.1 收敛速度对比

除成功率外，测试效率对黑盒 WAF 评估尤为关键。图5-2给出了在固定预算 $B = 2000$ 下，不同方法的有效绕过成功率随查询次数的变化曲线。随机策略的成

功率上升缓慢且波动明显，贪心策略在早期能获得一定提升，但较容易陷入局部最优并在后期趋于停滞；GA 与 RL 能逐步改善探索质量，但在高维组合空间中仍存在收敛速度不足的问题。

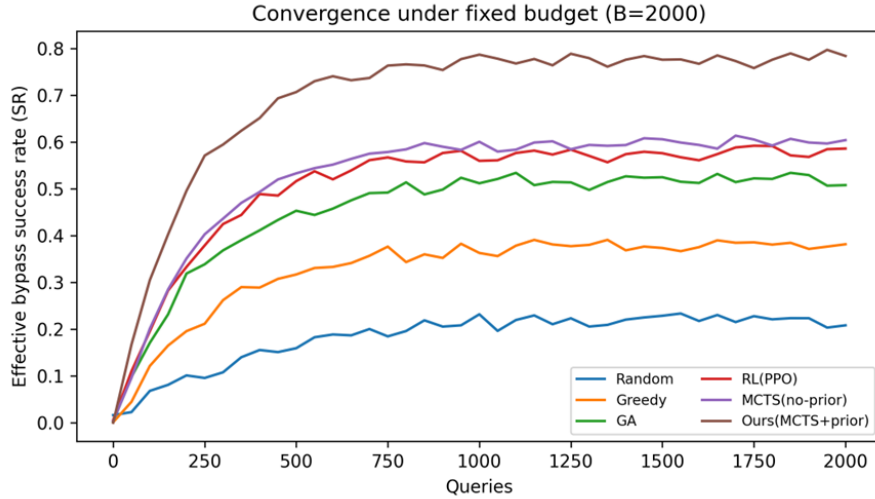


图 5-2 不同方法在固定预算下的收敛曲线 ($B = 2000$)

Fig.5-2 Convergence curves of different methods under a fixed budget ($B=2000$)

本文提出的方案的曲线表现出更明显的快启动与稳收敛的特征，早期受益于映射表先验与剪枝机制，避免在高风险必拦分支上消耗预算，中后期通过 MCTS 的 UCT' 机制持续平衡探索与利用，使搜索能够在多个潜在绕过结构之间动态分配查询资源。类似基于树搜索的调度提升收敛效率的结论也在 Alphuzz 的 fuzzing 调度研究中得到体现。

5.6.2 查询成本与早期有效性

除核心有效性验证外，本节进一步对比不同方法在查询成本以及早期有效性方面的差异，该项对比核心用于衡量模型获取首个可成功绕过 WAF 且语义有效 Payload 的效率，直观反映搜索策略的收敛速度与成本优化能力，是评估黑盒检测方法实用性的关键指标，图5-3展示了不同方法在 QC_{first} 指标上的分布。

从图中可以观察到，本文提出的方案的箱体整体分布更低，它的 QC_{first} 数值整体更小，即成功更早、成本更低。说明在多数重复实验中能以更少查询获得首次有效成功样本。且方法稳定性显著优于对照组，适配云 WAF 动态限速与查询预算受限的实际场景。该结果与 3.5 节的理论预期一致，硬剪枝减少了明显无效路径，软剪枝权重通过 MCTS 选择策略转化为搜索偏置，使预算更快集中到可能放行且语义仍可触发的路径区域。

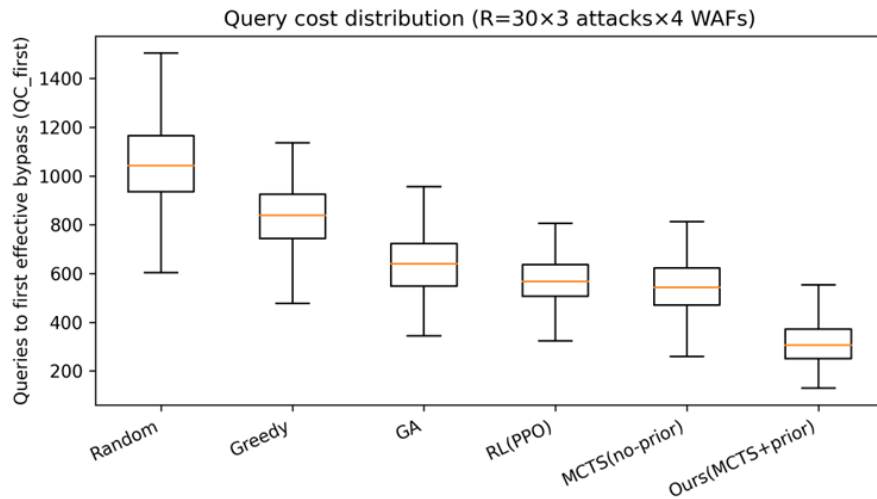


图 5-3 首次有效成功查询成本分布

Fig.5-3 First successful query of cost distribution

5.6.3 概念漂移场景下的恢复能力

云 WAF 具有持续更新规则与策略的特点，导致映射表 M 的历史估计随时间衰减。图5-4给出了模拟规则更新后，各方法的 SR 恢复曲线。

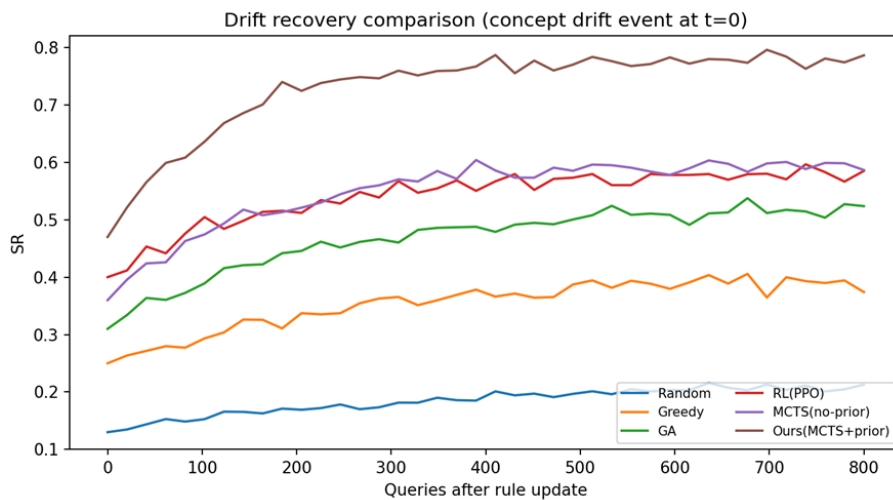


图 5-4 概念漂移场景下的恢复能力对比

Fig.5-4 Comparison of recovery capability in concept drift scenarios

本文提出的方案在漂移发生后能够更快回到稳定水平，体现出 4.3.3 节提出的漂移触发重探测与子树衰减机制能够有效修正历史偏置。相比之下，缺少在线更新机制的方法恢复较慢，说明仅依靠静态策略难以应对云环境的动态性。WAFFLED 在真实云 WAF 上发现大量可复现绕过点，同时也强调解析行为差异与规则变化对安全性的影响，本节的漂移恢复实验为这一现象提供了进一步证明。

5.6.4 模块整体性能统计

对整体模块的性能进行统计，主要使用 SR 、 QC_{first} 、 T_{rec} 、 $SemanticHit$ 四个指标来进行性能测试。表5-7汇总了各方法在四个指标上的均值与标准差。

表 5-7 不同方法总体性能统计
Table5-7 Overall performance statistics for different methods

Method	SR	QC_{first}	T_{rec}	$SemanticHit$
GA	0.496 ± 0.070	641 ± 124	489 ± 78	0.674 ± 0.067
Greedy	0.360 ± 0.070	838 ± 140	630 ± 94	0.581 ± 0.055
MCTS(no-prior)	0.580 ± 0.071	549 ± 113	405 ± 73	0.722 ± 0.059
MCTS+prior	0.761 ± 0.071	314 ± 89	236 ± 63	0.839 ± 0.055
RL(PPO)	0.559 ± 0.069	569 ± 109	429 ± 76	0.710 ± 0.063
Random	0.199 ± 0.068	1052 ± 176	787 ± 112	0.480 ± 0.058

结果表明，本文提出的方案在成功率与收敛效率上均取得优势，并保持较高语义命中率，说明提升并非来自放行样本数量增加，而是来自有效可执行绕过的比例提升。该结果与 WAFBOOSTER 所揭示的现实问题形成呼应，WAF 在面对变异载荷时存在检出缺口，而更高质量的载荷生成与搜索调度能够系统化暴露这些薄弱环节。

5.6.5 参数敏感性分析

本研究在 UCT' 中引入探索系数 c 与风险先验融合系数 β 。图5-5，图5-6，图5-7分别展示了不同参数配置对 SR 、 QC_{first} 、 T_{rec} 的影响趋势。

结果显示， c 过小会导致搜索过度利用已知路径，难以跳出局部最优； c 过大则会带来探索开销上升，导致查询成本增加。参数 β 过低时映射表先验作用不足，MCTS 会在高风险分支上消耗更多预算； β 过高时又可能产生过强的风险回避，降低对边界缺口的探索强度。表5-8给出了基于综合指标 $Score$ 筛选下的综合指标最优组合，可作为工程复现时的默认取值参考。

表 5-8 UCT' 超参数最优组合
Table5-8 UCT' Optimal Hyperparameter Combination

c	β	SR	QC_{first} / T_{rec}	$Score$
0.8	0.6	0.791	236 / 291	0.6879

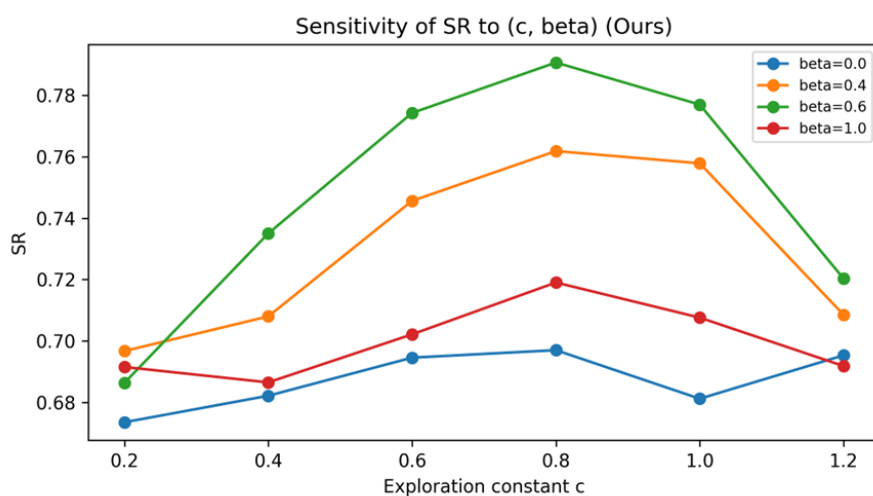


图 5-5 超参数 (c, β) 对 SR 的敏感性分析
Fig.5-5 Sensitivity Analysis of SR to Hyperparameters (c, β)

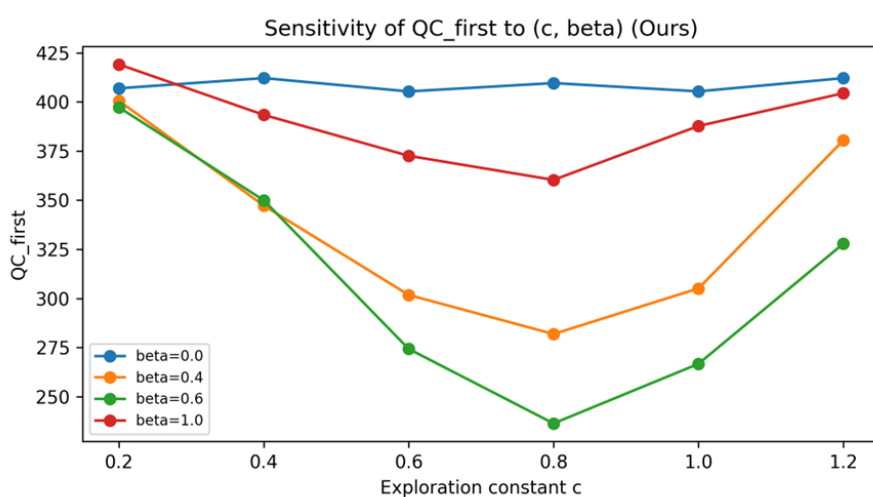
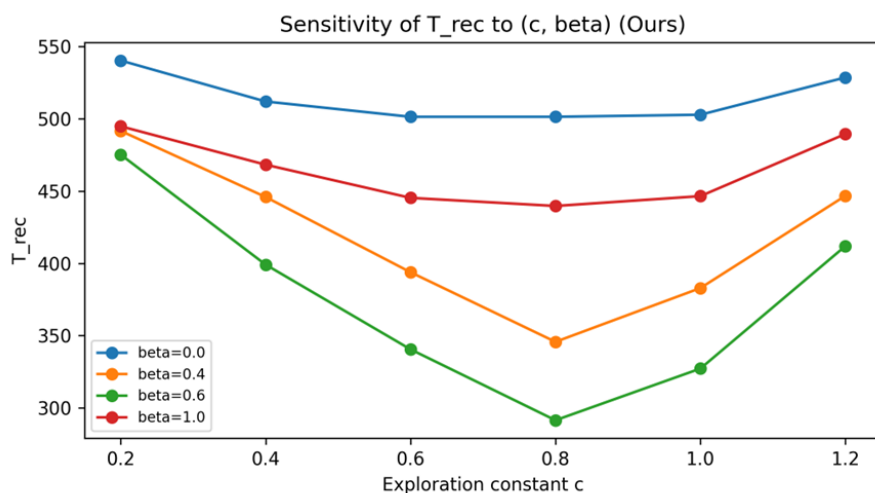


图 5-6 超参数 (c, β) 对 QC_{first} 的敏感性分析
Fig.5-6 Sensitivity Analysis of Hyperparameters (c, β) on QC_{first}

图 5-7 超参数 (c, β) 对 T_{rec} 的敏感性分析Fig.5-7 Sensitivity Analysis of Hyperparameters (c, β) on T_{rec}

5.7 本章小结

本章从统一规则库、语义保留变异、WAF 绕过能力与搜索效率四个层面完成实验验证，并通过消融实验明确了关键模块的独立贡献。结果表明，探测集 D 与多信号指纹机制能够以较低成本建立可靠映射先验；在线贝叶斯更新与剪枝策略能够显著降低无效查询并提升早期有效性；在此基础上，融合风险先验的 MCTS 搜索能够进一步提升收敛效率并增强漂移恢复能力。上述实验结果与第三、四章的设计目标一致，形成了语义空间构建、先验剪枝、优化搜索、在线更新的完整验证方法。

第 6 章 工作总结和展望

6.1 工作总结

WAF 安全测试在黑盒条件下面临着诸多严峻挑战，本文针对传统黑盒测试在指数级载荷空间中极易耗尽预算、云 WAF 复杂机制引入大量反馈噪声，以及动态规则导致评估环境非平稳等痛点问题，提出了一种基于语义保持变异与启发式搜索优化的 WAF 漏洞自动化检测方法，构建了结构化载荷生成—先验获取与剪枝—反馈驱动调度的完整技术闭环。全文的核心工作与贡献主要体现在以下三个层面：

在载荷空间构建层面，本文提出面向多攻击类型，包括 SQLi、XSS 与 CMDi 的统一语法化构建机制。利用上下文无关文法将攻击语义结构化，并将载荷解析为抽象语法树，使变异操作从无序的字符级扰动，转化为节点级的等价替换、多层编码包裹与干扰插入。该机制不仅确保了对抗样本的语法合法性与语义一致性，更将经验式的变异升级为可解析、可回放且具备结构解释力的系统化过程。

在先验建模与空间压缩层面，针对弱反馈环境设计了探测—映射—剪枝机制。本文通过构建最小区分探测集与多信号响应指纹，有效规避了单一状态码带来的测量噪声。基于探测结果，引入 Beta-Bernoulli 在线更新机制，建立了高置信度的 Token-Context 拦截概率映射表。据此，系统通过硬剪枝直接剔除高置信度的必拦区域，通过软剪枝以权重形式延后对潜在缺口的探索。依托精准先验知识压缩无效搜索空间，大幅降低冗余查询，适配云 WAF 限速与预算受限的实际场景。该机制在不牺牲覆盖率的前提下大幅压缩了搜索空间，并为后续搜索注入了可靠的先验偏置。

在搜索调度与动态自适应层面，本文将变异路径选择建模为序列决策问题，提出融合风险先验与经验回报的蒙特卡洛树搜索（MCTS）优化框架，引导测试预算在探索与利用间达成严谨的统计平衡。为应对云 WAF 拦截行为随时间漂移的非平稳问题，进一步引入了漂移触发重探测与子树统计衰减机制，实现对失效统计信息的动态校准与回收。全面的系统评估与消融实验表明，本方法能够在有限预算下显著提升收敛效率，稳定定位 WAF 防护缺口，形成了从理论模型到实验验证的完整闭环。

6.2 展望

后续研究可从以下两个方向进一步深化。其一，面向极端复杂业务逻辑的深度语义验证扩展。当前系统的执行反馈主要依托于回显、延迟及轻量级侧信道，在主

流 Web 场景下能够实现高效判定。然而，在面对存在强网络隔离、异步处理或无回显，如盲注等极端对抗场景的复杂业务系统时，如何在不扩大安全影响的前提下，结合轻量级程序分析探索更为隐蔽、鲁棒的安全语义判定器，是进一步拓展该方法适用边界的重要方向。其二，面向持续安全运营的自动化闭环赋能。当前的自适应调度机制已能有效应对云 WAF 的非平稳漂移问题。为了进一步推动本研究在企业安全体系中的工程落地，后续可重点探索如何将高价值的绕过样本与路径图，自动化转化为可操作的 WAF 防御规则修复补丁与回归测试用例集。通过打通从漏洞发现、评估到规则加固、防护的全链路，从而为现代 Web 应用的安全运营提供更具实战价值的自动化闭环支撑。

参考文献

- [1] OWASP Foundation. Owasp top 10-2021: the ten most critical web application security risks[EB/OL]. 2021-09[2026-01-29]. <https://owasp.org/www-project-top-ten/>.
- [2] MOISEI L, MOCANU L. Mitigating owasp api security top 5 risks through api gateway patterns[C]. International Conference on Machine Intelligence & Security for Smart Cities (TRUST) Proceedings, Bucharest, Romania, 2025: 29-36.
- [3] LOOSE N, MÄCHTLE F, POTT C, et al. Madvex: instrumentation-based adversarial attacks on machine learning malware detection[C]. International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Hamburg, Germany, 2023: 69-88.
- [4] SCRAVAGLIERI L, POPOV M, PILLA L L, et al. Optimizing performance and energy across problem sizes through a search space exploration and machine learning[J]. Journal of Parallel and Distributed Computing, 2023, 180: 104720.
- [5] AKHAVANI S A, JABIYEV B, KALLUS B, et al. Waffled: exploiting parsing discrepancies to bypass web application firewalls[J]. Arxiv Preprint Arxiv:2503.10846, 2025.
- [6] DEMETRIO L, BIGGIO B, LOTTI L, et al. Waf-a-mole: evading web application firewalls through adversarial machine learning[C]. Proceedings of the 36th Annual Computer Security Applications Conference (ACSAC), Austin, USA, 2020: 1745-1752.
- [7] NAGENDRAN K, BALAJI S, RAJ B A, et al. Web application firewall evasion techniques[C]. 2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2020:1764-1769.
- [8] 张若愚, 黄方怡, 张志勇, 等.Web 应用防火墙识别, 绕过与加固技术的综述 [J]. 通信技术, 2024, 57(10):1000-1005.
- [9] 龙安康, 罗云.Web 应用防火墙 (WAF) 技术演进与发展趋势 [J]. 中国信息界, 2024(3):26-28.
- [10] SIKORSKYI R, HARASYMCHUK O. A multi-stage model for detecting and preventing sql injections in the xampp/mysql environment[J]. CEUR Workshop Proceedings, 2025, 11(2): 3-7.
- [11] OWASP Foundation. Modsecurity handbook (v3)[EB/OL]. [2026-01-29]. [https://github.com/SpiderLabs/ModSecurity/wiki/Reference-Manual-\(v3.x\)](https://github.com/SpiderLabs/ModSecurity/wiki/Reference-Manual-(v3.x)).
- [12] OWASP Foundation. Owasp core rule set (crs) documentation[EB/OL]. [2026-01-29]. <https://coreruleset.org/>.

- [13] ANUVARSHINI M K, BALA K S S, SONTI S S T, et al. An empirical study on the evaluation and enhancement of owasp crs (core rule set) in modsecurity[J]. Computers & Security, 2025: 104714.
- [14] SATO D M V, DE FREITAS S C, BARDDAL J P, et al. A survey on concept drift in process mining[J]. ACM Computing Surveys (CSUR), 2021, 54(9): 1-38.
- [15] BROWNE C B, POWLEY E, WHITEHOUSE D, et al. A survey of monte carlo tree search methods[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(1):1-43.
- [16] ŚWIECHOWSKI M, GODLEWSKI K, SAWICKI B, et al. Monte carlo tree search: a review of recent modifications and applications[J]. Artificial Intelligence Review, 2023, 56(3): 2497-2562.
- [17] LUCZ G, FORSTNER B. A thirty-day dataset of malicious http requests blocked by owasp modsecurity on a production web server[J]. Data, 2025, 10(11): 186.
- [18] APPLEBAUM S, GABER T, AHMED A. Signature-based and machine-learning-based web application firewalls: a short survey[J]. Procedia Computer Science, 2021, 189: 359-367.
- [19] AL-ALI G M, AL-EMADI K A, AL-NAIMI R A, et al. Hybrid web application firewall using rule-based and feature fusion in convolutional neural network[C]. 2025 16th Student Research Conference on Applied Computing (SRC), Dubai, UAE, 2025: 1-6.
- [20] PHAM V H, CHUONG N P, THAI P T, et al. A coverage-guided fuzzing method for automatic software vulnerability detection using reinforcement learning-enabled multi-level input mutation[J]. IEEE Access, 2024, 12: 129064-129080.
- [21] MANÈS V J M, HAN H S, HAN C, et al. The art, science, and engineering of fuzzing: a survey[J]. IEEE Transactions on Software Engineering, 2019, 47(11): 2312-2331.
- [22] DHARMAADI I P A, ATHANASOPOULOS E, TURKMEN F. Fuzzing frameworks for server-side web applications: a survey[J]. International Journal of Information Security, 2025, 24(2): 73.
- [23] LIN S. Hybrid fuzzing with llm-guided input mutation and semantic feedback[C]. 2025 Cyber Awareness and Research Symposium (CARS), San Antonio, USA, 2025: 1-9.
- [24] XIA C S, PALTENGHI M, TIAN J, et al. Fuzz4all: universal fuzzing with large language models[C]. 46th International Conference on Software Engineering (ICSE), Lisbon, Portugal, 2024. DOI:10.1145/3597503.3639121.
- [25] ZHAO Y, WANG X, ZHAO L, et al. Alphuzz: monte carlo search on seed-mutation tree for coverage-guided fuzzing[C]. Proceedings of the 38th Annual Computer Security Applications Conference, Arizona, USA, 2022: 534-547.

-
- [26] PHAN T D, DUC LUONG T, HOANG QUOC AN N, et al. Leveraging reinforcement learning and generative adversarial networks to craft mutants of windows malware against black-box malware detectors[C]. Proceedings of the 11th International Symposium on Information and Communication Technology, Hanoi, Vietnam, 2022: 31-38.
- [27] STATON S, STEIN D, YANG H, et al. The beta-bernoulli process and algebraic effects[J]. Arxiv Preprint Arxiv:1802.09598, 2018.
- [28] 陆鹏程, 钟晓峰, 陈杰, 等.” 衔尾蛇”: 基于符号增强网络与深度强化学习的自动化 WAF 安全测试框架 [J]. 信息对抗技术, 2025, 4(5):66-76.
- [29] WANG Q, CHEN J, JIANG Z, et al. Break the wall from bottom: automated discovery of protocol-level evasion vulnerabilities in web application firewalls[C]. 2024 IEEE Symposium on Security and Privacy (SP), San Francisco, USA, 2024: 185-202.
- [30] ANDORNO P. Research, testing, and mitigation solutions for web application firewalls evasion techniques[D]. Turin: Politecnico di Torino, 2024: 1-52.
- [31] GRANDE F. Security automation for web-based attacks[D]. Turin: Politecnico di Torino, 2024:1-47.
- [32] ZAKI A, MOHAMMED S. Artificial intelligence for web application firewall (WAF): a comprehensive review[J]. International Research Journal of Innovations in Engineering and Technology, 2024, 8(11): 219-224.
- [33] ABDULLAYEV V, CHAUHAN A S. Sql injection attack: quick view[J]. Mesopotamian Journal of Cybersecurity, 2023: 30-34.
- [34] PAUL A, SHARMA V, OLUKOYA O. Sql injection attack: detection, prioritization & prevention[J]. Journal of Information Security and Applications, 2024, 85: 103871.
- [35] 李德印, 凌祥, 吴春明. 一种 Web 应用防火墙 SQL 注入防御正则规则提取方法和装置: CN202310771061.7[P]. 2023-08-11.
- [36] NEUPANE S. Detecting and mitigating sql injection vulnerabilities in web applications[J]. Arxiv Preprint Arxiv:2506.17245, 2025.
- [37] NASEREDDIN M, ALKHAMAISEH A, QASAMEH M, et al. A systematic review of detection and prevention techniques of sql injection attacks[J]. Information Security Journal: A Global Perspective, 2023, 32(4): 252-265.
- [38] 张敏. 基于深度学习的工业互联网 XSS 漏洞自动化检测技术研究 [J]. 无线互联科技, 2025, 22(09):105-108.
- [39] 王亚东. 跨站脚本 (XSS) 攻击与防护方法研究 [J]. 计算机科学与应用, 2021, 11(1):12.DOI:10.12677/CSA.2021.111020.
- [40] KAUR J, GARG U, BATHLA G. Detection of cross-site scripting (XSS) attacks using machine learning techniques: a review[J]. Artificial Intelligence Review, 2023, 56(11): 12725-12769.

- [41] SAMO A, HALEPOTO M A, AWAN K, et al. An in-depth analysis of cross-site scripting (XSS): threats, mechanisms, and mitigation strategies[J]. Organized by: The Benazir Bhutto Shaheed University of Technology and Skill Development, Khairpur Mirs, 2024, 2024(2nd): 122.
- [42] WANG Q, YANG H, WU G, et al. Black-box adversarial attacks on xss attack detection model[J]. Computers & Security, 2022, 113: 102554.
- [43] ALAHMAD M, ALKANDARI A, ALAWADHI N. Survey of os command injection web application vulnerability attack[J]. Journal of Engineering Science and Technology, 2022, 17(1): 1-5.
- [44] WANG X, ZHAI J, YANG H. Detecting command injection attacks in web applications based on novel deep learning methods[J]. Scientific Reports, 2024, 14(1): 25487.
- [45] CHOMSKY N. Topics in the theory of generative grammar[M]. Berlin: Walter de Gruyter, 1978.
- [46] KALLMEYER L. Parsing beyond context-free grammars[M]. Dordrecht: Springer Science & Business Media, 2010.
- [47] XIE J, ZHANG B, FENG C, et al. Eam: a vulnerability exploitability analysis model based on program variable semantics[C]. 2024 6th International Conference on Communications, Information System and Computer Engineering (CISCE), Shanghai, China, 2024: 1059-1067.
- [48] SRIVASTAVA P, PAYER M. Gramatron: effective grammar-aware fuzzing[C]. Proceedings Of The 30th Acm Sigsoft International Symposium On Software Testing And Analysis, New York, USA, 2021: 244-256.
- [49] 曲振青. 云环境下的 WAF 自动绕过方法研究 [D]. 杭州: 浙江大学, 2023. DOI:10.27461/d.cnki.gzjdx. 2023.000107.
- [50] MCCracken D D, Reilly E D. Backus-NAUR form (BNF)[M]. New York: Encyclopedia Of Computer Science. 2003: 129-131.
- [51] MALVAI H, HUSSAIN S, NEVEN G, et al. Practical proofs of parsing for context-free grammars[J]. Cryptology ePrint Archive, 2024: 562.
- [52] JABIYEV B, GAVAZZI A, ONARLIOGLU K, et al. Gudifu: guided differential fuzzing for http request parsing discrepancies[C]. Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses, Athens, Greece, 2024: 235-247.
- [53] SUN W, FANG C, MIAO Y, et al. Abstract syntax tree for programming language understanding and representation: how far are we?[J]. Arxiv Preprint Arxiv:2312.00413, 2023.
- [54] ALOTAIBI F M, VASSILAKIS V G. Toward an sdn-based web application firewall: defending against sql injection attacks[J]. Future Internet, 2023, 15(5): 170.

-
- [55] KOCSIS L, SZEPESVÁRI C. Bandit based monte-carlo planning[C]. European Conference on Machine Learning (ECML), Berlin, Germany, 2006: 282–293.
- [56] LATTIMORE T, SZEPESVÁRI C. Bandit algorithms[M]. Cambridge: Cambridge University Press, 2020: 1–416.
- [57] AUER P, ORTNER R. Ucb revisited: improved regret bounds for the stochastic multi-armed bandit problem[J]. Periodica Mathematica Hungarica, 2010, 61(1-2): 55-65.
- [58] LIU Y C, TSURUOKA Y. Modification of improved upper confidence bounds for regulating exploration in monte-carlo tree search[J]. Theoretical Computer Science, 2016, 644: 92-105.
- [59] JABIYEV B, SPRECHER S, ONARLIOGLU K, et al. T-reqs: http request smuggling with differential fuzzing[C]. Proceedings of the 2021 Acm Sigsac Conference on Computer and Communications Security, Seoul, Korea, 2021: 1805-1820.
- [60] QAMAR U, RAZA M S. Lexical analysis and parsing using deep learning[J]. Applied Text Mining, 2024: 401-447.
- [61] REN P, XIAO Y, CHANG X, et al. A survey of deep active learning[J]. Acm Computing Surveys (CSUR), 2021, 54(9): 1-40.
- [62] GÜLER E, SCHUMILO S, SCHLOEGEL M, et al. Atropos: effective fuzzing of web applications for {server-side} vulnerabilities[C]. 33rd Usenix Security Symposium (USENIX Security 24), Philadelphia, USA, 2024: 4765-4782.
- [63] 武智广. WAF 在保护 Web 应用免受 DDoS 攻击中的策略与实践 [J]. 中国战略新兴产业, 2025(20):74-77.
- [64] XU L, DING X, PENG H, et al. Adtcd: an adaptive anomaly detection approach toward concept drift in iot[J]. IEEE Internet Of Things Journal, 2023, 10(18): 15931-15942.
- [65] LIU W, LI X, GU Y, et al. an adaptive hoeffding tree model based on differential entropy and relative entropy for concept drift detection[C]. 2024 International Joint Conference on Neural Networks (IJCNN), Yokohama, Japan, 2024: 1-8.
- [66] ARDEBILI Y N. Scalable methods for monte carlo tree search[D]. Detroit: Wayne State University, 2025:11-55.
- [67] WANG Y, JIA P, LIU L, et al. A systematic review of fuzzing based on machine learning techniques[J]. PloS one, 2020, 15(8): e0237749.
- [68] ŚWIECHOWSKI M, GODLEWSKI K, SAWICKI B, et al. Monte carlo tree search: a review of recent modifications and applications[J]. Artificial Intelligence Review, 2023, 56(3): 2497-2562.
- [69] GULCAN E B, CAN F. Unsupervised concept drift detection for multi-label data streams[J]. Artificial Intelligence Review, 2023, 56(3): 2401-2434.

- [70] LIU A, LU J, SONG Y, et al. Concept drift detection delay index[J]. IEEE Transactions on Knowledge and Data Engineering, 2022, 35(5): 4585-4597.
- [71] XU J, YUAN M, MA Y. Feature selection using self-information and entropy-based uncertainty measure for fuzzy neighborhood rough set[J]. Complex & Intelligent Systems, 2022, 8(1): 287-305.
- [72] XU W, YUAN K, LI W, et al. An emerging fuzzy feature selection method using composite entropy-based uncertainty measure and data distribution[J]. IEEE Transactions on Emerging Topics in Computational Intelligence, 2022, 7(1): 76-88.
- [73] HINDER F, VAQUET V, HAMMER B. One or two things we know about concept drift-a survey on monitoring in evolving environments. Part A: detecting concept drift[J]. Frontiers in Artificial Intelligence, 2024, 7: 1330257.
- [74] HINDER F, VAQUET V, HAMMER B. One or two things we know about concept drift-a survey on monitoring in evolving environments. Part B: locating and explaining concept drift[J]. Frontiers in Artificial Intelligence, 2024, 7: 1330258.
- [75] SUN Y, ZHANG X, ZHONG Q, et al. Suifs: a symmetric uncertainty based interactive feature selection method[C]. International Symposium on Bioinformatics Research and Applications, Singapore: Springer Nature Singapore, 2025: 60-71.
- [76] WU C, CHEN J, ZHU S, et al. Wafbooster: automatic boosting of waf security against mutated malicious payloads[J]. IEEE Transactions on Dependable and Secure Computing, 2025.DOI:10.1109/TDSC.2024.3429271.
- [77] AKSOY A, VALLE L, KAR G. Automated network incident identification through genetic algorithm-driven feature selection[J]. Electronics, 2024, 13(2): 293.
- [78] ARIF R M, ASLAM M, AL-OTAIBI S, et al. A deep reinforcement learning framework to evade black-box machine learning based IoT malware detectors using GAN-generated influential features[J]. IEEE Access, 2023, 11: 133717-133729.

作者简介

1. 攻读学位期间的研究成果

1.1 申请（授权）专利

[1] 第 2 作者（导师第 1 作者）. 授权日期.

1.2 参与的科研项目及获奖

[1] 网络安全学院学生创新资助计划项目, 华为云 WAF 安全测试, 2024.05-2025.03, 项目负责人.

[2] 重庆邮电大学硕士研究生三等学业奖学金. 2024

[3] 重庆邮电大学硕士研究生二等学业奖学金. 2025

[4] 第四届全国大学生算法挑战赛 Python 组全国优秀奖. 2024

[5] 第五届全国大学生计算机能力挑战赛 Python 组西南赛区二等奖. 2024

致 谢